Apuntes de Python para el examen

Todo el material en un solo documento. Usa **Ctrl+F** para buscar una palabra (p. ej. `enumerate`, `match`, `lambda`, `__str__`). La chuleta resumida está aparte en `06_chuleta_y_kit_examen/chuleta_python.pdf`.

Índice

- **1-3 · Entorno, operadores y condicionales**
  - 00_entorno_comandos.md
  - 00_INDICE.md
  - 01_entorno_hola_mundo.py
  - 02_operadores.py
  - 03_cond_nota_if_elif_else.py
  - 04_cond_dias_match_case.py
  - 05_cond_paquetes_match_guardas.py
  - 06_cond_temperatura_encadenada.py
  - 07_cond_raiz_guard_clause.py
  - 08_repo_nota_match_guardas.py
  - 09_repo_temperatura_estados.py
  - 10_repo_saneamiento_entrada.py
  - 11_repo_comparacion_lexicografica.py
  - 12_repo_http_match_agrupado.py
  - 99_resumen_formas_exigidas.py
- **4 · Bucles**
  - 00_INDICE.md
  - 01_total_carrito_sin_sum.py
  - 02_contar_vocales.py
  - 03_tabla_del_8_range.py
  - 04_cuenta_atras_pares.py
  - 05_tareas_enumerate.py
  - 06_combinaciones_anidados.py
  - 07_cubos_comprension.py
  - 08_filtrar_palabras_comprension.py
  - 09_contrasena_while.py
  - 10_contador_acumulador_while.py
  - 11_login_cajero_while.py
  - 12_calculadora_while_true_break.py
  - 13_saltar_multiplos_continue.py
  - 14_buscar_99_for_else.py
  - 15_inventario_dict_items.py
  - 16_resumen_bucles.py
- **5 · Funciones**
  - 00_INDICE.md
  - 01_primera_funcion.py
  - 02_parametros_simples.py
  - 03_retornar_valor.py
  - 04_mayor_de_edad.py
  - 05_contar_vocales.py
  - 06_parametros_por_defecto.py
  - 07_retornar_multiples_valores.py
  - 08_filtrar_pares.py
  - 09_promedio_args.py
  - 10a_kwargs_perfil_diccionario.py
  - 10b_kwargs_perfil_imprimir.py
  - 11_type_hints_celsius.py
  - 12_decorador.py
  - 13_map_lambda.py
  - 14_orden_superior.py
  - 15_recursividad_factorial.py
  - 16_resumen_funciones.py
- **6 · Clases, listas de objetos, diccionarios, comprensiones y lambda**
  - 00_INDICE.md
  - 01_teletubbies.py
  - 02_teletubbies_ampliado.py
  - 03_clases_basico.py
  - 04_encapsulamiento_getters_setters.py
  - 05_herencia_super.py
  - 06_listas_de_objetos.py
  - 07_diccionarios.py
  - 08_comprensiones_y_lambda.py
  - 09_resumen_poo.py
- **Variantes tipo examen (todo mezclado)**
  - 00_INDICE.md
  - 01_tamagotchi_mascotas.py
  - 02_biblioteca_prestamos.py
  - 03_tienda_carrito.py
  - 04_gestor_alumnos_notas.py
  - 05_cuenta_bancaria.py
  - 06_videojuego_herencia.py

- **Kit de examen: plantillas, errores y checklist**
  - 00_INDICE.md
  - checklist_examen.md
  - errores_y_traceback.md
  - plantillas_examen.py

1-3 · Entorno, operadores y condicionales

01_entorno_operadores_condicionales/00_entorno_comandos.md

Tema 1 - Chuleta de entorno

1. Crear y ejecutar un .py (ejercicio "Hola, Mundo" del profesor)

- En VS Code: Archivo > Nuevo archivo > guárdalo como `hola_mundo.py` (la extensión `.py` es obligatoria).
- Escribe: `print("¡Hola, Mundo!")`
- Abre la terminal integrada: **Ctrl + ñ** (o menú Terminal > Nuevo terminal).
- Ejecuta, situada en la carpeta del archivo:

Sistema	Orden
Windows / macOS	`python hola_mundo.py`
Linux / Ubuntu / WSL / Docker	`python3 hola_mundo.py` (el profesor lo EXIGE en Linux)

- `python --version` / `python3 --version` -> comprueba qué versión tienes (match-case necesita 3.10+).
- `python` sin archivo abre el intérprete interactivo (REPL, prompt `>>>`). Sal con `exit()`.
- Si dice "No such file": no estás en la carpeta. Muévete con `cd carpeta`, mira dónde estás con `pwd` (Linux) o `cd` (Windows), lista con `ls` (o `dir` en cmd).

2. pip y uv (instalar librerías)

- `pip install requests` (en Linux `pip3`, o mejor `python3 -m pip install requests`).
- `pip list` -> qué hay instalado. `pip freeze > requirements.txt` -> guardar dependencias. `pip install -r requirements.txt` -> instalarlas.
- `uv` es un gestor moderno y muy rápido: `uv venv`, `uv pip install requests`, `uv run archivo.py`.
- En el examen NO hay internet: usa solo la biblioteca estándar.

3. Entorno virtual (venv)

```
python -m venv .venv              # crear (Linux: python3 -m venv .venv)
.venv\Scripts\activate            # activar en Windows
source .venv/bin/activate         # activar en Linux / macOS / WSL
deactivate                        # salir
```

Sirve para que cada proyecto tenga sus propias librerías sin mezclarse. Al activarlo, el prompt muestra `(.venv)`.

4. Docker y WSL (básico)

- **WSL** = Linux (Ubuntu) dentro de Windows. Abrir: `wsl` en la terminal. Ahí se usa `python3`.
- **Regla de oro del profesor**: guarda los proyectos SIEMPRE en el home de Linux (`~/` o `~/proyectos_clase`) y NUNCA en `/mnt/c/Users/...` (Escritorio, Documentos), porque con Docker/WSL2 el rendimiento cae muchísimo.
- **Docker**: `docker ps` (contenedores en marcha), `docker run -it python:3.12 bash` (contenedor con Python), `docker compose up -d` (levantar el proyecto), `docker exec -it nombre bash` (entrar en un contenedor).
- **Prompt con la ruta (PS1)**: para ver el directorio actual en la terminal de Ubuntu se modifica la variable de entorno `PS1` dentro de `~/.bashrc`. Ejemplo: añadir al final `export PS1='\u@\h:\w\$ '` (`\w` = ruta actual) y recargar con `source ~/.bashrc`.

5. `if __name__ == "__main__":`

```
def saludar(nombre):
    """Devuelve un saludo."""
    return f"Hola, {nombre}"

if __name__ == "__main__":      # solo se ejecuta si lanzas ESTE archivo directamente
    print(saludar("Ana"))
```

- Si ejecutas `python archivo.py`, `__name__` vale `"__main__"` y se ejecuta el bloque.
- Si otro archivo hace `import archivo`, `__name__` vale `"archivo"` y el bloque NO se ejecuta (así las pruebas no se disparan al importar).
- Equivale al `public static void main(String[] args)` de Java.
- Funciones y clases: pon las pruebas siempre dentro de este bloque.

6. Diferencias rápidas con Java

- Sin `;` ni llaves: los bloques se marcan con `:` y **4 espacios** de sangría.
- Sin tipos al declarar: `edad = 18`. `input()` devuelve SIEMPRE `str` -> convierte con `int()` / `float()`.
- `&&`, `||`, `!` -> `and`, `or`, `not`. `true/false/null` -> `True/False/None`.

01_entorno_operadores_condicionales/00_INDICE.md

Índice - Temas 1, 2 y 3: Entorno, Operadores y Condicionales

Archivo	Qué pide	Forma exigida	Conceptos clave
`00_entorno_comandos.md`	Chuleta de entorno	En Linux se ejecuta con `python3`; proyectos en `~/`, nunca en `/mnt/c`	.py, terminal Ctrl+ñ, pip/uv, venv, Docker/WSL, PS1, `__main__`
`01_entorno_hola_mundo.py`	Tema 1 Ej. 1: Hola, Mundo	`python` (Win/mac) / `python3` (Linux)	print(), ejecutar desde la terminal
`02_operadores.py`	Tema 2 Ej. 1-3: mayoría de edad EPS, `verificar_acceso`, == vs is + demo	Entrada-Proceso-Salida; paridad con `%`; `==` contenido, `is` identidad (no existe `===`)	input/int, %, ==, is, aritméticos, lógicos, truthy/falsy, in, ternario, int/float/str, round
`03_cond_nota_if_elif_else.py`	Tema 3 Ej. 1: clasificar nota = 75	if / elif / else de mayor a menor	orden de condiciones
`04_cond_dias_match_case.py`	Tema 3 Ej. 2: día de la semana	match-case obligatorio + `case _`	patrones literales, comodín

`05_cond_paquetes_match_guardas.py`	Tema 3 Ej. 3: paquete por peso	guardas `case p if ...` + `case _`	patrón de captura, guarda, encadenada	
`06_cond_temperatura_encadenada.py`	Tema 3 Ej. 4: temp en [18.0, 27.0]	`18.0 <= temp <= 27.0` (prohibido `and`)	comparación encadenada	
`07_cond_raiz_guard_clause.py`	Tema 3 Ej. 5: `calcular_raiz_cuadrada(x)`	guard clause `if x <= 0: ...; return` sin else	return temprano, `** 0.5`	
`08_repo_nota_match_guardas.py`	Clase 0: nota con match	match con guardas, error primero, `case _`	float(input()), rangos semiabiertos	
`09_repo_temperatura_estados.py`	Clase 1: ÓPTIMO / FRÍO / CALOR	encadenada `18 <= temp <= 27` (prohibido `and`)	float(input()), if/elif/else	
`10_repo_saneamiento_entrada.py`	Clase 2: validar nombre y CP	`.strip()`, `if not nombre_limpio:`, `.isdigit()`	métodos de str, falsy	
`11_repo_comparacion_lexicografica.py`	Clase 3: qué producto va antes	1ª comparación directa, 2ª con `.lower()`	orden Unicode, 'A' < 'a'	
`12_repo_http_match_agrupado.py`	Clase 4: categorías HTTP	match con `case 200 | 201: agrupados + case _`	patrón OR `|`	`` ` ``
`99_resumen_formas_exigidas.py`	Plantillas mínimas para copiar	todas las anteriores	if/elif/else, ternario, encadenada, match, `|`	`` ` ``, guardas, guard clause

01_entorno_operadores_condicionales/01_entorno_hola_mundo.py

```
"""
TEMA 1 - Ejercicio 1: Primer programa "Hola, Mundo"
====================================================
ENUNCIADO (literal del profesor):
    "Crear un nuevo archivo en el editor y guardarlo con la extensión .py
    (por ejemplo, hola_mundo.py). Escribir la instrucción de salida por
    consola: print("¡Hola, Mundo!"). Abrir la terminal integrada (mediante
    Ctrl+ñ). Ejecutar el comando correspondiente: En Windows / macOS:
    python hola_mundo.py. En Linux: python3 hola_mundo.py."

FORMA QUE EXIGE EL PROFESOR:
    - En Linux/Ubuntu (y WSL/Docker) se ejecuta con python3, no con python.

CONCEPTOS QUE PRACTICA:
    - Archivo .py, print(), terminal integrada de VS Code (Ctrl+ñ).

CÓMO EJECUTARLO:  python 01_entorno_hola_mundo.py   (en Linux/Docker: python3 01_entorno_hola_mundo.py)
"""

# ------------------------------------------------------------------
# PASO ÚNICO: print() escribe en la consola y salta de línea al final.
# Las comillas pueden ser dobles "..." o simples '...': da igual en Python.
# ------------------------------------------------------------------
print("¡Hola, Mundo!")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# ¡Hola, Mundo!
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Guardar sin extensión .py o con .py.txt: el intérprete no lo encuentra.
# - Ejecutar desde otra carpeta: "No such file or directory". Haz cd a la carpeta.
# - En Linux, "python: command not found" -> usa python3.
# EN JAVA SERÍA: System.out.println("¡Hola, Mundo!"); dentro de una clase y un main.
```

01_entorno_operadores_condicionales/02_operadores.py

```
"""
TEMA 2 - Operadores y expresiones lógicas (3 ejercicios + demo)
================================================================
ENUNCIADO EJERCICIO 1 (literal): Verificación de mayoría de edad (Entrada-Proceso-Salida)
    "Entrada: El usuario ingresa su edad en años. Proceso: El programa compara
    la edad con el número 18. Salida: Se muestra el mensaje 'Sos mayor de edad'
    o 'Sos menor de edad'."

ENUNCIADO EJERCICIO 2 (literal): Comprobación de paridad y acceso (verificar_acceso)
    "Evaluación de mayoría de edad (Plan A vs Plan B): si la edad es mayor o
    igual a 18, imprimir 'Acceso concedido: Eres mayor de edad.', de lo
    contrario 'Acceso denegado: Eres menor de edad.'. Evaluación de paridad
    utilizando el operador módulo: si numero_verificacion % 2 == 0, imprimir
    'El código {numero_verificacion} es PAR.', si no, 'El código
    {numero_verificacion} es IMPAR.'"

ENUNCIADO EJERCICIO 3 (literal): Valor vs. identidad en memoria
    lista1 = [1, 15, 16]
    lista2 = [1, 15, 16]
    lista3 = lista1
    print(lista1 == lista2)  # True (Tienen el mismo valor)
    print(lista1 is lista2)  # False (Son objetos distintos en memoria)
    print(lista1 is lista3)  # True (Ambas variables apuntan al mismo objeto)

FORMA QUE EXIGE EL PROFESOR:
    - Ej. 1: modelo Entrada (input + int) -> Proceso (comparar con 18) -> Salida (print).
    - Ej. 2: la paridad se comprueba con el operador MÓDULO: numero % 2 == 0.
    - Ej. 3: == compara CONTENIDO; is / is not comparan IDENTIDAD (mismo objeto).
      En Python NO existe === (eso es de JavaScript/PHP).

CONCEPTOS QUE PRACTICA:
    - input(), int(), if/else, %, ==, is, f-strings.
    - Demo final: aritméticos, relacionales, lógicos, truthy/falsy, in/not in,
      ternario, conversión de tipos y round().

CÓMO EJECUTARLO:  python 02_operadores.py   (en Linux/Docker: python3 02_operadores.py)
    Pide una edad por teclado (ejercicio 1).
"""
```

```python
# ============================================================
# EJERCICIO 1: mayoría de edad con modelo Entrada-Proceso-Salida
# ============================================================
def ejercicio_1():
    """Pide la edad y dice si es mayor o menor de edad (EPS)."""
    # ENTRADA: input() devuelve SIEMPRE un str ("17"), aunque escribas números.
    # Por eso lo convertimos con int(); si no, "17" >= 18 da TypeError.
    edad = int(input("Introduce tu edad en años: "))

    # PROCESO: comparamos con 18 usando el operador relacional >=.
    # SALIDA: un print según el resultado (textos literales del enunciado).
    if edad >= 18:
        print("Sos mayor de edad")
    else:
        print("Sos menor de edad")


# ============================================================
# EJERCICIO 2: verificar_acceso (edad + paridad con %)
# ============================================================
def verificar_acceso(edad, numero_verificacion):
    """Comprueba la mayoría de edad y si el código de verificación es par."""
    # Plan A (if) vs Plan B (else): solo se ejecuta una de las dos ramas.
    if edad >= 18:
        print("Acceso concedido: Eres mayor de edad.")
    else:
        print("Acceso denegado: Eres menor de edad.")

    # % es el operador MÓDULO: devuelve el RESTO de la división entera.
    # Un número es par si al dividirlo entre 2 el resto es 0.
    #   8 % 2 -> 0 (par)      7 % 2 -> 1 (impar)
    if numero_verificacion % 2 == 0:
        print(f"El código {numero_verificacion} es PAR.")
    else:
        print(f"El código {numero_verificacion} es IMPAR.")


# ============================================================
# EJERCICIO 3: == (valor) frente a is (identidad)
# ============================================================
def ejercicio_3():
    """Muestra la diferencia entre == e is con listas."""
    lista1 = [1, 15, 16]
    lista2 = [1, 15, 16]   # otra lista NUEVA con el mismo contenido
    lista3 = lista1        # NO copia: lista3 es otro nombre para el MISMO objeto

    print(lista1 == lista2)  # True  -> mismo contenido
    print(lista1 is lista2)  # False -> son dos objetos distintos en memoria
    print(lista1 is lista3)  # True  -> apuntan al mismo objeto

    # id() devuelve la "dirección" del objeto: is equivale a id(a) == id(b).
    print(id(lista1) == id(lista3))  # True

    # Consecuencia práctica: si modificas lista3, también cambia lista1.
    lista3.append(99)
    print(lista1)  # [1, 15, 16, 99]

    # El uso correcto de is: comparar con None (es un objeto único).
    valor = None
    print(valor is None)       # True  (forma recomendada, NO "valor == None")
    print(valor is not None)   # False


# ============================================================
# DEMO COMENTADA DE OPERADORES (para consultar)
# ============================================================
def demo_operadores():
    """Ejemplos de todos los operadores del tema 2."""
    print("\n--- Aritméticos ---")
    print(7 / 2)    # 3.5  -> / SIEMPRE da float (en Java 7/2 da 3; aquí NO)
    print(7 // 2)   # 3    -> // división entera (redondea hacia abajo)
    print(-7 // 2)  # -4   -> ¡hacia abajo (suelo), no hacia el cero como Java!
    print(7 % 2)    # 1    -> % resto (módulo)
    print(2 ** 3)   # 8    -> ** potencia (en Java: Math.pow(2, 3))
    print(9 ** 0.5)  # 3.0 -> raíz cuadrada con ** 0.5

    print("\n--- Relacionales (devuelven True/False) ---")
    a, b = 5, 10
    print(a == b, a != b, a < b, a <= b, a > b, a >= b)  # False True True True False False
    print(1 < a < 10)  # True -> comparación ENCADENADA (equivale a 1 < a and a < 10)
    print("Ana" < "ana")  # True -> los str se comparan por código Unicode: 'A'(65) < 'a'(97)

    print("\n--- Lógicos: and, or, not (en Java: &&, ||, !) ---")
    print(True and False)  # False -> and: True solo si AMBOS son True
    print(True or False)   # True  -> or: True si ALGUNO es True
    print(not True)        # False -> not: invierte
    # Cortocircuito: si el primer operando ya decide, el segundo no se evalúa.
    x = 0
    print(x != 0 and 10 / x > 1)  # False, y NO da ZeroDivisionError

    print("\n--- Truthy / Falsy ---")
    # Son FALSY (cuentan como False en un if): 0, 0.0, "", [], {}, (), set(), None, False
    # TODO lo demás es TRUTHY (" " con un espacio, [0], "0", -1...)
    for valor in [0, 0.0, "", [], {}, None, "0", " ", [0], -1]:
        print(repr(valor), "->", bool(valor))
    nombre = ""
    if not nombre:  # forma pythónica de comprobar "está vacío"
        print("El nombre está vacío")

    print("\n--- Pertenencia: in / not in ---")
```

```
    print(3 in [1, 2, 3])          # True
    print("py" in "python")        # True  -> busca subcadenas
    print("x" not in "python")     # True
    print("edad" in {"edad": 20})  # True  -> en diccionarios busca en las CLAVES

    print("\n--- Operador ternario (expresión condicional) ---")
    # valor_si_true if condición else valor_si_false   (en Java: cond ? a : b)
    edad = 20
    mensaje = "mayor" if edad >= 18 else "menor"
    print(mensaje)  # mayor

    print("\n--- Conversión de tipos ---")
    print(int("42") + 1)     # 43    -> str a int
    print(float("3.5") * 2)  # 7.0   -> str a float
    print(int(3.99))         # 3     -> int() TRUNCA (corta decimales), no redondea
    print(str(10) + " años")  # "10 años" -> int a str para concatenar con +
    print(type(3.0))         # <class 'float'>
    # int("3.5") -> ValueError: primero float() y luego int(): int(float("3.5")) = 3

    print("\n--- round() ---")
    print(round(3.14159, 2))  # 3.14 -> redondea a 2 decimales
    print(round(2.5))         # 2    -> ¡OJO! redondeo "del banquero": a par
    print(round(3.5))         # 4
    print(f"{3.14159:.2f}")   # 3.14 -> alternativa: formatear solo al mostrar


if __name__ == "__main__":
    print("=== Ejercicio 1 ===")
    ejercicio_1()

    print("\n=== Ejercicio 2 ===")
    verificar_acceso(20, 8)
    verificar_acceso(15, 7)

    print("\n=== Ejercicio 3 ===")
    ejercicio_3()

    demo_operadores()

# ------------------------------------------------------------------
# SALIDA ESPERADA (introduciendo 18):
# === Ejercicio 1 ===
# Introduce tu edad en años: Sos mayor de edad
#
# === Ejercicio 2 ===
# Acceso concedido: Eres mayor de edad.
# El código 8 es PAR.
# Acceso denegado: Eres menor de edad.
# El código 7 es IMPAR.
#
# === Ejercicio 3 ===
# True
# False
# True
# True
# [1, 15, 16, 99]
# True
# False
# ... (después, la demo de operadores con los valores comentados en cada línea)
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar int() en input(): comparar str con int da TypeError.
# - Usar = (asignación) en vez de == (comparación) dentro de un if: SyntaxError.
# - Usar is para comparar números o textos: usa ==. is solo para None/identidad.
# - 18 es MAYOR de edad: la condición es >= 18, no > 18.
# EN JAVA SERÍA: == en objetos compara referencias (como is); .equals() compara contenido (como ==).
# ALTERNATIVA (funciona, pero NO es lo que pide el ej. 2): comprobar par con
#   numero_verificacion / 2 == numero_verificacion // 2  -> el enunciado exige %.
```

01_entorno_operadores_condicionales/03_cond_nota_if_elif_else.py

```
"""
TEMA 3 - Ejercicio 1: Clasificación escolar con if-elif-else
============================================================
ENUNCIADO (literal del profesor):
    "Dada la variable nota = 75, clasificar mediante condicionales encadenados:
    if nota >= 90: 'Excelente', elif nota >= 75: 'Muy bien', elif nota >= 60:
    'Bien', elif nota >= 40: 'Suficiente', else: 'Insuficiente'."

FORMA QUE EXIGE EL PROFESOR:
    - Cadena if / elif / else (un solo bloque, NO varios if sueltos).
    - Ordenado de MAYOR a MENOR para poder usar solo ">=".

CONCEPTOS QUE PRACTICA:
    - if-elif-else, operador >=, orden de las condiciones.

CÓMO EJECUTARLO:  python 03_cond_nota_if_elif_else.py   (en Linux/Docker: python3 ...)
"""

# ------------------------------------------------------------------
# PASO 1: el dato viene dado en una variable (no se pide input).
# ------------------------------------------------------------------
nota = 75

# ------------------------------------------------------------------
# PASO 2: cadena if-elif-else.
# Python evalúa de arriba abajo y entra SOLO en la primera condición True;
# el resto se salta. Por eso basta con ">=": si llegamos a "nota >= 75"
# ya sabemos que nota < 90 (si no, habría entrado en la primera).
# "elif" = "else if" de Java, pero en una sola palabra.
```

```python
# ------------------------------------------------------------------
if nota >= 90:
    print("Excelente")
elif nota >= 75:
    print("Muy bien")
elif nota >= 60:
    print("Bien")
elif nota >= 40:
    print("Suficiente")
else:  # cualquier valor < 40
    print("Insuficiente")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Muy bien
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Si ordenas al revés (primero >= 40) un 95 diría "Suficiente".
# - Varios "if" sueltos en vez de elif: un 95 imprimiría 4 mensajes.
# - No olvides los dos puntos ":" al final de cada if/elif/else.
# EN JAVA SERÍA: if (nota >= 90) {...} else if (nota >= 75) {...} else {...}
```

01_entorno_operadores_condicionales/04_cond_dias_match_case.py

```python
"""
TEMA 3 - Ejercicio 2: Menú de días con match-case
=================================================
ENUNCIADO (literal del profesor):
    "Dada la variable dia_numero = 3, evaluar con la estructura match
    dia_numero: case 1: 'Lunes', case 2: 'Martes', case 3: 'Miércoles',
    case 4: 'Jueves', case 5: 'Viernes', case _: 'Día no válido o fin de semana'."

FORMA QUE EXIGE EL PROFESOR:
    - OBLIGATORIO match-case (Python 3.10+).
    - OBLIGATORIA la cláusula comodín  case _:  al final.

CONCEPTOS QUE PRACTICA:
    - match / case con patrones literales, comodín _.

CÓMO EJECUTARLO:  python 04_cond_dias_match_case.py   (en Linux/Docker: python3 ...)
"""

# ------------------------------------------------------------------
# PASO 1: el dato a evaluar.
# ------------------------------------------------------------------
dia_numero = 3

# ------------------------------------------------------------------
# PASO 2: match compara dia_numero con cada case de arriba abajo
# y ejecuta el PRIMERO que coincide. No hace falta "break" como en el
# switch de Java: nunca "se cae" al siguiente case.
# ------------------------------------------------------------------
match dia_numero:
    case 1:
        print("Lunes")
    case 2:
        print("Martes")
    case 3:
        print("Miércoles")
    case 4:
        print("Jueves")
    case 5:
        print("Viernes")
    case _:
        # _ es el COMODÍN: coincide con cualquier cosa (como "default" en Java).
        # Debe ir el ÚLTIMO: si lo pones antes, Python da SyntaxError
        # ("wildcard makes remaining patterns unreachable").
        print("Día no válido o fin de semana")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Miércoles
# (con dia_numero = 6 -> Día no válido o fin de semana)
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Falta el case _: el profesor lo exige; sin él, un 6 no imprime nada.
# - Si el número viene de input(), conviértelo con int(): "3" no coincide con case 3.
# - match necesita Python 3.10 o superior (comprueba con python --version).
# EN JAVA SERÍA: switch (diaNumero) { case 1: ...; break; default: ... }
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if dia_numero == 1: ... elif dia_numero == 2: ... else: ...
```

01_entorno_operadores_condicionales/05_cond_paquetes_match_guardas.py

```python
"""
TEMA 3 - Ejercicio 3: Clasificación de paquetes con guardas condicionales
=========================================================================
ENUNCIADO (literal del profesor):
    "Dada la variable peso = 12.5, clasificar con match peso:
    case p if p <= 0: 'Peso no válido.', case p if p <= 5: 'Paquete pequeño.',
    case p if 5 < p <= 20: 'Paquete mediano.', case p if p > 20:
    'Paquete grande.', case _: 'Entrada no reconocida.'"

FORMA QUE EXIGE EL PROFESOR:
    - match-case con GUARDAS explícitas:  case p if condición:
    - case _ al final.
    - En el rango, comparación encadenada (5 < p <= 20), no "and".
```

```
CONCEPTOS QUE PRACTICA:
    - Patrón de captura (p) + guarda (if ...), comparación encadenada, comodín.

CÓMO EJECUTARLO:  python 05_cond_paquetes_match_guardas.py   (en Linux/Docker: python3 ...)
"""


def clasificar_paquete(peso):
    """Devuelve la categoría del paquete según su peso en kg."""
    # ------------------------------------------------------------
    # ¿Cómo funciona "case p if ...":?
    #   1) "p" es un PATRÓN DE CAPTURA: siempre coincide y guarda el valor
    #      de peso en la variable p.
    #   2) "if ..." es la GUARDA: el case solo se elige si además la
    #      condición es True. Si es False, se pasa al siguiente case.
    # Así match puede trabajar con RANGOS (un case 5: solo valdría para 5 exacto).
    # ------------------------------------------------------------
    match peso:
        case p if p <= 0:
            return "Peso no válido."
        case p if p <= 5:
            return "Paquete pequeño."
        case p if 5 < p <= 20:
            # Comparación encadenada: equivale a 5 < p and p <= 20.
            # (El "5 <" es redundante porque el case anterior ya filtró p <= 5,
            # pero el enunciado lo escribe así: déjalo igual.)
            return "Paquete mediano."
        case p if p > 20:
            return "Paquete grande."
        case _:
            # Con números nunca se llega aquí, pero el profesor exige el comodín.
            return "Entrada no reconocida."


if __name__ == "__main__":
    peso = 12.5
    print(clasificar_paquete(peso))

    # Pruebas de los límites de cada rango:
    for prueba in [-1, 0, 5, 5.1, 20, 20.5]:
        print(f"{prueba} kg -> {clasificar_paquete(prueba)}")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# Paquete mediano.
# -1 kg -> Peso no válido.
# 0 kg -> Peso no válido.
# 5 kg -> Paquete pequeño.
# 5.1 kg -> Paquete mediano.
# 20 kg -> Paquete mediano.
# 20.5 kg -> Paquete grande.
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - "case p <= 5:" SIN el "p if" es SyntaxError: la condición va SIEMPRE tras "if".
# - El orden importa: el primer case cuya guarda sea True gana.
# - Se puede usar cualquier nombre (p, n, x); no reutilices el nombre de la
#   variable evaluada (case peso if ...) porque la sobrescribe y confunde.
# EN JAVA SERÍA: switch con "case Double p when p <= 0 ->" (Java 21) o if-else.
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if peso <= 0: ... elif peso <= 5: ... -> el enunciado exige match con guardas.
```

01_entorno_operadores_condicionales/06_cond_temperatura_encadenada.py

```
"""
TEMA 3 - Ejercicio 4: Rangos con comparaciones encadenadas
============================================================
ENUNCIADO (literal del profesor):
    "Dada la variable temp = 22.5, comprobar si está en el rango [18.0, 27.0]
    utilizando la sintaxis encadenada: if 18.0 <= temp <= 27.0:
    'La temperatura es agradable.' else: 'Hace demasiado frío o demasiado calor.'"

FORMA QUE EXIGE EL PROFESOR:
    - Comparación ENCADENADA de estilo matemático:  18.0 <= temp <= 27.0
    - PROHIBIDO:  temp >= 18.0 and temp <= 27.0   (resta puntos aunque funcione)

CONCEPTOS QUE PRACTICA:
    - Comparación encadenada, intervalo cerrado [a, b] -> <= en ambos lados.

CÓMO EJECUTARLO:  python 06_cond_temperatura_encadenada.py   (en Linux/Docker: python3 ...)
"""

# ------------------------------------------------------------
# PASO 1: el dato.
# ------------------------------------------------------------
temp = 22.5

# ------------------------------------------------------------
# PASO 2: comparación encadenada.
# Python permite escribir el rango como en matemáticas. Internamente
# hace "18.0 <= temp and temp <= 27.0", pero evalúa temp UNA sola vez
# y se lee mejor. Rango con corchetes [18.0, 27.0] = extremos INCLUIDOS,
# por eso <= en los dos lados.
# ------------------------------------------------------------
if 18.0 <= temp <= 27.0:
    print("La temperatura es agradable.")
else:
    print("Hace demasiado frío o demasiado calor.")

# ------------------------------------------------------------
```

```
# SALIDA ESPERADA:
# La temperatura es agradable.
# (con temp = 30 -> Hace demasiado frío o demasiado calor.)
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Usar < en vez de <=: 18.0 y 27.0 quedarían fuera y el rango es cerrado.
# - Para "fuera del rango" puedes negar todo: if not (18.0 <= temp <= 27.0):
# EN JAVA SERÍA: if (temp >= 18.0 && temp <= 27.0)  -> en Python NO se hace así.
# ALTERNATIVA (funciona, pero el profesor la PROHÍBE):
#   if temp >= 18.0 and temp <= 27.0:
```

01_entorno_operadores_condicionales/07_cond_raiz_guard_clause.py

```
"""
TEMA 3 - Ejercicio 5: Salidas tempranas con cláusula de guarda
============================================================
ENUNCIADO (literal del profesor):
    "Definir calcular_raiz_cuadrada(x) usando Detección temprana de condición
    de error (Guard Clause): if x <= 0: print("Error: Por favor, use solo
    números positivos."); return. Flujo principal limpio de anidamiento:
    resultado = x ** 0.5."

FORMA QUE EXIGE EL PROFESOR:
    - Cláusula de guarda: el caso de ERROR se comprueba PRIMERO y se sale
      con return temprano.
    - El flujo normal va DESPUÉS, sin else y sin sangría extra (estructura plana,
      sin código en "flecha").

CONCEPTOS QUE PRACTICA:
    - def, return anticipado, potencia ** 0.5 como raíz cuadrada.

CÓMO EJECUTARLO:  python 07_cond_raiz_guard_clause.py   (en Linux/Docker: python3 ...)
"""


def calcular_raiz_cuadrada(x):
    """Calcula e imprime la raíz cuadrada de x si es positivo."""
    # ------------------------------------------------------------
    # PASO 1: CLÁUSULA DE GUARDA. Si el dato es inválido, avisamos y
    # salimos YA con return. Un "return" sin valor devuelve None y
    # termina la función: lo de debajo no se ejecuta.
    # (El enunciado usa x <= 0, así que el 0 también da error, aunque
    # matemáticamente su raíz sea 0. Respeta el enunciado.)
    # ------------------------------------------------------------
    if x <= 0:
        print("Error: Por favor, use solo números positivos.")
        return

    # ------------------------------------------------------------
    # PASO 2: FLUJO PRINCIPAL. Si llegamos aquí es seguro que x > 0,
    # así que no hace falta else ni sangrar más.
    # x ** 0.5 = raíz cuadrada (sin importar math).
    # ------------------------------------------------------------
    resultado = x ** 0.5
    print(f"La raíz cuadrada de {x} es {resultado}")
    return resultado


if __name__ == "__main__":
    calcular_raiz_cuadrada(25)
    calcular_raiz_cuadrada(2)
    calcular_raiz_cuadrada(-4)
    calcular_raiz_cuadrada(0)

# ------------------------------------------------------------
# SALIDA ESPERADA:
# La raíz cuadrada de 25 es 5.0
# La raíz cuadrada de 2 es 1.4142135623730951
# Error: Por favor, use solo números positivos.
# Error: Por favor, use solo números positivos.
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar el return dentro del if: imprime el error Y sigue calculando
#   ((-4) ** 0.5 da un número complejo, no un error).
# - Poner el cálculo dentro de un else: funciona, pero es justo el anidamiento
#   que la guard clause quiere evitar.
# EN JAVA SERÍA: if (x <= 0) { System.out.println("Error..."); return; }
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado, código en "flecha"):
#   if x > 0:
#       resultado = x ** 0.5
#       print(...)
#   else:
#       print("Error: ...")
```

01_entorno_operadores_condicionales/08_repo_nota_match_guardas.py

```
"""
TEMA 3 - Ejercicio de clase 0: Nota con match y guardas
=======================================================
ENUNCIADO (del repo de clase):
    Un programa que lea una nota por teclado con match: si la nota es menor
    que 5 está suspendido; si es entre 5 y 7 aprobado, sin incluir el 7; de 7
    a 8,5 notable, sin incluir el 8,5; y de 8,5 a 10 sobresaliente, los dos
    incluidos. Además, si la nota es negativa o mayor que 10 es un error de dato.

FORMA QUE EXIGE EL PROFESOR:
    - match con GUARDAS  (case n if condición:)  y  case _  al final.
```

```
    - El caso de error se comprueba PRIMERO.

CONCEPTOS QUE PRACTICA:
    - input() + float(), match con guardas, rangos semiabiertos, orden de los case.

CÓMO EJECUTARLO:  python 08_repo_nota_match_guardas.py   (en Linux/Docker: python3 ...)
"""

# ------------------------------------------------------------------
# PASO 1: ENTRADA. float() porque la nota puede llevar decimales (8.5).
# OJO: el decimal se escribe con PUNTO (8.5), no con coma (8,5 -> ValueError).
# ------------------------------------------------------------------
nota = float(input("Introduce una nota: "))

# ------------------------------------------------------------------
# PASO 2: match con guardas. "n" captura el valor de nota y el "if"
# decide si ese case vale. Se evalúan en orden y gana el primero True,
# así que cada case solo necesita el límite SUPERIOR:
#   - Si llega a "n < 7" ya sabemos que n >= 5 (si no, habría entrado antes).
# Tabla de rangos:
#   [0, 5)     Suspenso
#   [5, 7)     Aprobado       (7 no incluido)
#   [7, 8.5)   Notable        (8.5 no incluido)
#   [8.5, 10]  Sobresaliente  (ambos incluidos)
# ------------------------------------------------------------------
match nota:
    case n if n < 0 or n > 10:
        # Va PRIMERO: si no, un -3 entraría en "n < 5" y diría Suspenso.
        # Aquí "or" es correcto: son dos zonas separadas, no un rango.
        print("Error: nota fuera de rango")
    case n if n < 5:
        print("Suspenso")
    case n if n < 7:
        print("Aprobado")
    case n if n < 8.5:
        print("Notable")
    case n if n <= 10:
        print("Sobresaliente")
    case _:
        # Nunca se llega con números válidos, pero el profesor exige el comodín.
        print("Entrada no reconocida")

# ------------------------------------------------------------------
# SALIDA ESPERADA (ejemplos):
# 4.9 -> Suspenso | 5 -> Aprobado | 7 -> Notable | 8.5 -> Sobresaliente
# 10 -> Sobresaliente | 11 -> Error: nota fuera de rango | -1 -> Error: nota fuera de rango
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - En la versión original se escribía "case nota if ...": funciona, pero
#   reutiliza el nombre de la variable evaluada; mejor un nombre corto (n).
# - Faltaba el case _ en la versión original: añádelo siempre.
# - Si escriben letras, float() lanza ValueError (se controla con try/except).
# - Variante con encadenada (también válida): case n if 5 <= n < 7:
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if nota < 0 or nota > 10: ... elif nota < 5: ... -> se pide match.
```

01_entorno_operadores_condicionales/09_repo_temperatura_estados.py

```
"""
TEMA 3 - Ejercicio de clase 1: Verificador de rangos y estilo de comparación
================================================================================
ENUNCIADO (del repo de clase):
    En Java, para comprobar si una temperatura T está en un rango seguro entre
    18 ºC y 27 ºC, escribirías if (temp >= 18 && temp <= 27). Escribe un
    programa en Python que pida al usuario una temperatura flotante e indique
    si la temperatura está en estado ÓPTIMO (18 <= T <= 27), ALERTA FRÍO (T < 18)
    o ALERTA CALOR (T > 27).
    Requisito: debes utilizar la sintaxis de comparaciones encadenadas directa
    de Python en lugar del operador and tradicional para el rango óptimo.

FORMA QUE EXIGE EL PROFESOR:
    - Rango óptimo con comparación ENCADENADA: 18 <= temp <= 27
    - PROHIBIDO: temp >= 18 and temp <= 27

CONCEPTOS QUE PRACTICA:
    - input() + float(), if/elif/else, comparación encadenada.

CÓMO EJECUTARLO:  python 09_repo_temperatura_estados.py   (en Linux/Docker: python3 ...)
"""

# ------------------------------------------------------------------
# PASO 1: ENTRADA. input() devuelve str; float() permite decimales (22.5).
# ------------------------------------------------------------------
temp = float(input("Introduce una temperatura en grados Celsius: "))

# ------------------------------------------------------------------
# PASO 2: el rango óptimo se pregunta PRIMERO y con la forma exigida
# (encadenada). Así queda claro en el código que se cumple el requisito.
# Los otros dos casos son lo que queda: si no está en [18, 27] y es
# menor que 18 -> frío; si no, calor.
# ------------------------------------------------------------------
if 18 <= temp <= 27:
    print("ESTADO: ÓPTIMO")
elif temp < 18:
    print("ESTADO: ALERTA FRÍO")
else:  # temp > 27
    print("ESTADO: ALERTA CALOR")

# ------------------------------------------------------------------
# SALIDA ESPERADA (ejemplos):
```

```
# 22.5 -> ESTADO: ÓPTIMO | 18 -> ESTADO: ÓPTIMO | 27 -> ESTADO: ÓPTIMO
# 17.9 -> ESTADO: ALERTA FRÍO | 27.1 -> ESTADO: ALERTA CALOR
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Los mensajes del enunciado son "ALERTA FRÍO" / "ALERTA CALOR"; respétalos.
# - Escribe el decimal con punto: 22.5 (22,5 -> ValueError).
# EN JAVA SERÍA: if (temp >= 18 && temp <= 27) -> en Python se escribe encadenado.
# ALTERNATIVA (funciona, pero el profesor la PROHÍBE):
#   if temp >= 18 and temp <= 27:
```

01_entorno_operadores_condicionales/10_repo_saneamiento_entrada.py

```
"""
TEMA 3 - Ejercicio de clase 2: Saneamiento de entrada y validación de datos
============================================================================
ENUNCIADO (del repo de clase):
    Crea un script para validar el formulario de registro de un usuario en un
    sistema web. El programa solicitará el nombre de usuario y el código
    postal (CP).
    Requisitos:
    - Aplica .strip() a ambas entradas para eliminar espacios accidentales
      al inicio y final.
    - Verifica si el nombre está vacío aprovechando la evaluación de falsedad
      de cadenas vacías (if not nombre_limpio:).
    - Comprueba con .isdigit() si el código postal contiene únicamente dígitos.
    - Si todo es correcto, muestra "Registro válido". Si no, muestra el mensaje
      de error específico correspondiente (campo vacío o CP no numérico).

FORMA QUE EXIGE EL PROFESOR:
    - .strip() en las dos entradas.
    - Nombre vacío con  if not nombre_limpio:   (NO con == "" ni len() == 0).
    - CP con  .isdigit().
    - if / elif / else con un mensaje de error distinto para cada fallo.

CONCEPTOS QUE PRACTICA:
    - Métodos de str (.strip(), .lower(), .isdigit()), falsy, if-elif-else, f-strings.

CÓMO EJECUTARLO:  python 10_repo_saneamiento_entrada.py   (en Linux/Docker: python3 ...)
    Pide dos datos: nombre y código postal.
"""

# ----------------------------------------------------------------
# PASO 1: ENTRADA + SANEAMIENTO.
# Los métodos se encadenan: input() devuelve un str y sobre él llamamos
# .strip() (quita espacios, tabuladores y saltos al principio y al final,
# NO los del medio) y .lower() (todo a minúsculas, para normalizar el usuario).
# El CP no se pasa a minúsculas: solo debe tener dígitos.
# ----------------------------------------------------------------
nombre_limpio = input("Introduce el nombre de usuario: ").strip().lower()
cp_limpio = input("Introduce el código postal (CP): ").strip()

# ----------------------------------------------------------------
# PASO 2: VALIDACIONES en cadena (se informa del PRIMER error encontrado).
# ----------------------------------------------------------------
# Una cadena vacía "" es FALSY: "not nombre_limpio" es True si está vacía.
# Gracias al strip(), "   " (solo espacios) también queda "" y se detecta.
if not nombre_limpio:
    print("Error: El campo de nombre de usuario no puede estar vacío.")

# .isdigit() devuelve True solo si TODOS los caracteres son dígitos y hay
# al menos uno. "28001" -> True ; "28A01" -> False ; "" -> False ; "-5" -> False
elif not cp_limpio.isdigit():
    print("Error: El código postal debe contener únicamente dígitos.")

else:
    print("Registro válido")
    # Mostramos los datos ya limpios entre comillas para ver que no hay espacios.
    print(f"Usuario guardado: '{nombre_limpio}' | Código postal guardado: '{cp_limpio}'")

# ----------------------------------------------------------------
# SALIDA ESPERADA (entradas "  Laura  " y " 28001 "):
# Registro válido
# Usuario guardado: 'laura' | Código postal guardado: '28001'
# (entrada nombre "   " -> Error: El campo de nombre de usuario no puede estar vacío.)
# (entrada CP "28A01"   -> Error: El código postal debe contener únicamente dígitos.)
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - .strip() NO modifica la variable (los str son inmutables): devuelve uno
#   nuevo. Hay que guardarlo: nombre = nombre.strip()
# - Olvidar los paréntesis: cp.isdigit sin () es el método, no su resultado
#   (y siempre sería truthy).
# EN JAVA SERÍA: nombre.trim().toLowerCase(); nombre.isEmpty(); cp.matches("\\d+")
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if nombre_limpio == "":   o   if len(nombre_limpio) == 0:  -> se pide "if not".
```

01_entorno_operadores_condicionales/11_repo_comparacion_lexicografica.py

```
"""
TEMA 3 - Ejercicio de clase 3: Comparación lexicográfica y mayúsculas/minúsculas
================================================================================
ENUNCIADO (del repo de clase):
    Un sistema de catálogo necesita comparar dos nombres de productos (prod1 y
    prod2) introducidos por el usuario para determinar cuál aparecerá primero
    alfabéticamente.
    Requisitos:
    - Muestra qué producto va primero evaluando las cadenas directamente sin
      modificar (recordando que en ASCII 'A' < 'a').
```

```
    - A continuación, realiza una 2ª comparación insensible a
      mayúsculas/minúsculas empleando .lower().
    - Si ambas cadenas normalizadas son idénticas, indica que son el mismo
      producto independientemente de la caja (case-insensitive).

FORMA QUE EXIGE EL PROFESOR:
    - DOS comparaciones: 1ª directa (<, >, == sobre los str tal cual);
      2ª con .lower().
    - Comparar str con los operadores relacionales (no hace falta compareTo).

CONCEPTOS QUE PRACTICA:
    - Comparación lexicográfica (Unicode/ASCII), .lower(), .strip(), if-elif-else.

CÓMO EJECUTARLO:  python 11_repo_comparacion_lexicografica.py   (en Linux/Docker: python3 ...)
    Pide dos productos.
"""

# ------------------------------------------------------------------
# PASO 1: ENTRADA (sin tocar: la 1ª comparación es "en bruto").
# ------------------------------------------------------------------
prod1 = input("Introduce el primer producto: ")
prod2 = input("Introduce el segundo producto: ")

# ------------------------------------------------------------------
# PASO 2: comparación DIRECTA.
# Python compara carácter a carácter por su código Unicode (ord()).
# En cuanto encuentra una diferencia, decide. Las MAYÚSCULAS van antes
# que las minúsculas: ord('Z') = 90 < ord('a') = 97, así que "Zapato" < "abrigo".
# ------------------------------------------------------------------
print("\n--- 1. Comparación directa (ASCII) ---")
if prod1 < prod2:
    print(f"'{prod1}' va antes que '{prod2}'.")
elif prod1 > prod2:
    print(f"'{prod2}' va antes que '{prod1}'.")
else:
    print("Ambos productos son idénticos.")

# ------------------------------------------------------------------
# PASO 3: comparación NORMALIZADA. Con .lower() (y .strip() para quitar
# espacios sobrantes) ya no influye cómo se escribió.
# ------------------------------------------------------------------
prod1_norm = prod1.strip().lower()
prod2_norm = prod2.strip().lower()

print("\n--- 2. Comparación normalizada (ignorando mayúsculas/minúsculas) ---")
if prod1_norm < prod2_norm:
    print(f"'{prod1}' va antes que '{prod2}' (normalizado).")
elif prod1_norm > prod2_norm:
    print(f"'{prod2}' va antes que '{prod1}' (normalizado).")
else:
    print("Son el mismo producto independientemente de la caja (case-insensitive).")

# ------------------------------------------------------------------
# SALIDA ESPERADA (entradas "Pepe" y "pepe"):
# --- 1. Comparación directa (ASCII) ---
# 'Pepe' va antes que 'pepe'.
#
# --- 2. Comparación normalizada (ignorando mayúsculas/minúsculas) ---
# Son el mismo producto independientemente de la caja (case-insensitive).
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Las tildes van DESPUÉS de todas las letras: "ábaco" > "zorro" (ord('á') = 225).
# - Los números como texto se comparan por caracteres: "10" < "9" es True.
# EN JAVA SERÍA: prod1.compareTo(prod2) < 0 ; prod1.equalsIgnoreCase(prod2)
```

01_entorno_operadores_condicionales/12_repo_http_match_agrupado.py

```
"""
TEMA 3 - Ejercicio de clase 4: Manejador de respuestas HTTP con match-case
============================================================================
ENUNCIADO (del repo de clase):
    Diseña un controlador de API REST que reciba un código de estado HTTP
    entero (por ejemplo: 200, 201, 400, 401, 403, 404, 500, 503) y devuelva
    una categoría estándar.
    Requisitos:
    - Usa una estructura match-case.
    - Agrupa en una sola rama (case 200 | 201:) las respuestas exitosas.
    - Agrupa en una sola rama los errores de cliente conocidos (400, 401, 403, 404).
    - Agrupa en una rama los errores de servidor (500, 503).
    - Usa el comodín "_" para capturar cualquier otro código no contemplado e
      informar de "Código de estado no reconocido".

FORMA QUE EXIGE EL PROFESOR:
    - match-case con PATRONES AGRUPADOS con |  (una rama por grupo).
    - case _ al final.

CONCEPTOS QUE PRACTICA:
    - Patrones literales, patrón OR (|), comodín, int(input()).

CÓMO EJECUTARLO:  python 12_repo_http_match_agrupado.py   (en Linux/Docker: python3 ...)
    Pide un código HTTP.
"""

# ------------------------------------------------------------------
# PASO 1: ENTRADA. int() porque los case comparan con números:
# el texto "200" NO coincidiría con case 200.
# ------------------------------------------------------------------
codigo_http = int(input("Introduce un código de estado HTTP (ej. 200, 404, 500): "))

# ------------------------------------------------------------------
```

```python
# PASO 2: match con patrones agrupados.
# "|" dentro de un case significa "o": case 200 | 201 coincide con 200 O con 201.
# OJO: en un case se usa |, NUNCA "or" (case 200 or 201 es SyntaxError).
# ------------------------------------------------------------------
match codigo_http:
    case 200 | 201:
        print("Éxito: La solicitud se ha completado correctamente.")
    case 400 | 401 | 403 | 404:
        print("Error del cliente: Revise los datos o permisos enviados.")
    case 500 | 503:
        print("Error del servidor: Hubo un fallo en el sistema remoto.")
    case _:
        # Comodín: cualquier otro código (302, 418...). Es el "default" de Java.
        print("Código de estado no reconocido.")

# ------------------------------------------------------------------
# SALIDA ESPERADA (ejemplos):
# 201 -> Éxito: La solicitud se ha completado correctamente.
# 404 -> Error del cliente: Revise los datos o permisos enviados.
# 503 -> Error del servidor: Hubo un fallo en el sistema remoto.
# 418 -> Código de estado no reconocido.
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Una rama por código (case 400: ... case 401: ...) funciona, pero NO agrupa:
#   el enunciado pide una sola rama por grupo con |.
# - Si quisieras saber CUÁL coincidió: case 400 | 401 | 403 | 404 as codigo:
# EN JAVA SERÍA: switch (codigo) { case 200, 201 -> ...; default -> ...; }
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if codigo_http in (200, 201): ... elif codigo_http in (400, 401, 403, 404): ...
```

01_entorno_operadores_condicionales/99_resumen_formas_exigidas.py

```python
"""
TEMA 3 - RESUMEN: cada estructura condicional en su versión MÍNIMA
==================================================================
Para copiar rápido en el examen. Cada bloque indica cuándo usarlo.

FORMAS QUE EXIGE EL PROFESOR (resumen):
    - Rangos: comparación ENCADENADA  a <= x <= b   (PROHIBIDO x >= a and x <= b)
    - match-case SIEMPRE con  case _  al final.
    - Rangos dentro de match: guardas  case n if condición:
    - Varios valores en una rama: patrón agrupado  case 1 | 2 | 3:
    - Errores al principio de una función: guard clause con return temprano.

CÓMO EJECUTARLO:  python 99_resumen_formas_exigidas.py   (en Linux/Docker: python3 ...)
"""

x = 7

# --- 1. if / elif / else (de mayor a menor, gana la primera condición True) ---
if x >= 9:
    print("alto")
elif x >= 5:
    print("medio")
else:
    print("bajo")

# --- 2. Ternario: valor_si_true if condición else valor_si_false ---
paridad = "par" if x % 2 == 0 else "impar"
print(paridad)

# --- 3. Comparación encadenada (rango [5, 10]) - FORMA EXIGIDA para rangos ---
if 5 <= x <= 10:
    print("dentro del rango")
else:
    print("fuera del rango")

# --- 4. match literal + case _ (comodín obligatorio, siempre el último) ---
match x:
    case 1:
        print("uno")
    case 7:
        print("siete")
    case _:
        print("otro")

# --- 5. match con patrón agrupado | (NUNCA "or" dentro de un case) ---
match x:
    case 1 | 3 | 5 | 7 | 9:
        print("impar de una cifra")
    case 0 | 2 | 4 | 6 | 8:
        print("par de una cifra")
    case _:
        print("más de una cifra")

# --- 6. match con guardas: case n if ... (para rangos dentro de match) ---
match x:
    case n if n < 0:
        print("negativo")
    case n if 0 <= n < 5:
        print("bajo")
    case n if n >= 5:
        print("alto")
    case _:
        print("no reconocido")


# --- 7. Guard clause: el error primero y return temprano, sin else ---
def dividir(a, b):
    """Divide a entre b; si b es 0 avisa y sale antes."""
```

```python
    if b == 0:
        print("Error: no se puede dividir entre 0.")
        return
    return a / b


if __name__ == "__main__":
    print(dividir(10, 4))
    print(dividir(1, 0))

# ------------------------------------------------------------
# SALIDA ESPERADA:
# medio
# impar
# dentro del rango
# siete
# impar de una cifra
# alto
# 2.5
# Error: no se puede dividir entre 0.
# None
# ------------------------------------------------------------
# OJO: el "None" final sale porque dividir(1, 0) hace "return" sin valor
# y el print lo muestra. Es normal.
```

4 · Bucles

02_bucles/00_INDICE.md

Tema 4 - Bucles (Ejercicios Bucles python.pdf)

Archivo	Qué pide	Forma exigida	Conceptos clave
`01_total_carrito_sin_sum.py`	Total de la lista de precios	for sobre lista, **sin sum()**	acumulador `total += precio`
`02_contar_vocales.py`	Contar vocales de una palabra (input)	for letra a letra	`for letra in palabra`, `in`, contador
`03_tabla_del_8_range.py`	Tabla del 8 del 1 al 10	`range()`	`range(1, 11)`, el fin se excluye
`04_cuenta_atras_pares.py`	Pares del 20 al 0	`range` con **paso**	`range(20, -1, -2)`
`05_tareas_enumerate.py`	Tareas numeradas desde 1	`enumerate(tareas, start=1)`	desempaquetado `for n, t in ...`
`06_combinaciones_anidados.py`	Tallas x colores	for **anidados**	2 x 3 = 6 combinaciones
`07_cubos_comprension.py`	Cubos de una lista	comprensión en **una línea**	`[n ** 3 for n in numeros]`
`08_filtrar_palabras_comprension.py`	Palabras de más de 3 letras	comprensión con **if**	`[p for p in palabras if len(p) > 3]`
`09_contrasena_while.py`	Pedir contraseña hasta 'python123' (input)	**while**	condición `!=`, input dentro del bucle
`10_contador_acumulador_while.py`	Sumar hasta superar 50 y contar (input)	**while**	contador vs acumulador, `suma <= 50`
`11_login_cajero_while.py`	PIN '1234', máx. 3 intentos (input)	**while** con condición compuesta	`and`, De Morgan, por qué salió
`12_calculadora_while_true_break.py`	Sumas hasta 'salir' (input)	**while True + break**	comprobar 'salir' antes de convertir; isdigit / try-except (comentado)
`13_saltar_multiplos_continue.py`	1-10 sin múltiplos de 3	**continue**	`% 3 == 0`
`14_buscar_99_for_else.py`	Buscar 99 en una lista	**else del bucle** + break	el else solo si NO hubo break
`15_inventario_dict_items.py`	Imprimir clave y valor del inventario	**.items()**	`for fruta, cantidad in d.items()`
`16_resumen_bucles.py`	Chuleta ejecutable	—	for/range/enumerate/zip/dict, while, break, continue, else, anidados, comprensiones, tabla "si me piden X → uso Y"

Notas: en los ejercicios 7 y 14 la lista del enunciado salió cortada; se usan `[1, 2, 3, 4, 5]` y `[12, 45, 99, 7, 30]` (más una lista sin el 99 para ver el `else`).

02_bucles/01_total_carrito_sin_sum.py

```python
"""
TEMA 4 - Ejercicio 1: Bucle for (iterar listas)
================================================
ENUNCIADO (literal del profesor):
    Dada una lista de precios de un carrito de la compra [15.50, 20.0, 35.75, 10.0],
    calcula el total iterando sobre la lista (sin usar la función sum()).

FORMA QUE EXIGE EL PROFESOR:
    - Recorrer la lista con un bucle for.
    - PROHIBIDO usar sum(). Hay que sumar "a mano" con un acumulador.

CONCEPTOS QUE PRACTICA:
    - for elemento in lista
    - Variable ACUMULADORA (empieza en 0 y en cada vuelta se le suma algo)

CÓMO EJECUTARLO:  python 01_total_carrito_sin_sum.py   (en Linux/Docker: python3 01_total_carrito_sin_sum.py)
"""

# ---------------------------------------------------------------
# PASO 1: los datos del enunciado
# ---------------------------------------------------------------
precios = [15.50, 20.0, 35.75, 10.0]

# ---------------------------------------------------------------
# PASO 2: el ACUMULADOR
# Un acumulador es una variable que va "guardando la suma" de todo lo
# que ya hemos recorrido. Se crea ANTES del bucle y vale 0, porque
# sumar 0 no cambia nada (es el "elemento neutro" de la suma).
# Si lo creo DENTRO del bucle, se pondría a 0 en cada vuelta: error típico.
# ---------------------------------------------------------------
total = 0

# ---------------------------------------------------------------
# PASO 3: recorrer la lista
# En cada vuelta, la variable 'precio' toma un valor de la lista:
# 1ª vuelta 15.50, 2ª vuelta 20.0, 3ª vuelta 35.75, 4ª vuelta 10.0.
# No hacen falta índices (no es como el for (int i = 0; ...) de Java).
# ---------------------------------------------------------------
for precio in precios:
    total += precio  # es lo mismo que: total = total + precio

# ---------------------------------------------------------------
# PASO 4: mostrar el resultado (FUERA del bucle, sin sangría,
# para que se imprima una sola vez al final y no en cada vuelta)
# :.2f -> redondea a 2 decimales, como un precio.
# ---------------------------------------------------------------
print(f"El total del carrito es: {total:.2f} €")
```

```
# ------------------------------------------------------------------
# SALIDA ESPERADA:
# El total del carrito es: 81.25 €
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner el print dentro del for (con sangría): saldrían 4 líneas.
# - Olvidar inicializar total = 0 -> NameError.
# EN JAVA SERÍA: for (double precio : precios) { total += precio; }
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado, te bajaría nota):
#     total = sum(precios)
```

02_bucles/02_contar_vocales.py

```
"""
TEMA 4 - Ejercicio 2: Bucle for (iterar cadenas)
================================================
ENUNCIADO (literal del profesor):
    Pide una palabra al usuario y cuenta cuántas vocales (a, e, i, o, u) contiene
    iterando letra por letra.

FORMA QUE EXIGE EL PROFESOR:
    - Recorrer el string letra por letra con un for (for letra in palabra).

CONCEPTOS QUE PRACTICA:
    - Un string se puede recorrer con for como si fuera una lista de letras.
    - Operador 'in' para preguntar "¿está esta letra dentro de este texto?".
    - Variable CONTADORA (suma 1 cada vez que pasa algo).

CÓMO EJECUTARLO:  python 02_contar_vocales.py   (en Linux/Docker: python3 02_contar_vocales.py)
"""

# ------------------------------------------------------------------
# PASO 1: pedir la palabra
# .lower() la pasa a minúsculas, así "Árbol" o "CASA" también cuentan
# sus vocales sin tener que comparar con "AEIOU".
# ------------------------------------------------------------------
palabra = input("Introduce una palabra: ").lower()

# ------------------------------------------------------------------
# PASO 2: preparar el CONTADOR y las vocales a buscar
# Un contador suma SIEMPRE 1 (cuenta cuántas veces ocurre algo).
# Incluyo las vocales con tilde y la ü para que "murciélago" cuente bien.
# ------------------------------------------------------------------
vocales = "aeiouáéíóúü"
contador_vocales = 0

# ------------------------------------------------------------------
# PASO 3: recorrer la palabra letra a letra
# En cada vuelta 'letra' es un string de 1 carácter.
# ------------------------------------------------------------------
for letra in palabra:
    if letra in vocales:       # ¿la letra está dentro de "aeiou..."?
        contador_vocales += 1  # sí -> sumo 1 al contador

# ------------------------------------------------------------------
# PASO 4: resultado (fuera del bucle)
# ------------------------------------------------------------------
print(f"La palabra '{palabra}' tiene {contador_vocales} vocales.")

# ------------------------------------------------------------------
# SALIDA ESPERADA (si escribo Murciélago):
# Introduce una palabra: La palabra 'murciélago' tiene 5 vocales.
# (Si escribo una palabra sin vocales, p. ej. "bsk", dice 0 vocales.)
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Sin .lower(), las mayúsculas no cuentan (o hay que poner "aeiouAEIOU").
# - Si el enunciado solo dice "a, e, i, o, u" y no quieres tildes, usa vocales = "aeiou".
# EN JAVA SERÍA: for (char letra : palabra.toCharArray()) { ... }
# ALTERNATIVA (funciona, pero NO es "iterando letra por letra" con un for clásico):
#     contador_vocales = sum(1 for letra in palabra if letra in vocales)
```

02_bucles/03_tabla_del_8_range.py

```
"""
TEMA 4 - Ejercicio 3: Bucle for con range()
===========================================
ENUNCIADO (literal del profesor):
    Genera la tabla de multiplicar del 8, desde el 1 hasta el 10 usando la función range().

FORMA QUE EXIGE EL PROFESOR:
    - Usar range() en el for.

CONCEPTOS QUE PRACTICA:
    - range(inicio, fin): genera números desde 'inicio' hasta 'fin - 1'
      (el final NO se incluye).

CÓMO EJECUTARLO:  python 03_tabla_del_8_range.py   (en Linux/Docker: python3 03_tabla_del_8_range.py)
"""

# ------------------------------------------------------------------
# PASO 1: el número de la tabla
# Guardarlo en una variable permite cambiar de tabla tocando una sola línea.
# ------------------------------------------------------------------
numero = 8
print(f"Tabla del {numero}:")

# ------------------------------------------------------------------
```

```
# PASO 2: recorrer del 1 al 10
# range(1, 11) -> 1, 2, 3, ..., 10. Pongo 11 porque el final se EXCLUYE.
# Si pusiera range(1, 10) me faltaría el 8 x 10 (error típico).
# ------------------------------------------------------------
for i in range(1, 11):
    resultado = numero * i
    print(f"{numero} x {i} = {resultado}")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# Tabla del 8:
# 8 x 1 = 8
# 8 x 2 = 16
# ...
# 8 x 10 = 80
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - range(10) empieza en 0 y llega a 9: saldría 8 x 0 y faltaría 8 x 10.
# EN JAVA SERÍA: for (int i = 1; i <= 10; i++) { ... }
```

02_bucles/04_cuenta_atras_pares.py

```
"""
TEMA 4 - Ejercicio 4: Bucle for con paso personalizado
======================================================
ENUNCIADO (literal del profesor):
    Crea una cuenta regresiva que imprima solamente los números pares desde el 20 hasta el 0.

FORMA QUE EXIGE EL PROFESOR:
    - range con TERCER argumento (paso): range(inicio, fin, paso). Paso negativo.

CONCEPTOS QUE PRACTICA:
    - range(inicio, fin, paso)
    - Paso negativo para contar hacia atrás (el inicio tiene que ser MAYOR que el fin).

CÓMO EJECUTARLO:  python 04_cuenta_atras_pares.py   (en Linux/Docker: python3 04_cuenta_atras_pares.py)
"""

# ------------------------------------------------------------
# PASO 1: el range con paso
#   inicio = 20  -> el primer número que sale
#   fin    = -1  -> el final NO se incluye; para que el 0 SÍ salga,
#                   tengo que parar "uno más allá", o sea en -1
#   paso   = -2  -> resta 2 en cada vuelta: 20, 18, 16, ..., 0
# Como empiezo en un par y salto de 2 en 2, solo salen pares:
# no hace falta ningún if.
# ------------------------------------------------------------
for numero in range(20, -1, -2):
    print(numero)

print("¡Despegue!")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# 20
# 18
# 16
# 14
# 12
# 10
# 8
# 6
# 4
# 2
# 0
# ¡Despegue!
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - range(20, 0, -2) NO imprime el 0 (el fin se excluye).
# - range(0, 20, -2) no imprime NADA: con paso negativo el inicio debe ser mayor.
# EN JAVA SERÍA: for (int i = 20; i >= 0; i -= 2) { ... }
# ALTERNATIVA (funciona, pero NO es lo que pide: no usa el paso personalizado):
#     for numero in range(20, -1, -1):
#         if numero % 2 == 0:
#             print(numero)
```

02_bucles/05_tareas_enumerate.py

```
"""
TEMA 4 - Ejercicio 5: Bucle for con enumerate()
===============================================
ENUNCIADO (literal del profesor):
    Tienes una lista de tareas pendientes: ["Comprar leche", "Llamar al médico", "Estudiar Python"].
    Imprímelas numeradas empezando desde el 1 para mostrarlas como una lista ordenada.

FORMA QUE EXIGE EL PROFESOR:
    - enumerate(tareas, start=1)

CONCEPTOS QUE PRACTICA:
    - enumerate() da en cada vuelta DOS cosas: (posición, elemento).
    - start=1 hace que la numeración empiece en 1 en vez de en 0.
    - Desempaquetado: "for numero, tarea in ..." recoge las dos cosas a la vez.

CÓMO EJECUTARLO:  python 05_tareas_enumerate.py   (en Linux/Docker: python3 05_tareas_enumerate.py)
"""

# ------------------------------------------------------------
```

```python
# PASO 1: los datos
# ------------------------------------------------------------------
tareas = ["Comprar leche", "Llamar al médico", "Estudiar Python"]

# ------------------------------------------------------------------
# PASO 2: recorrer con enumerate
# enumerate(tareas, start=1) produce: (1, "Comprar leche"), (2, "Llamar..."), ...
# Con dos variables en el for, Python separa cada pareja:
#   numero <- 1, 2, 3      tarea <- el texto
# Sin start=1 empezaría en 0 (como los índices) y saldría "0. Comprar leche".
# ------------------------------------------------------------------
print("Tareas pendientes:")
for numero, tarea in enumerate(tareas, start=1):
    print(f"{numero}. {tarea}")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Tareas pendientes:
# 1. Comprar leche
# 2. Llamar al médico
# 3. Estudiar Python
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - El orden es (número, elemento): "for tarea, numero in ..." los cambia de sitio.
# ALTERNATIVAS (funcionan, pero NO son lo que pide el enunciado):
#     for i in range(len(tareas)):
#         print(f"{i + 1}. {tareas[i]}")
#     # o bien enumerate(tareas) sin start y luego {i + 1}
```

02_bucles/06_combinaciones_anidados.py

```python
"""
TEMA 4 - Ejercicio 6: Bucles for anidados
=========================================
ENUNCIADO (literal del profesor):
    Una tienda de ropa ofrece camisetas en dos tallas (M, L) y tres colores (Rojo, Azul, Negro).
    Genera y muestra todas las combinaciones posibles.

FORMA QUE EXIGE EL PROFESOR:
    - Bucles for ANIDADOS (un for dentro de otro).

CONCEPTOS QUE PRACTICA:
    - Por cada vuelta del bucle de fuera, el de dentro da TODAS sus vueltas.
    - Nº total de combinaciones = 2 tallas x 3 colores = 6.

CÓMO EJECUTARLO:  python 06_combinaciones_anidados.py   (en Linux/Docker: python3 06_combinaciones_anidados.py)
"""

# ------------------------------------------------------------------
# PASO 1: los datos en dos listas
# ------------------------------------------------------------------
tallas = ["M", "L"]
colores = ["Rojo", "Azul", "Negro"]

# ------------------------------------------------------------------
# PASO 2: bucles anidados
# Bucle de FUERA: tallas (2 vueltas).
# Bucle de DENTRO: colores (3 vueltas por cada talla).
# Fíjate en la sangría: el for de colores está DENTRO del de tallas.
# Llevo un contador para numerar y comprobar que salen 6.
# ------------------------------------------------------------------
contador = 0
for talla in tallas:
    for color in colores:
        contador += 1
        print(f"{contador}. Camiseta talla {talla} - color {color}")

print(f"Total de combinaciones: {contador}")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# 1. Camiseta talla M - color Rojo
# 2. Camiseta talla M - color Azul
# 3. Camiseta talla M - color Negro
# 4. Camiseta talla L - color Rojo
# 5. Camiseta talla L - color Azul
# 6. Camiseta talla L - color Negro
# Total de combinaciones: 6
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Si el segundo for no tiene sangría, NO está anidado: se ejecuta después
#   y solo combina la última talla.
# - Usar la misma variable en los dos for (for x ... for x ...) machaca la de fuera.
# ALTERNATIVA (funciona, pero NO es lo que pide: comprensión en vez de bucles anidados):
#     combinaciones = [(t, c) for t in tallas for c in colores]
```

02_bucles/07_cubos_comprension.py

```python
"""
TEMA 4 - Ejercicio 7: List comprehensions (básico)
==================================================
ENUNCIADO (literal del profesor):
    A partir de la lista numeros = [...], genera una nueva lista que contenga el cubo
    de cada número en una sola línea de código.
    (La lista salió cortada en la fuente: uso numeros = [1, 2, 3, 4, 5].)

FORMA QUE EXIGE EL PROFESOR:
```

```
    - Comprensión de listas en UNA SOLA LÍNEA.

CONCEPTOS QUE PRACTICA:
    - Sintaxis: [expresion for elemento in iterable]
    - Potencia con ** (n ** 3 = n al cubo).

CÓMO EJECUTARLO:  python 07_cubos_comprension.py   (en Linux/Docker: python3 07_cubos_comprension.py)
"""

numeros = [1, 2, 3, 4, 5]

# ------------------------------------------------------------
# LA LÍNEA QUE PIDE EL EJERCICIO
# Se lee de izquierda a derecha así:
#   "n ** 3"        -> QUÉ meto en la lista nueva (el cubo)
#   "for n in numeros" -> DE DÓNDE saco cada n
# Los corchetes [] indican que el resultado es una lista nueva.
# La lista original 'numeros' no cambia.
# ------------------------------------------------------------
cubos = [n ** 3 for n in numeros]

print(f"Números: {numeros}")
print(f"Cubos:   {cubos}")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# Números: [1, 2, 3, 4, 5]
# Cubos:   [1, 8, 27, 64, 125]
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - n ^ 3 NO es potencia en Python (es XOR a nivel de bits). Usa n ** 3.
# - Con paréntesis ( ) en vez de [ ] no obtienes una lista sino un generador.
# ALTERNATIVA (funciona, pero NO es lo que pide: no es una sola línea):
#     cubos = []
#     for n in numeros:
#         cubos.append(n ** 3)
```

02_bucles/08_filtrar_palabras_comprension.py

```
"""
TEMA 4 - Ejercicio 8: List comprehensions con condicional
============================================================
ENUNCIADO (literal del profesor):
    Dada la lista palabras = ["sol", "murciélago", "mar", "elefante", "luz"], usa una list
    comprehension para filtrar y guardar en una nueva lista solo las palabras que tengan
    más de 3 letras.

FORMA QUE EXIGE EL PROFESOR:
    - List comprehension con if (filtro), en una línea.

CONCEPTOS QUE PRACTICA:
    - Sintaxis con filtro: [expresion for elemento in iterable if condicion]
    - len(texto) devuelve el número de caracteres.

CÓMO EJECUTARLO:  python 08_filtrar_palabras_comprension.py   (en Linux/Docker: python3 08_filtrar_palabras_comprension.py)
"""

palabras = ["sol", "murciélago", "mar", "elefante", "luz"]

# ------------------------------------------------------------
# LA LÍNEA QUE PIDE EL EJERCICIO
#   "p"                -> QUÉ guardo (la palabra tal cual)
#   "for p in palabras" -> recorro todas
#   "if len(p) > 3"     -> FILTRO: solo entra si tiene más de 3 letras
# El if va AL FINAL cuando sirve para filtrar (quitar elementos).
# ------------------------------------------------------------
largas = [p for p in palabras if len(p) > 3]

print(f"Palabras de más de 3 letras: {largas}")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# Palabras de más de 3 letras: ['murciélago', 'elefante']
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - "más de 3" es > 3 (no >= 3; con >= 3 entrarían sol, mar y luz).
# - El if de FILTRO va al final y SIN else. Si pones if-else, va DELANTE del for
#   y sirve para transformar, no para filtrar:
#     [p.upper() if len(p) > 3 else p for p in palabras]
# ALTERNATIVA (funciona, pero NO es lo que pide: no es comprensión):
#     largas = []
#     for p in palabras:
#         if len(p) > 3:
#             largas.append(p)
```

02_bucles/09_contrasena_while.py

```
"""
TEMA 4 - Ejercicio 9: Bucle while básico
========================================
ENUNCIADO (literal del profesor):
    Pide al usuario que ingrese una contraseña. El programa debe seguir denegando el acceso
    y preguntando de nuevo hasta que el usuario escriba exactamente 'python123'.

FORMA QUE EXIGE EL PROFESOR:
    - Bucle while con condición (no sabemos cuántas veces se repetirá).
```

```
CONCEPTOS QUE PRACTICA:
    - while condicion: se repite MIENTRAS la condición sea True.
    - Cuándo usar while y no for: cuando NO sé de antemano cuántas vueltas habrá.
    - Cómo evitar un bucle infinito: la variable de la condición debe cambiar DENTRO del bucle.

CÓMO EJECUTARLO:  python 09_contrasena_while.py   (en Linux/Docker: python3 09_contrasena_while.py)
"""

# ------------------------------------------------------------------
# PASO 1: pedir la contraseña UNA vez ANTES del bucle
# El while comprueba la condición al principio, así que la variable
# 'contrasena' tiene que existir ya antes de llegar a él.
# ------------------------------------------------------------------
CONTRASENA_CORRECTA = "python123"   # en MAYÚSCULAS porque es una constante

contrasena = input("Introduce la contraseña: ")

# ------------------------------------------------------------------
# PASO 2: el bucle while
# "Mientras lo escrito sea DISTINTO de la correcta, deniego y vuelvo a pedir".
# 'exactamente' -> comparo con != tal cual, sin .lower() ni .strip():
# "Python123" o "python123 " (con espacio) NO valen.
#
# ¿CÓMO EVITO EL BUCLE INFINITO? Dentro del bucle vuelvo a hacer input(),
# así 'contrasena' cambia y en algún momento la condición puede ser False.
# Si olvidara el input de dentro, preguntaría... nunca: repetiría
# "Acceso denegado" para siempre (Ctrl + C para cortarlo).
# ------------------------------------------------------------------
while contrasena != CONTRASENA_CORRECTA:
    print("Acceso denegado.")
    contrasena = input("Introduce la contraseña: ")

# ------------------------------------------------------------------
# PASO 3: si llego aquí es porque la condición del while fue False,
# o sea, la contraseña es la correcta.
# ------------------------------------------------------------------
print("Acceso concedido. ¡Bienvenida!")

# ------------------------------------------------------------------
# SALIDA ESPERADA (escribiendo: hola, Python123, python123):
# Introduce la contraseña: Acceso denegado.
# Introduce la contraseña: Acceso denegado.
# Introduce la contraseña: Acceso concedido. ¡Bienvenida!
# (En la consola real verás lo que escribes detrás de cada "contraseña: ")
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Usar == en vez de != en la condición: el bucle no se ejecutaría nunca con
#   contraseñas malas.
# - Olvidar el input() dentro del bucle -> bucle infinito.
# EN JAVA SERÍA: while (!contrasena.equals("python123")) { ... }
#   (en Python los strings se comparan con == / != directamente)
# ALTERNATIVA (funciona, pero aquí piden while básico con condición):
#     while True:
#         contrasena = input("Introduce la contraseña: ")
#         if contrasena == CONTRASENA_CORRECTA:
#             break
#         print("Acceso denegado.")
```

02_bucles/10_contador_acumulador_while.py

```
"""
TEMA 4 - Ejercicio 10: Contador y acumulador en while
=====================================================
ENUNCIADO (literal del profesor):
    Escribe un programa que pida números al usuario sin parar y los vaya sumando.
    El bucle debe terminar cuando la suma total (acumulador) supere el valor de 50.
    Muestra cuántos números se ingresaron (contador).

FORMA QUE EXIGE EL PROFESOR:
    - Bucle while con una variable CONTADOR y otra ACUMULADOR.

CONCEPTOS QUE PRACTICA:
    - CONTADOR: cuenta CUÁNTAS veces pasa algo -> siempre suma 1   (contador += 1)
    - ACUMULADOR: guarda la SUMA de valores    -> suma el valor     (suma += numero)
    - Los dos se inicializan a 0 ANTES del bucle.
    - "supere 50" significa > 50, así que el bucle sigue MIENTRAS suma <= 50.

CÓMO EJECUTARLO:  python 10_contador_acumulador_while.py   (en Linux/Docker: python3 10_contador_acumulador_while.py)
"""

# ------------------------------------------------------------------
# PASO 1: inicializar contador y acumulador
# Van FUERA (antes) del bucle. Si estuvieran dentro, se pondrían a 0
# en cada vuelta y nunca llegaríamos a 50 -> bucle infinito.
# ------------------------------------------------------------------
LIMITE = 50
suma = 0       # ACUMULADOR: la suma de todos los números
contador = 0   # CONTADOR: cuántos números se han escrito

# ------------------------------------------------------------------
# PASO 2: el bucle
# Condición: seguir MIENTRAS la suma NO supere 50  ->  suma <= 50.
# (Pensar al revés es lo que más cuesta: el enunciado dice cuándo
#  PARAR -> suma > 50; el while necesita cuándo SEGUIR -> lo contrario.)
#
# ¿CÓMO EVITO EL BUCLE INFINITO? La variable de la condición (suma)
# cambia dentro del bucle en cada vuelta. Ojo: si el usuario solo
# mete ceros o negativos, la suma nunca supera 50 y el bucle sigue:
# es lo que pide el enunciado ("sin parar"), no es un fallo del código.
# ------------------------------------------------------------------
```

```python
while suma <= LIMITE:
    # int() convierte el texto de input() en número entero.
    # Si quieres admitir decimales (2.5), usa float() en su lugar.
    numero = int(input("Introduce un número: "))
    suma += numero      # acumulador: suma el VALOR
    contador += 1       # contador: suma 1, da igual qué número fuera
    print(f"  Suma parcial: {suma}")

# ------------------------------------------------------------------
# PASO 3: resultados (fuera del bucle)
# ------------------------------------------------------------------
print(f"La suma ({suma}) ha superado {LIMITE}.")
print(f"Has introducido {contador} números.")

# ------------------------------------------------------------------
# SALIDA ESPERADA (escribiendo 20, 15, 10, 8):
# Introduce un número:   Suma parcial: 20
# Introduce un número:   Suma parcial: 35
# Introduce un número:   Suma parcial: 45
# Introduce un número:   Suma parcial: 53
# La suma (53) ha superado 50.
# Has introducido 4 números.
# Caso límite: si la suma llega EXACTAMENTE a 50, sigue pidiendo (50 no "supera" 50).
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Confundir contador y acumulador: contador += numero estaría MAL.
# - while suma < 50 pararía al llegar a 50 justo, y el enunciado dice "supere".
# - Si escriben letras, int() da ValueError (ver ejercicio 12 para controlarlo).
```

02_bucles/11_login_cajero_while.py

```python
"""
TEMA 4 - Ejercicio 11: Bucle while con condición compleja
============================================================
ENUNCIADO (literal del profesor):
    Simula el login de un cajero. El usuario debe ingresar su PIN ('1234'). El bucle termina
    cuando acierta el PIN o cuando se superan los 3 intentos máximos permitidos.

FORMA QUE EXIGE EL PROFESOR:
    - while con una condición COMPUESTA (dos condiciones unidas con and).

CONCEPTOS QUE PRACTICA:
    - Operadores lógicos en la condición del while: and / or / not.
    - Contador de intentos para garantizar que el bucle termina.
    - Después del bucle, averiguar POR QUÉ terminó (¿acertó o se quedó sin intentos?).

CÓMO EJECUTARLO:  python 11_login_cajero_while.py   (en Linux/Docker: python3 11_login_cajero_while.py)
"""

# ------------------------------------------------------------------
# PASO 1: datos y variables de control
# El PIN se compara como TEXTO ('1234' entre comillas, como en el
# enunciado), así no hace falta int() y no falla si escriben letras.
# ------------------------------------------------------------------
PIN_CORRECTO = "1234"
MAX_INTENTOS = 3

intentos = 0     # CONTADOR de intentos usados
pin = ""         # vacío para que la primera vez la condición sea True

# ------------------------------------------------------------------
# PASO 2: la condición compleja
# El enunciado dice cuándo TERMINA: "acierta  O  se acaban los intentos".
# El while necesita cuándo SIGUE, que es lo contrario:
#     NO acierta  Y  le quedan intentos
#     pin != PIN_CORRECTO  and  intentos < MAX_INTENTOS
# (Regla de De Morgan: lo contrario de "A or B" es "no A and no B".)
#
# ¿CÓMO EVITO EL BUCLE INFINITO? 'intentos' sube 1 en cada vuelta, así
# que como mucho habrá 3 vueltas aunque nunca acierte.
# ------------------------------------------------------------------
while pin != PIN_CORRECTO and intentos < MAX_INTENTOS:
    pin = input("Introduce tu PIN: ")
    intentos += 1
    if pin != PIN_CORRECTO:
        restantes = MAX_INTENTOS - intentos
        print(f"PIN incorrecto. Intentos restantes: {restantes}")

# ------------------------------------------------------------------
# PASO 3: ¿por qué ha salido del bucle?
# Hay dos motivos posibles, así que lo compruebo con un if.
# ------------------------------------------------------------------
if pin == PIN_CORRECTO:
    print(f"PIN correcto. Acceso concedido (intentos usados: {intentos}).")
else:
    print("Has superado los 3 intentos. Tarjeta bloqueada.")

# ------------------------------------------------------------------
# SALIDA ESPERADA 1 (escribiendo 1111, 1234):
# Introduce tu PIN: PIN incorrecto. Intentos restantes: 2
# Introduce tu PIN: PIN correcto. Acceso concedido (intentos usados: 2).
#
# SALIDA ESPERADA 2 (escribiendo 0000, 1111, 2222):
# Introduce tu PIN: PIN incorrecto. Intentos restantes: 2
# Introduce tu PIN: PIN incorrecto. Intentos restantes: 1
# Introduce tu PIN: PIN incorrecto. Intentos restantes: 0
# Has superado los 3 intentos. Tarjeta bloqueada.
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner "or" en la condición del while: con un PIN malo seguiría para siempre
```

```
#   (porque pin != PIN_CORRECTO es True aunque se acaben los intentos).
# - intentos <= 3 daría 4 oportunidades.
# - Comparar con el número 1234 (sin comillas): input() devuelve texto y "1234" != 1234.
# EN JAVA SERÍA: while (!pin.equals("1234") && intentos < 3) { ... }
```

02_bucles/12_calculadora_while_true_break.py

```
"""
TEMA 4 - Ejercicio 12: Bucle while True con break
==================================================
ENUNCIADO (literal del profesor):
    Crea un bucle infinito que funcione como una calculadora de sumas continuas.
    El programa pide un número. Si el usuario escribe 'salir', el bucle se interrumpe
    con break y muestra el total.

FORMA QUE EXIGE EL PROFESOR:
    - while True (bucle infinito a propósito) + break para salir.

CONCEPTOS QUE PRACTICA:
    - while True: la condición siempre es verdadera; la ÚNICA salida es un break.
    - break: corta el bucle en ese momento y salta a la primera línea de después.
    - Comprobar 'salir' ANTES de convertir a número (si no, int("salir") revienta).
    - Acumulador (total).

CÓMO EJECUTARLO:  python 12_calculadora_while_true_break.py   (en Linux/Docker: python3 12_calculadora_while_true_break.py)
"""

# ------------------------------------------------------------------
# PASO 1: el acumulador, antes del bucle
# ------------------------------------------------------------------
total = 0

print("Calculadora de sumas. Escribe 'salir' para terminar.")

# ------------------------------------------------------------------
# PASO 2: el bucle infinito
# while True no termina solo. ¿CÓMO EVITO QUE SEA INFINITO DE VERDAD?
# Tiene que haber un break que se pueda alcanzar: aquí, cuando escriben 'salir'.
# ------------------------------------------------------------------
while True:
    # Leo como TEXTO. .strip() quita espacios sobrantes y .lower()
    # permite que "SALIR" o " Salir " también funcionen.
    entrada = input("Número (o 'salir'): ").strip().lower()

    # 1º) ¿Quiere salir? Esto va PRIMERO: si hiciera int(entrada) antes,
    #     int("salir") daría ValueError y el programa se pararía con error.
    if entrada == "salir":
        break   # sale del while; continúa en el print del PASO 3

    # 2º) No es 'salir' -> lo trato como número y lo acumulo.
    numero = float(entrada)   # float para admitir también decimales (2.5)
    total += numero
    print(f"  Total parcial: {total}")

# ------------------------------------------------------------------
# PASO 3: después del break llegamos aquí
# ------------------------------------------------------------------
print(f"Total final: {total}")

# ------------------------------------------------------------------
# SALIDA ESPERADA (escribiendo 10, 5.5, 4.5, salir):
# Calculadora de sumas. Escribe 'salir' para terminar.
# Número (o 'salir'):   Total parcial: 10.0
# Número (o 'salir'):   Total parcial: 15.5
# Número (o 'salir'):   Total parcial: 20.0
# Número (o 'salir'): Total final: 20.0
# Caso límite: si escribe 'salir' a la primera -> Total final: 0
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner el int()/float() antes del if 'salir' -> ValueError al escribir salir.
# - Olvidar el break -> bucle infinito de verdad (Ctrl + C para pararlo).
# - El break solo sale del bucle MÁS CERCANO (si hay bucles anidados, solo del de dentro).
#
# OPCIONAL (si quieres que NO se rompa cuando escriben letras que no son 'salir'):
#
# OPCIÓN A: .isdigit() -> True si TODOS los caracteres son dígitos 0-9.
#   Limitación: "-5" y "2.5" dan False (el signo y el punto no son dígitos),
#   así que solo sirve para enteros positivos.
#
#     if entrada.isdigit():
#         total += int(entrada)
#     else:
#         print("Eso no es un número. Prueba otra vez.")
#         continue   # vuelve al principio del while sin sumar
#
# OPCIÓN B: try / except -> intento convertir y, si falla, capturo el error.
#   Funciona con negativos y decimales.
#
#     try:
#         numero = float(entrada)
#     except ValueError:
#         print("Eso no es un número. Prueba otra vez.")
#         continue
#     total += numero
#
# ALTERNATIVA (funciona, pero NO es lo que pide: no usa while True + break):
#     entrada = input(...)
#     while entrada != "salir":
```

```
#         total += float(entrada)
#         entrada = input(...)
```

02_bucles/13_saltar_multiplos_continue.py

```
"""
TEMA 4 - Ejercicio 13: Uso de continue
======================================
ENUNCIADO (literal del profesor):
    Itera los números del 1 al 10 e imprímelos. Sin embargo, si el número es divisible
    por 3 (ej. 3, 6, 9), utiliza continue para saltarlo y no imprimirlo.

FORMA QUE EXIGE EL PROFESOR:
    - Usar continue.

CONCEPTOS QUE PRACTICA:
    - continue: se salta LO QUE QUEDA de la vuelta actual y pasa a la siguiente.
      (break, en cambio, sale del bucle entero.)
    - Divisible por 3  <=>  numero % 3 == 0  (el resto de dividir entre 3 es 0).

CÓMO EJECUTARLO:  python 13_saltar_multiplos_continue.py   (en Linux/Docker: python3 13_saltar_multiplos_continue.py)
"""

# ------------------------------------------------------------------
# range(1, 11) -> del 1 al 10 (el 11 no se incluye).
# Si el número es divisible por 3, continue hace que el print de abajo
# NO se ejecute en esa vuelta; el bucle sigue con el siguiente número.
# ------------------------------------------------------------------
for numero in range(1, 11):
    if numero % 3 == 0:
        continue          # salta el 3, el 6 y el 9
    print(numero)

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# 1
# 2
# 4
# 5
# 7
# 8
# 10
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner break en vez de continue: imprimiría 1, 2 y pararía en el 3.
# - Poner el print ANTES del if: imprimiría también los múltiplos de 3.
# - En un while, si el continue va antes de actualizar el contador, se crea un
#   bucle infinito (el contador no cambia nunca). En el for no pasa porque
#   range avanza solo.
# ALTERNATIVA (funciona, pero NO es lo que pide: no usa continue):
#     for numero in range(1, 11):
#         if numero % 3 != 0:
#             print(numero)
```

02_bucles/14_buscar_99_for_else.py

```
"""
TEMA 4 - Ejercicio 14: Cláusula else en bucles
==============================================
ENUNCIADO (literal del profesor):
    Busca el número 99 en la lista [...]. Si lo encuentras, imprime '¡Encontrado!' y sal
    del bucle. Si terminas de recorrer la lista y no está, usa el bloque else del bucle
    para imprimir 'Número no encontrado'.
    (La lista salió cortada en la fuente: uso [12, 45, 99, 7, 30] y pruebo también
    una lista sin el 99 para ver el else.)

FORMA QUE EXIGE EL PROFESOR:
    - for ... else (el else va pegado al FOR, no al if) + break.

CONCEPTOS QUE PRACTICA:
    - El else de un bucle se ejecuta SOLO si el bucle terminó "normalmente",
      es decir, SIN haber hecho break.
    - Si hay break, el else se salta.
    - Truco para recordarlo: piensa en el else del bucle como "si no hubo break".

CÓMO EJECUTARLO:  python 14_buscar_99_for_else.py   (en Linux/Docker: python3 14_buscar_99_for_else.py)
"""

# ------------------------------------------------------------------
# CASO 1: la lista SÍ contiene el 99
# ------------------------------------------------------------------
numeros = [12, 45, 99, 7, 30]
buscado = 99

for numero in numeros:
    if numero == buscado:
        print("¡Encontrado!")
        break              # salgo: no hace falta mirar el resto (7 y 30)
else:
    # ATENCIÓN A LA SANGRÍA: este else está a la altura del 'for',
    # NO del 'if'. Solo se ejecuta si el for llegó al final sin break.
    print("Número no encontrado")

# ------------------------------------------------------------------
# CASO 2 (prueba extra): una lista SIN el 99 -> el bucle no hace break,
# termina de recorrerla entera y entonces SÍ entra en el else.
# ------------------------------------------------------------------
```

```python
otros_numeros = [1, 2, 3, 4, 5]

for numero in otros_numeros:
    if numero == buscado:
        print("¡Encontrado!")
        break
else:
    print("Número no encontrado")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# ¡Encontrado!
# Número no encontrado
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner el else a la altura del if (if/else normal): imprimiría
#   "Número no encontrado" por CADA número que no sea 99. ¡Eso NO es lo que piden!
# - Olvidar el break: el else se ejecutaría SIEMPRE (el bucle nunca se interrumpe).
# - El while también admite else, con la misma regla.
# EN JAVA: no existe; habría que usar una variable boolean encontrado = false.
# ALTERNATIVAS (funcionan, pero NO son lo que pide: no usan el else del bucle):
#     if 99 in numeros:
#         print("¡Encontrado!")
#     else:
#         print("Número no encontrado")
#     # o una bandera: encontrado = False ... if not encontrado: ...
```

02_bucles/15_inventario_dict_items.py

```python
"""
TEMA 4 - Ejercicio 15: Bucle for con diccionarios
=================================================
ENUNCIADO (literal del profesor):
    Tienes el inventario de una tienda de frutas en un diccionario:
    {"manzanas": 50, "peras": 30, "naranjas": 20}. Itera sobre el diccionario e imprime
    tanto la clave como el valor en una frase descriptiva.

FORMA QUE EXIGE EL PROFESOR:
    - Recorrer el diccionario con .items() para tener clave y valor a la vez.

CONCEPTOS QUE PRACTICA:
    - .items()  -> parejas (clave, valor)   -> for clave, valor in d.items()
    - .keys()   -> solo las claves          (es lo que da "for clave in d")
    - .values() -> solo los valores

CÓMO EJECUTARLO:  python 15_inventario_dict_items.py   (en Linux/Docker: python3 15_inventario_dict_items.py)
"""

# ------------------------------------------------------------------
# PASO 1: el diccionario  { clave: valor, ... }
# clave = nombre de la fruta, valor = unidades en stock.
# ------------------------------------------------------------------
inventario = {"manzanas": 50, "peras": 30, "naranjas": 20}

# ------------------------------------------------------------------
# PASO 2: recorrer con .items()
# Cada vuelta da una pareja ("manzanas", 50) y el for la separa en
# dos variables (desempaquetado), igual que con enumerate.
# ------------------------------------------------------------------
print("Inventario de la frutería:")
for fruta, cantidad in inventario.items():
    print(f"Tenemos {cantidad} unidades de {fruta}.")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Inventario de la frutería:
# Tenemos 50 unidades de manzanas.
# Tenemos 30 unidades de peras.
# Tenemos 20 unidades de naranjas.
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - "for fruta, cantidad in inventario:" (sin .items()) da ValueError: recorrer
#   un diccionario directamente solo da las CLAVES.
# - .items lleva paréntesis: inventario.items()
# EN JAVA SERÍA: for (Map.Entry<String,Integer> e : inventario.entrySet()) { ... }
# ALTERNATIVA (funciona, pero NO es la forma recomendada; mejor .items()):
#     for fruta in inventario:
#         print(f"Tenemos {inventario[fruta]} unidades de {fruta}.")
```

02_bucles/16_resumen_bucles.py

```python
"""
TEMA 4 - Resumen: CHULETA DE BUCLES (ejecutable)
================================================
Cada bloque es un ejemplo mínimo y comentado. Ejecuta el archivo y compara
lo que sale con lo que pone en el comentario "->".

CÓMO EJECUTARLO:  python 16_resumen_bucles.py   (en Linux/Docker: python3 16_resumen_bucles.py)

=====================================================================
TABLA "SI ME PIDEN X  ->  USO Y"
=====================================================================
 Si me piden...                                  -> Uso...
 -----------------------------------------------------------------
 Recorrer una lista / string elemento a elemento -> for x in lista:
 Repetir N veces / números de A a B              -> for i in range(A, B + 1):
 Contar hacia atrás o saltando (pares, de 5 en 5) -> range(inicio, fin, paso)  (paso negativo = atrás)
```

```
 Numerar elementos empezando en 1               -> for n, x in enumerate(lista, start=1):
 Recorrer dos listas a la vez (en paralelo)     -> for a, b in zip(lista1, lista2):
 Clave y valor de un diccionario                -> for k, v in d.items():
 Solo claves / solo valores                     -> d.keys()  /  d.values()
 Todas las combinaciones de dos listas          -> for anidados (for dentro de for)
 Sumar sin sum()                                -> acumulador: total = 0 ; total += x
 Contar cuántos cumplen algo                    -> contador: c = 0 ; c += 1
 Repetir sin saber cuántas veces (hasta que...) -> while condicion:
 Repetir "para siempre" hasta que escriban algo -> while True: ... if ...: break
 Salir del bucle antes de tiempo                -> break
 Saltarse un elemento y seguir                  -> continue
 "Si no se encuentra..." al acabar de buscar    -> else del bucle (for/while ... else)
 Crear lista nueva en UNA línea                 -> [expr for x in lista]
 ... filtrando                                  -> [expr for x in lista if cond]        (if al FINAL)
 ... transformando según condición              -> [a if cond else b for x in lista]    (if-else DELANTE)
 Crear diccionario en UNA línea                 -> {clave: valor for x in lista}
=====================================================================
CONTADOR vs ACUMULADOR
 - Contador:   cuenta CUÁNTAS veces pasa algo -> suma siempre 1.
 - Acumulador: guarda la SUMA de valores      -> suma el valor.
 - Los dos se inicializan (normalmente a 0) ANTES del bucle.
EVITAR BUCLES INFINITOS EN WHILE
 - La variable de la condición tiene que CAMBIAR dentro del bucle.
 - En while True tiene que haber un break alcanzable.
 - Si se cuelga: Ctrl + C en la terminal.
=====================================================================
"""

# =====================================================================
# 1. FOR SOBRE UNA LISTA
# =====================================================================
frutas = ["manzana", "pera", "uva"]
for fruta in frutas:
    print(fruta)                    # -> manzana / pera / uva (una por línea)

# =====================================================================
# 2. FOR SOBRE UN STRING (letra a letra)
# =====================================================================
for letra in "hola":
    print(letra, end=" ")           # end=" " -> no salta de línea: h o l a
print()                             # salto de línea final

# =====================================================================
# 3. FOR CON RANGE
#    range(fin)             -> 0 .. fin-1
#    range(inicio, fin)     -> inicio .. fin-1   (el fin NUNCA se incluye)
#    range(inicio, fin, paso)
# =====================================================================
print(list(range(5)))               # -> [0, 1, 2, 3, 4]
print(list(range(1, 6)))            # -> [1, 2, 3, 4, 5]
print(list(range(0, 11, 5)))        # -> [0, 5, 10]
print(list(range(10, 0, -3)))       # -> [10, 7, 4, 1]   (hacia atrás)

# =====================================================================
# 4. ENUMERATE (posición + elemento)
# =====================================================================
for numero, fruta in enumerate(frutas, start=1):
    print(f"{numero}. {fruta}")     # -> 1. manzana / 2. pera / 3. uva

# =====================================================================
# 5. ZIP (dos listas en paralelo; para en la más corta)
# =====================================================================
nombres = ["Ana", "Luis"]
notas = [8, 6]
for nombre, nota in zip(nombres, notas):
    print(f"{nombre}: {nota}")      # -> Ana: 8 / Luis: 6

# =====================================================================
# 6. DICCIONARIOS
# =====================================================================
stock = {"peras": 30, "uvas": 12}
for clave, valor in stock.items():  # clave y valor
    print(f"{clave} -> {valor}")    # -> peras -> 30 / uvas -> 12
for clave in stock.keys():          # solo claves (igual que: for clave in stock)
    print(clave)                    # -> peras / uvas
total_stock = 0
for valor in stock.values():        # solo valores
    total_stock += valor            # ACUMULADOR
print(total_stock)                  # -> 42

# =====================================================================
# 7. WHILE (se repite MIENTRAS la condición sea True)
# =====================================================================
contador = 3
while contador > 0:
    print(contador, end=" ")
    contador -= 1                   # ¡la condición cambia! si no, bucle infinito
print("¡Ya!")                       # -> 3 2 1 ¡Ya!

# =====================================================================
# 8. BREAK (salir del bucle) -- típico con while True
# =====================================================================
n = 0
while True:                         # infinito a propósito...
    n += 1
    if n == 4:
        break                       # ...la única salida es el break
print(f"Salí con n = {n}")          # -> Salí con n = 4

# =====================================================================
# 9. CONTINUE (saltar a la siguiente vuelta)
# =====================================================================
```

```python
for i in range(1, 6):
    if i % 2 == 0:
        continue                    # los pares no se imprimen
    print(i, end=" ")               # -> 1 3 5
print()

# ======================================================================
# 10. ELSE DEL BUCLE (solo se ejecuta si NO hubo break)
#     El else va a la altura del FOR, no del if.
# ======================================================================
for x in [1, 3, 5]:
    if x == 4:
        print("Encontrado")
        break
else:
    print("No encontrado")           # -> No encontrado (no hubo break)

# ======================================================================
# 11. BUCLES ANIDADOS (por cada vuelta de fuera, todas las de dentro)
# ======================================================================
for fila in range(1, 3):
    for col in ["A", "B"]:
        print(f"{fila}{col}", end=" ")  # -> 1A 1B 2A 2B
print()

# ======================================================================
# 12. COMPRENSIONES (bucle + lista nueva en UNA línea)
# ======================================================================
numeros = [1, 2, 3, 4, 5]
# a) Básica:  [expresion for x in iterable]
cuadrados = [x ** 2 for x in numeros]
print(cuadrados)                    # -> [1, 4, 9, 16, 25]
# b) Con if (FILTRO, va al FINAL):  [x for x in iterable if cond]
pares = [x for x in numeros if x % 2 == 0]
print(pares)                        # -> [2, 4]
# c) Con if-else (TRANSFORMA, va DELANTE del for):  [a if cond else b for x in iterable]
etiquetas = ["par" if x % 2 == 0 else "impar" for x in numeros]
print(etiquetas)                    # -> ['impar', 'par', 'impar', 'par', 'impar']
# d) De diccionario:  {clave: valor for x in iterable}
dobles = {x: x * 2 for x in numeros}
print(dobles)                       # -> {1: 2, 2: 4, 3: 6, 4: 8, 5: 10}
#    ...o a partir de otro diccionario con .items():
stock_doble = {fruta: cant * 2 for fruta, cant in stock.items()}
print(stock_doble)                  # -> {'peras': 60, 'uvas': 24}

# ======================================================================
# 13. CONTADOR y ACUMULADOR juntos (sin sum() ni len())
# ======================================================================
precios = [10, 25, 5, 40]
total = 0          # acumulador
caros = 0          # contador
for p in precios:
    total += p                      # suma el VALOR
    if p > 20:
        caros += 1                  # suma 1
print(f"Total: {total}, caros: {caros}")  # -> Total: 80, caros: 2
```

5 · Funciones

03_funciones/00_INDICE.md

Tema 5 - Funciones (15 Ejercicios Funciones Python)

Archivo	Qué pide	Forma exigida	Conceptos clave	
01_primera_funcion.py	`saludar_mundo()` imprime "¡Hola, mundo de las funciones!"	Sin parámetros; imprime dentro	def, llamada con ()	
02_parametros_simples.py	`saludo_personalizado(nombre)` imprime saludo	Un parámetro `nombre`	parámetro vs argumento, f-string	
03_retornar_valor.py	`multiplicar(a, b)`	**return obligatorio, NADA de print dentro**	return vs print, None	
04_mayor_de_edad.py	`es_mayor_de_edad(edad)` -> True/False	`return edad >= 18` (if/else = menos bueno)	booleanos, función predicado	
05_contar_vocales.py	`contar_vocales(texto)` (may. y min.)	Devolver con return	for sobre str, `in`, contador	
06_parametros_por_defecto.py	`presentar_usuario(nombre, rol)`	`rol="Estudiante"`, formato `Usuario: x \`	Rol: y`, 2 llamadas	valores por defecto al final
07_retornar_multiples_valores.py	`obtener_extremos(lista)`	`return min(l), max(l)` + desempaquetar en 2 variables	tuplas, desempaquetado	
08_filtrar_pares.py	`filtrar_pares(lista)`	Iterar con for y retornar lista NUEVA	append, `%`	
09_promedio_args.py	`calcular_promedio(*args)`	** `*args` obligatorio**; controla 0 argumentos	args = tupla, sum/len	
10a_kwargs_perfil_diccionario.py	Variante A: `crear_perfil_usuario(nombre, apellido, **kwargs)`	`kwargs` **, RETORNA diccionario	kwargs = dict, update	
10b_kwargs_perfil_imprimir.py	Variante B: `crear_perfil_usuario(**kwargs)`	`kwargs` **, IMPRIME `Clave: valor` con capitalize	`.items()`, capitalize	
11_type_hints_celsius.py	`celsius_a_fahrenheit`	`(celsius: float) -> float` + docstring triple comilla	type hints, docstring	
12_decorador.py	decorador `anunciar_ejecucion` + `tarea_importante`	**Aplicar con `@anunciar_ejecucion`**	función que envuelve a otra	
13_map_lambda.py	Cuadrados de una lista	`list(map(lambda x: x 2, numeros))` **	lambda, map, list()	
14_orden_superior.py	`aplicar_operacion(a, b, funcion)` + sumar/restar	Función como 3er parámetro, `return funcion(a, b)`	funciones como valores	
15_recursividad_factorial.py	`calcular_factorial(n)` recursiva	**Recursiva + caso base 0 y 1**; traza de factorial(4)	caso base, caso recursivo	
16_resumen_funciones.py	Chuleta ejecutable de todo el tema	-	def, parámetros, return/None, *args, **kwargs, orden, global, hints, docstring, lambda+map/filter/sorted, orden superior, decorador, recursividad, mutable por defecto	

03_funciones/01_primera_funcion.py

```
"""
TEMA 5 - Ejercicio 1: Tu primera función
========================================
ENUNCIADO (literal del profesor):
    "Escribe una función llamada saludar_mundo que no reciba ningún parámetro
    y simplemente imprima en consola el mensaje '¡Hola, mundo de las funciones!'."

FORMA QUE EXIGE EL PROFESOR:
    - Función con nombre exacto saludar_mundo, SIN parámetros.
    - Aquí SÍ se imprime dentro de la función (lo pide el enunciado: "imprima").

CONCEPTOS QUE PRACTICA:
    - def, nombre en snake_case, paréntesis vacíos, dos puntos y sangría.
    - Diferencia entre DEFINIR una función y LLAMARLA.

CÓMO EJECUTARLO:  python 01_primera_funcion.py   (en Linux/Docker: python3 01_primera_funcion.py)
"""


# ------------------------------------------------------------------
# PASO 1: Definir la función
# ------------------------------------------------------------------
# "def" + nombre + () + ":"  ->  el cuerpo va sangrado 4 espacios.
# Definirla NO la ejecuta: solo la deja guardada para usarla después.
def saludar_mundo():
    """Imprime un saludo fijo por consola. No recibe parámetros."""
    print("¡Hola, mundo de las funciones!")


# ------------------------------------------------------------------
# PASO 2: Llamar a la función (pruebas)
# ------------------------------------------------------------------
# if __name__ == "__main__": hace que esto solo se ejecute cuando lanzas
# este archivo directamente (no si otro archivo lo importa).
if __name__ == "__main__":
    saludar_mundo()   # Los paréntesis son OBLIGATORIOS para ejecutarla

# ------------------------------------------------------------------
```

```
# SALIDA ESPERADA:
# ¡Hola, mundo de las funciones!
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Escribir "saludar_mundo" sin () -> no se ejecuta nada (solo nombras
#   la función, no la llamas).
# - Olvidar los ":" tras def o no sangrar el cuerpo -> SyntaxError /
#   IndentationError.
# EN JAVA SERÍA... public static void saludarMundo() { System.out.println(...); }
#   En Python no hay tipo de retorno ni llaves: manda la sangría.
```

03_funciones/02_parametros_simples.py

```
"""
TEMA 5 - Ejercicio 2: Parámetros simples
========================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada saludo_personalizado que reciba un parámetro
    nombre (cadena de texto) e imprima '¡Hola, [nombre]! Bienvenido al curso
    de Python'."

FORMA QUE EXIGE EL PROFESOR:
    - Función saludo_personalizado con UN parámetro llamado nombre.
    - Imprime dentro de la función (el enunciado dice "imprima").

CONCEPTOS QUE PRACTICA:
    - Parámetro (en la definición) vs argumento (el valor en la llamada).
    - f-strings para meter variables dentro del texto.

CÓMO EJECUTARLO:  python 02_parametros_simples.py   (en Linux/Docker: python3 02_parametros_simples.py)
"""


# ------------------------------------------------------------
# PASO 1: Definir la función con un parámetro
# ------------------------------------------------------------
# "nombre" es el PARÁMETRO: una variable que solo existe dentro de la
# función y que toma el valor que le pasemos al llamarla.
# ": str" es una pista de tipo (opcional): indica que se espera texto.
def saludo_personalizado(nombre: str):
    """Imprime un saludo de bienvenida con el nombre recibido."""
    # f-string: la f delante permite poner {variable} dentro del texto
    print(f"¡Hola, {nombre}! Bienvenido al curso de Python")


# ------------------------------------------------------------
# PASO 2: Pruebas
# ------------------------------------------------------------
if __name__ == "__main__":
    saludo_personalizado("Laura")   # "Laura" es el ARGUMENTO
    saludo_personalizado("Ana")     # Reutilizamos la misma función

# ------------------------------------------------------------
# SALIDA ESPERADA:
# ¡Hola, Laura! Bienvenido al curso de Python
# ¡Hola, Ana! Bienvenido al curso de Python
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Llamar saludo_personalizado() sin argumento -> TypeError: missing 1
#   required positional argument: 'nombre'.
# - Olvidar la f del f-string -> imprime literalmente "{nombre}".
# EN JAVA SERÍA... void saludoPersonalizado(String nombre). En Python el
#   tipo no es obligatorio (": str" es solo informativo).
```

03_funciones/03_retornar_valor.py

```
"""
TEMA 5 - Ejercicio 3: Retornar un valor
=======================================
ENUNCIADO (literal del profesor):
    "Desarrolla una función llamada multiplicar que reciba dos números como
    argumentos y retorne (usando return) el resultado de multiplicarlos. No
    debe imprimir el resultado dentro de la función."

FORMA QUE EXIGE EL PROFESOR:
    - OBLIGATORIO usar return.
    - PROHIBIDO poner print() dentro de la función. El print va FUERA,
      en el código que la llama.

CONCEPTOS QUE PRACTICA:
    - return: la función DEVUELVE un valor al que la llamó.
    - Diferencia return (devuelve, se puede guardar) vs print (solo muestra).

CÓMO EJECUTARLO:  python 03_retornar_valor.py   (en Linux/Docker: python3 03_retornar_valor.py)
"""


# ------------------------------------------------------------
# PASO 1: Definir la función que DEVUELVE el producto
# ------------------------------------------------------------
def multiplicar(a, b):
    """Devuelve el producto de a por b. No imprime nada."""
    # return termina la función y entrega el valor a quien la llamó.
    # NADA de print aquí: así el resultado se puede reutilizar.
    return a * b
```

```python
# ------------------------------------------------------------
# PASO 2: Pruebas (el print va FUERA de la función)
# ------------------------------------------------------------
if __name__ == "__main__":
    resultado = multiplicar(6, 7)          # Guardamos lo que devuelve
    print(f"6 x 7 = {resultado}")

    # Como devuelve un valor, podemos usarlo en otras operaciones:
    doble = multiplicar(3, 4) * 2
    print(f"(3 x 4) x 2 = {doble}")

    # También funciona con decimales
    print(f"2.5 x 4 = {multiplicar(2.5, 4)}")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# 6 x 7 = 42
# (3 x 4) x 2 = 24
# 2.5 x 4 = 10.0
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner print(a * b) dentro de la función en vez de return: "funciona"
#   en pantalla, pero la función devuelve None y PIERDES PUNTOS porque el
#   enunciado lo prohíbe.
#       def multiplicar(a, b):
#           print(a * b)         # MAL: devuelve None
#       x = multiplicar(2, 3)    # imprime 6, pero x vale None
# - Código después de return dentro de la función nunca se ejecuta.
# EN JAVA SERÍA... int multiplicar(int a, int b) { return a * b; }
```

03_funciones/04_mayor_de_edad.py

```python
"""
TEMA 5 - Ejercicio 4: Lógica condicional dentro de una función
============================================================
ENUNCIADO (literal del profesor):
    "Escribe una función llamada es_mayor_de_edad que reciba la edad de una
    persona como número entero y devuelva True si es mayor o igual a 18, y
    False en caso contrario."

FORMA QUE EXIGE EL PROFESOR:
    - Devolver (return) un booleano, no imprimirlo.
    - El profesor marca como MEJOR PRÁCTICA (pythónica) devolver directamente
      la comparación:  return edad >= 18
      en lugar de un if/else redundante.

CONCEPTOS QUE PRACTICA:
    - Una comparación (>=) YA es un valor True/False: se puede devolver tal cual.
    - Funciones "predicado" (nombre es_..., devuelven bool).

CÓMO EJECUTARLO:  python 04_mayor_de_edad.py   (en Linux/Docker: python3 04_mayor_de_edad.py)
"""


# ------------------------------------------------------------
# PASO 1: Forma pythónica (LA QUE QUIERE EL PROFESOR)
# ------------------------------------------------------------
# "edad >= 18" se evalúa a True o False, así que lo devolvemos directamente.
# "-> bool" indica (opcional) que la función devuelve un booleano.
def es_mayor_de_edad(edad: int) -> bool:
    """Devuelve True si edad >= 18 y False en caso contrario."""
    return edad >= 18


# ------------------------------------------------------------
# PASO 2: Pruebas (incluido el caso límite 18)
# ------------------------------------------------------------
if __name__ == "__main__":
    print(es_mayor_de_edad(20))   # True
    print(es_mayor_de_edad(15))   # False
    print(es_mayor_de_edad(18))   # True  (caso límite: >= incluye el 18)

    # Uso típico: el resultado sirve directamente como condición de un if
    edad = 17
    if es_mayor_de_edad(edad):
        print(f"Con {edad} años puedes votar")
    else:
        print(f"Con {edad} años aún no puedes votar")

# ------------------------------------------------------------
# SALIDA ESPERADA:
# True
# False
# True
# Con 17 años aún no puedes votar
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Usar > en vez de >=: con 18 devolvería False (mal).
# - Devolver los textos "True"/"False" (str) en vez de los booleanos True/False.
# - True y False van con MAYÚSCULA inicial (en Java son true/false).
#
# ALTERNATIVA (funciona, pero es MENOS BUENA: el profesor la considera
# redundante, porque el if solo sirve para repetir lo que ya vale la condición):
#   def es_mayor_de_edad(edad: int) -> bool:
#       if edad >= 18:
#           return True
#       else:
#           return False
```

03_funciones/05_contar_vocales.py

```
"""
TEMA 5 - Ejercicio 5: Contar vocales
====================================
ENUNCIADO (literal del profesor):
    "Escribe una función contar_vocales que reciba una cadena de texto y
    devuelva la cantidad total de vocales (a, e, i, o, u, tanto mayúsculas
    como minúsculas) que contiene."

FORMA QUE EXIGE EL PROFESOR:
    - No exige forma concreta, pero "devuelva" => usar return (no print dentro).
    - Debe contar mayúsculas Y minúsculas.

CONCEPTOS QUE PRACTICA:
    - Recorrer un string con for (carácter a carácter).
    - Operador "in" para comprobar pertenencia.
    - Patrón contador (contador = 0 ... contador += 1).

CÓMO EJECUTARLO:  python 05_contar_vocales.py   (en Linux/Docker: python3 05_contar_vocales.py)
"""


# ---------------------------------------------------------------
# PASO 1: Definir la función
# ---------------------------------------------------------------
def contar_vocales(texto: str) -> int:
    """Devuelve cuántas vocales (mayúsculas o minúsculas) tiene el texto."""
    # Metemos las vocales en ambos casos para no tener que convertir el texto
    vocales = "aeiouAEIOU"
    contador = 0                  # Empezamos a contar desde 0
    for letra in texto:           # Un for sobre un str da cada carácter
        if letra in vocales:      # "in" -> ¿está esta letra entre las vocales?
            contador += 1         # Si es vocal, sumamos 1
    return contador               # El return va FUERA del for (sin sangrar)


# ---------------------------------------------------------------
# PASO 2: Pruebas
# ---------------------------------------------------------------
if __name__ == "__main__":
    print(contar_vocales("Programacion en Python"))   # 7
    print(contar_vocales("AEIOU aeiou"))              # 10
    print(contar_vocales("xyz"))                      # 0 (caso sin vocales)
    print(contar_vocales(""))                         # 0 (cadena vacía)

# ---------------------------------------------------------------
# SALIDA ESPERADA:
# 7
# 10
# 0
# 0
# ---------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Poner el return DENTRO del for: devolvería tras mirar solo la 1ª letra.
# - Las vocales con tilde (á, é...) NO cuentan con "aeiouAEIOU". Si las
#   necesitases: vocales = "aeiouáéíóúAEIOUÁÉÍÓÚ".
# ALTERNATIVA (también válida): pasar a minúsculas y comparar solo "aeiou":
#   for letra in texto.lower():
#       if letra in "aeiou":
#           contador += 1
# ALTERNATIVA compacta (más avanzada):
#   return sum(1 for letra in texto if letra in "aeiouAEIOU")
```

03_funciones/06_parametros_por_defecto.py

```
"""
TEMA 5 - Ejercicio 6: Parámetros por defecto
============================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada presentar_usuario que reciba dos parámetros: el
    nombre de un usuario y su rol. El parámetro correspondiente al rol debe
    tener el valor por defecto 'Estudiante'. La función debe imprimir por
    pantalla un mensaje mostrando el nombre y el rol del usuario con el formato
    'Usuario: [nombre] | Rol: [rol]'. Por último, realiza dos llamadas de prueba
    a la función: una pasando únicamente el nombre y otra pasando tanto el
    nombre como un rol específico."

FORMA QUE EXIGE EL PROFESOR:
    - rol con valor por defecto EXACTO 'Estudiante':  rol="Estudiante"
    - Formato exacto: 'Usuario: [nombre] | Rol: [rol]'
    - DOS llamadas: una solo con nombre y otra con nombre + rol.

CONCEPTOS QUE PRACTICA:
    - Parámetros con valor por defecto (opcionales al llamar).
    - Los parámetros por defecto van SIEMPRE detrás de los obligatorios.

CÓMO EJECUTARLO:  python 06_parametros_por_defecto.py   (en Linux/Docker: python3 06_parametros_por_defecto.py)
"""


# ---------------------------------------------------------------
# PASO 1: Definir la función con un parámetro por defecto
# ---------------------------------------------------------------
# nombre -> obligatorio.  rol="Estudiante" -> opcional: si no lo pasas,
# vale "Estudiante". El de por defecto va DETRÁS del obligatorio.
def presentar_usuario(nombre, rol="Estudiante"):
    """Imprime 'Usuario: nombre | Rol: rol'. rol vale 'Estudiante' si no se indica."""
```

```
    print(f"Usuario: {nombre} | Rol: {rol}")


# ------------------------------------------------------------------
# PASO 2: Las dos llamadas de prueba que pide el enunciado
# ------------------------------------------------------------------
if __name__ == "__main__":
    presentar_usuario("Laura")                    # Solo nombre -> usa el valor por defecto
    presentar_usuario("Carlos", "Profesor")       # Nombre y rol -> sustituye al por defecto
    presentar_usuario("Marta", rol="Administradora")  # Extra: pasándolo por nombre

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Usuario: Laura | Rol: Estudiante
# Usuario: Carlos | Rol: Profesor
# Usuario: Marta | Rol: Administradora
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - def presentar_usuario(rol="Estudiante", nombre):  -> SyntaxError
#   ("non-default argument follows default argument"): los parámetros con
#   valor por defecto SIEMPRE al final.
# - Cuidado con el formato: espacios alrededor de la barra " | ".
# EN JAVA SERÍA... Java no tiene valores por defecto: se haría con sobrecarga
#   (dos métodos presentarUsuario con distintos parámetros).
```

03_funciones/07_retornar_multiples_valores.py

```
"""
TEMA 5 - Ejercicio 7: Retornar múltiples valores
================================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada obtener_extremos que reciba una lista de números
    como parámetro. La función debe encontrar y retornar simultáneamente el
    valor mínimo y el valor máximo de la lista utilizando retornos múltiples
    (una tupla). Luego, escribe un código de prueba que llame a la función
    pasándole una lista de ejemplo (como [...]), desempaquete los resultados en
    dos variables separadas e imprima por pantalla un mensaje mostrando ambos
    valores."
    (La lista de ejemplo salió cortada en la fuente: uso [4, 8, 15, 16, 23, 42].)

FORMA QUE EXIGE EL PROFESOR:
    - Retorno múltiple en tupla:  return min(lista), max(lista)
    - Desempaquetado en DOS variables:  minimo, maximo = obtener_extremos(...)
    - Imprimir un mensaje con ambos valores (fuera de la función).

CONCEPTOS QUE PRACTICA:
    - "return a, b" devuelve una TUPLA (a, b) aunque no pongas paréntesis.
    - Desempaquetado de tuplas.
    - Funciones integradas min() y max().

CÓMO EJECUTARLO:  python 07_retornar_multiples_valores.py   (en Linux/Docker: python3 07_retornar_multiples_valores.py)
"""


# ------------------------------------------------------------------
# PASO 1: Función que devuelve DOS valores a la vez
# ------------------------------------------------------------------
def obtener_extremos(lista):
    """Devuelve una tupla (mínimo, máximo) de la lista de números."""
    # La coma crea una tupla: esto equivale a return (min(lista), max(lista))
    return min(lista), max(lista)


# ------------------------------------------------------------------
# PASO 2: Prueba con desempaquetado
# ------------------------------------------------------------------
if __name__ == "__main__":
    numeros = [4, 8, 15, 16, 23, 42]

    # Desempaquetado: el 1er valor de la tupla va a minimo y el 2º a maximo.
    # Tiene que haber tantas variables como valores devuelve la función.
    minimo, maximo = obtener_extremos(numeros)
    print(f"El valor mínimo es {minimo} y el máximo es {maximo}")

    # Para ver que realmente es una tupla:
    resultado = obtener_extremos(numeros)
    print(resultado, type(resultado))

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# El valor mínimo es 4 y el máximo es 42
# (4, 42) <class 'tuple'>
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - minimo, maximo, otro = obtener_extremos(...) -> ValueError (not enough
#   values to unpack): el número de variables debe coincidir.
# - Con una lista vacía, min() y max() lanzan ValueError. Si te lo pidieran,
#   comprueba antes:  if not lista: return None, None
# EN JAVA SERÍA... Java no puede devolver dos valores: habría que devolver un
#   array o un objeto. En Python basta con separar por comas.
# ALTERNATIVA (funciona, pero NO es lo que pide: no usa el retorno múltiple
#   desempaquetado):  res = obtener_extremos(n); print(res[0], res[1])
```

03_funciones/08_filtrar_pares.py

```
"""
TEMA 5 - Ejercicio 8: Procesando listas en funciones
```

```
==================================================
ENUNCIADO (literal del profesor):
    "Desarrolla una función llamada filtrar_pares que reciba una lista de
    números enteros. La función debe iterar sobre la lista y retornar una nueva
    lista que contenga únicamente los números pares."

FORMA QUE EXIGE EL PROFESOR:
    - "iterar sobre la lista" => recorrerla con un for.
    - "retornar una NUEVA lista" => return de una lista nueva (no modificar
      la original ni imprimir dentro).

CONCEPTOS QUE PRACTICA:
    - Crear lista vacía y rellenarla con .append().
    - Operador módulo %: n % 2 == 0 => n es par.

CÓMO EJECUTARLO:  python 08_filtrar_pares.py   (en Linux/Docker: python3 08_filtrar_pares.py)
"""


# ------------------------------------------------------------
# PASO 1: Definir la función
# ------------------------------------------------------------
def filtrar_pares(numeros):
    """Devuelve una lista NUEVA solo con los números pares de 'numeros'."""
    pares = []                    # Lista nueva vacía donde iremos guardando
    for numero in numeros:        # Iteramos la lista elemento a elemento
        if numero % 2 == 0:       # Resto 0 al dividir entre 2 => es par
            pares.append(numero)  # append añade al final de la lista
    return pares                  # Fuera del for: devolvemos la lista completa


# ------------------------------------------------------------
# PASO 2: Pruebas
# ------------------------------------------------------------
if __name__ == "__main__":
    lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    resultado = filtrar_pares(lista)
    print(f"Lista original: {lista}")       # La original no cambia
    print(f"Solo pares:     {resultado}")
    print(f"Sin pares:      {filtrar_pares([1, 3, 5])}")   # Caso límite -> []

# ------------------------------------------------------------
# SALIDA ESPERADA:
# Lista original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
# Solo pares:     [2, 4, 6, 8, 10]
# Sin pares:      []
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Crear pares = [] DENTRO del for: se vaciaría en cada vuelta.
# - Borrar los impares de la lista original mientras la recorres: da
#   resultados raros y además el enunciado pide una lista NUEVA.
# ALTERNATIVA (funciona y es muy pythónica, pero el enunciado dice "iterar",
#   así que en el examen usa el for):
#   return [n for n in numeros if n % 2 == 0]          # comprensión de lista
#   return list(filter(lambda n: n % 2 == 0, numeros)) # filter + lambda
```

03_funciones/09_promedio_args.py

```
"""
TEMA 5 - Ejercicio 9: Número variable de argumentos (*args)
============================================================
ENUNCIADO (literal del profesor):
    "Escribe una función llamada calcular_promedio que pueda recibir una
    cantidad indefinida de números utilizando *args. La función debe retornar
    la media de todos los números pasados como argumento."

FORMA QUE EXIGE EL PROFESOR:
    - OBLIGATORIO el parámetro *args (no vale recibir una lista).
    - Retornar (return) la media, no imprimirla dentro.

CONCEPTOS QUE PRACTICA:
    - *args: Python EMPAQUETA todos los argumentos posicionales en una TUPLA.
    - sum() y len() sobre esa tupla.
    - Evitar la división por cero si no llega ningún número.

CÓMO EJECUTARLO:  python 09_promedio_args.py   (en Linux/Docker: python3 09_promedio_args.py)
"""


# ------------------------------------------------------------
# PASO 1: Función con *args
# ------------------------------------------------------------
# El asterisco es lo importante; "args" es solo el nombre por convención.
# Si llamas calcular_promedio(4, 6, 8), dentro args == (4, 6, 8).
def calcular_promedio(*args):
    """Devuelve la media de todos los números recibidos (0 si no llega ninguno)."""
    # Si no llega ningún número, args es la tupla vacía () y len(args) es 0:
    # dividir entre 0 daría ZeroDivisionError, así que lo controlamos antes.
    if len(args) == 0:
        return 0
    # Media = suma de todos / cuántos hay
    return sum(args) / len(args)


# ------------------------------------------------------------
# PASO 2: Pruebas con distinta cantidad de argumentos
# ------------------------------------------------------------
if __name__ == "__main__":
    print(calcular_promedio(4, 6, 8))          # 3 argumentos
    print(calcular_promedio(10, 5))            # 2 argumentos
```

```
    print(calcular_promedio(7, 8, 9, 10, 6))   # 5 argumentos
    print(calcular_promedio())                 # Ninguno: no explota, devuelve 0

    # Si ya tienes una LISTA, la "desempaquetas" con * al llamar:
    notas = [5, 7, 9]
    print(calcular_promedio(*notas))

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# 6.0
# 7.5
# 8.0
# 0
# 7.0
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - calcular_promedio(notas) sin * -> args sería ([5, 7, 9],) (una tupla
#   con UNA lista dentro) y sum() fallaría con TypeError.
# - La división / SIEMPRE da float (6.0), aunque el resultado sea exacto.
# - Sin el if, calcular_promedio() -> ZeroDivisionError.
# EN JAVA SERÍA... double calcularPromedio(double... numeros) (varargs).
# ALTERNATIVA (funciona, pero NO es lo que pide: no usa *args):
#   def calcular_promedio(lista): return sum(lista) / len(lista)
```

03_funciones/10a_kwargs_perfil_diccionario.py

```
"""
TEMA 5 - Ejercicio 10 (Variante A): Argumentos de palabras clave (**kwargs)
========================================================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada crear_perfil_usuario que reciba obligatoriamente
    un nombre y un apellido, y luego acepte cualquier cantidad de información
    adicional utilizando **kwargs (como edad, ciudad, profesión). La función
    debe retornar un diccionario completo con todos los datos del usuario."

    (La variante B, que IMPRIME cada par, está en 10b_kwargs_perfil_imprimir.py.
    Fíjate en cuál te ponen: A RETORNA un diccionario, B IMPRIME.)

FORMA QUE EXIGE EL PROFESOR:
    - Parámetros obligatorios nombre y apellido + **kwargs.
    - RETORNAR un diccionario con todo (no imprimir dentro).

CONCEPTOS QUE PRACTICA:
    - **kwargs: Python EMPAQUETA los argumentos con nombre (clave=valor) en
      un DICCIONARIO.
    - Combinar parámetros normales con **kwargs (kwargs siempre al final).
    - dict.update() para fusionar diccionarios.

CÓMO EJECUTARLO:  python 10a_kwargs_perfil_diccionario.py   (en Linux/Docker: python3 10a_kwargs_perfil_diccionario.py)
"""


# ------------------------------------------------------------------
# PASO 1: Función con obligatorios + **kwargs
# ------------------------------------------------------------------
# Si llamas crear_perfil_usuario("Laura", "Glez", edad=26, ciudad="Tenerife"):
#   nombre = "Laura", apellido = "Glez"
#   kwargs = {"edad": 26, "ciudad": "Tenerife"}
def crear_perfil_usuario(nombre, apellido, **kwargs):
    """Devuelve un diccionario con nombre, apellido y todos los datos extra."""
    # Diccionario base con los dos datos obligatorios
    perfil = {"nombre": nombre, "apellido": apellido}
    # update() añade al perfil todas las parejas clave-valor de kwargs
    perfil.update(kwargs)
    return perfil


# ------------------------------------------------------------------
# PASO 2: Pruebas
# ------------------------------------------------------------------
if __name__ == "__main__":
    # Con datos adicionales
    usuario1 = crear_perfil_usuario("Laura", "González", edad=26,
                                    ciudad="Tenerife", profesion="Programadora")
    print(usuario1)

    # Solo con los obligatorios: kwargs llega vacío {} y no pasa nada
    usuario2 = crear_perfil_usuario("Pedro", "Pérez")
    print(usuario2)

    # Como devuelve un diccionario, podemos acceder a sus claves
    print(f"{usuario1['nombre']} vive en {usuario1['ciudad']}")

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# {'nombre': 'Laura', 'apellido': 'González', 'edad': 26, 'ciudad': 'Tenerife', 'profesion': 'Programadora'}
# {'nombre': 'Pedro', 'apellido': 'Pérez'}
# Laura vive en Tenerife
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Los datos extra se pasan CON nombre (edad=26). Si pasas un tercer valor
#   suelto, crear_perfil_usuario("A", "B", 26) -> TypeError.
# - **kwargs debe ir el ÚLTIMO en la definición.
# - Las claves no llevan comillas al llamar (edad=26), pero en el
#   diccionario resultante son strings ('edad').
# ALTERNATIVA (igual de válida, sin update):
#   perfil = {"nombre": nombre, "apellido": apellido}
#   for clave, valor in kwargs.items():
```

```
#       perfil[clave] = valor
#   return perfil
```

03_funciones/10b_kwargs_perfil_imprimir.py

```
"""
TEMA 5 - Ejercicio 10 (Variante B): Argumentos de palabras clave (**kwargs)
================================================================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada crear_perfil_usuario que acepte una cantidad
    variable de argumentos con nombre (utilizando **kwargs). La función debe
    iterar sobre los elementos recibidos e imprimir cada par por pantalla con
    el formato 'Clave: valor', asegurándose de poner en mayúscula la primera
    letra de la clave. Finalmente, escribe un código de prueba llamando a la
    función y pasándole diferentes datos descriptivos, como el nombre, la edad
    y la ciudad de un usuario."

    (La variante A, que RETORNA un diccionario, está en 10a_kwargs_perfil_diccionario.py.)

FORMA QUE EXIGE EL PROFESOR:
    - Solo **kwargs como parámetro.
    - Iterar con for sobre kwargs.items() e IMPRIMIR 'Clave: valor'.
    - Primera letra de la clave en mayúscula -> clave.capitalize().

CONCEPTOS QUE PRACTICA:
    - **kwargs es un diccionario: se recorre con .items() (clave y valor).
    - Métodos de string: capitalize().

CÓMO EJECUTARLO:  python 10b_kwargs_perfil_imprimir.py   (en Linux/Docker: python3 10b_kwargs_perfil_imprimir.py)
"""


# ----------------------------------------------------------------
# PASO 1: Función que recorre e imprime los **kwargs
# ----------------------------------------------------------------
def crear_perfil_usuario(**kwargs):
    """Imprime cada dato recibido con el formato 'Clave: valor'."""
    # .items() da parejas (clave, valor) que desempaquetamos en el for
    for clave, valor in kwargs.items():
        # capitalize() -> primera letra en mayúscula ("edad" -> "Edad")
        print(f"{clave.capitalize()}: {valor}")


# ----------------------------------------------------------------
# PASO 2: Prueba con nombre, edad y ciudad
# ----------------------------------------------------------------
if __name__ == "__main__":
    crear_perfil_usuario(nombre="Laura", edad=26, ciudad="Tenerife")
    print("-" * 20)
    # Otra llamada con otros datos: la función se adapta sola
    crear_perfil_usuario(nombre="Carlos", profesion="Profesor")

# ----------------------------------------------------------------
# SALIDA ESPERADA:
# Nombre: Laura
# Edad: 26
# Ciudad: Tenerife
# --------------------
# Nombre: Carlos
# Profesion: Profesor
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - for clave in kwargs:  solo da las CLAVES; para clave y valor usa .items().
# - capitalize() pone en minúscula el resto ("nombreCompleto" -> "Nombrecompleto").
#   Para solo cambiar la primera letra: clave[0].upper() + clave[1:]
# - title() pondría mayúscula en cada palabra; aquí basta capitalize().
```

03_funciones/11_type_hints_celsius.py

```
"""
TEMA 5 - Ejercicio 11: Anotaciones de tipo (Type Hints)
================================================================
ENUNCIADO (literal del profesor):
    "Escribe la función para pasar de celsius_a_fahrenheit. Incluye Type Hints
    (anotaciones de tipo) tanto para los parámetros de entrada como para el
    valor de retorno. Asegúrate de documentar la función usando un Docstring
    explicativo. Fahrenheit = (celsius * 9/5) + 32."

FORMA QUE EXIGE EL PROFESOR:
    - Type hints en el parámetro Y en el retorno:
          def celsius_a_fahrenheit(celsius: float) -> float:
      (en la solución del profesor aparece con el nombre
      celsius_a_fahrenheit_tipado; cualquiera de los dos vale si respeta la firma)
    - Docstring con triple comilla (tres comillas dobles al abrir y al cerrar)
      justo debajo del def.

CONCEPTOS QUE PRACTICA:
    - Anotaciones de tipo:  parametro: tipo   y   -> tipo_de_retorno
    - Docstrings y cómo consultarlos (help() o .__doc__).

CÓMO EJECUTARLO:  python 11_type_hints_celsius.py   (en Linux/Docker: python3 11_type_hints_celsius.py)
"""


# ----------------------------------------------------------------
# PASO 1: Función con type hints y docstring
# ----------------------------------------------------------------
```

```python
# celsius: float  -> se espera un número decimal como entrada
# -> float        -> la función devuelve un número decimal
def celsius_a_fahrenheit(celsius: float) -> float:
    """
    Convierte una temperatura de grados Celsius a grados Fahrenheit.

    Fórmula: Fahrenheit = (celsius * 9/5) + 32

    Args:
        celsius (float): Temperatura en grados Celsius.

    Returns:
        float: Temperatura equivalente en grados Fahrenheit.
    """
    return (celsius * 9 / 5) + 32


# ------------------------------------------------------------------
# PASO 2: Pruebas
# ------------------------------------------------------------------
if __name__ == "__main__":
    print(f"0 °C = {celsius_a_fahrenheit(0)} °F")         # Punto de congelación
    print(f"100 °C = {celsius_a_fahrenheit(100)} °F")     # Punto de ebullición
    print(f"36.5 °C = {celsius_a_fahrenheit(36.5):.1f} °F")  # :.1f -> 1 decimal
    print(f"-40 °C = {celsius_a_fahrenheit(-40)} °F")     # Curiosidad: coinciden

    # Extra, no necesario: el docstring se puede leer desde el programa
    # (help(celsius_a_fahrenheit) también lo muestra).
    print(celsius_a_fahrenheit.__doc__.strip().splitlines()[0])

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# 0 °C = 32.0 °F
# 100 °C = 212.0 °F
# 36.5 °C = 97.7 °F
# -40 °C = -40.0 °F
# Convierte una temperatura de grados Celsius a grados Fahrenheit.
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Los type hints NO obligan: Python no da error si pasas un int o incluso
#   un str (fallaría luego al operar). Son documentación para personas y
#   herramientas, no comprobación como en Java.
# - El docstring debe ser lo PRIMERO dentro de la función; un comentario #
#   NO es un docstring.
# - 9/5 en Python 3 vale 1.8 (en Java 9/5 con enteros daría 1).
# EN JAVA SERÍA... double celsiusAFahrenheit(double celsius) — allí el tipo es
#   obligatorio y se comprueba al compilar.
```

03_funciones/12_decorador.py

```python
"""
TEMA 5 - Ejercicio 12: Decoradores
==================================
ENUNCIADO (literal del profesor):
    "Crea un decorador llamado anunciar_ejecucion que modifique el
    comportamiento de una función añadiendo mensajes adicionales: debe imprimir
    '--- Iniciando función ---' antes de la ejecución de la función decorada y
    '--- Función terminada ---' justo después. Luego, define una función
    llamada tarea_importante que simplemente imprima 'Procesando datos en la
    base de datos...', aplícale el decorador utilizando la sintaxis @ y,
    finalmente, llámala para comprobar su funcionamiento."

FORMA QUE EXIGE EL PROFESOR:
    - Decorador llamado anunciar_ejecucion.
    - Aplicarlo con la sintaxis @anunciar_ejecucion encima del def.
    - Llamar a tarea_importante() para comprobarlo.

CONCEPTOS QUE PRACTICA:
    - Un decorador es una función que RECIBE una función y DEVUELVE otra
      función (la "envoltura") que añade algo antes y/o después.
    - Funciones definidas dentro de funciones.
    - Las funciones se pueden pasar y devolver como cualquier valor.

CÓMO EJECUTARLO:  python 12_decorador.py   (en Linux/Docker: python3 12_decorador.py)
"""


# ------------------------------------------------------------------
# PASO 1: Crear el decorador
# ------------------------------------------------------------------
# Esquema que hay que memorizar:
#   def decorador(funcion):
#       def envoltura(*args, **kwargs):
#           ... antes ...
#           resultado = funcion(*args, **kwargs)
#           ... después ...
#           return resultado
#       return envoltura          # <- SIN paréntesis: devolvemos la función
def anunciar_ejecucion(funcion):
    """Decorador: imprime un mensaje antes y otro después de ejecutar 'funcion'."""

    # *args y **kwargs permiten que la envoltura acepte cualquier argumento
    # y se lo pase tal cual a la función original (aquí no hay, pero así el
    # decorador sirve para cualquier función).
    def envoltura(*args, **kwargs):
        print("--- Iniciando función ---")
        resultado = funcion(*args, **kwargs)   # Ejecutamos la función original
        print("--- Función terminada ---")
        return resultado                       # Devolvemos lo que devolvió ella
```

```
    return envoltura


# ------------------------------------------------------------------
# PASO 2: Aplicar el decorador con @
# ------------------------------------------------------------------
# @anunciar_ejecucion equivale a escribir, después del def:
#     tarea_importante = anunciar_ejecucion(tarea_importante)
@anunciar_ejecucion
def tarea_importante():
    """Simula una tarea que trabaja con la base de datos."""
    print("Procesando datos en la base de datos...")


# ------------------------------------------------------------------
# PASO 3: Llamarla para comprobar el funcionamiento
# ------------------------------------------------------------------
if __name__ == "__main__":
    tarea_importante()   # En realidad se ejecuta la "envoltura"

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# --- Iniciando función ---
# Procesando datos en la base de datos...
# --- Función terminada ---
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - return envoltura()  (CON paréntesis) -> MAL: ejecutaría la envoltura en
#   el momento de decorar y tarea_importante pasaría a valer None (al
#   llamarla después: TypeError). Se devuelve la función SIN ().
# - @anunciar_ejecucion() con paréntesis -> error: se pone sin ().
# - El decorador tiene que estar definido ANTES de usarlo con @.
# ALTERNATIVA (funciona, pero NO es lo que pide: el enunciado exige @):
#   tarea_importante = anunciar_ejecucion(tarea_importante)
```

03_funciones/13_map_lambda.py

```
"""
TEMA 5 - Ejercicio 13: Uso de map() con lambda
==============================================
ENUNCIADO (literal del profesor):
    "Dada la lista numeros = [...], utiliza la función incorporada map() junto
    con una función lambda para generar una nueva lista que contenga los
    números de la lista original elevados al cuadrado."
    (La lista salió cortada en la fuente: uso [1, 2, 3, 4, 5].)

FORMA QUE EXIGE EL PROFESOR:
    - Exactamente:  list(map(lambda x: x**2, numeros))
    - map + lambda + list (para convertir el resultado en lista).

CONCEPTOS QUE PRACTICA:
    - lambda: función anónima de UNA expresión:  lambda parametro: resultado
    - map(funcion, iterable): aplica la función a CADA elemento.
    - map devuelve un objeto "map" (perezoso): hay que pasarlo por list().

CÓMO EJECUTARLO:  python 13_map_lambda.py   (en Linux/Docker: python3 13_map_lambda.py)
"""


# ------------------------------------------------------------------
# PASO 1: Datos
# ------------------------------------------------------------------
numeros = [1, 2, 3, 4, 5]

# ------------------------------------------------------------------
# PASO 2: map + lambda (LA FORMA QUE PIDE EL PROFESOR)
# ------------------------------------------------------------------
# lambda x: x**2  -> "una función que recibe x y devuelve x al cuadrado"
# map(...)        -> se la aplica a 1, 2, 3, 4 y 5
# list(...)       -> convierte el objeto map en una lista de verdad
cuadrados = list(map(lambda x: x**2, numeros))

print(f"Original:  {numeros}")
print(f"Cuadrados: {cuadrados}")

# Para que veas por qué hace falta list():
print(map(lambda x: x**2, numeros))   # Muestra <map object at 0x...>, no los números

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Original:  [1, 2, 3, 4, 5]
# Cuadrados: [1, 4, 9, 16, 25]
# <map object at 0x...>     (la dirección cambia en cada ejecución)
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar list() -> imprimes "<map object ...>" en vez de la lista.
# - map(numeros, lambda x: x**2) -> orden al revés: PRIMERO la función,
#   DESPUÉS la lista.
# - x^2 NO es potencia en Python (es XOR a nivel de bits). Potencia: x**2.
# - La lista original NO se modifica: map genera valores nuevos.
# EN JAVA SERÍA... numeros.stream().map(x -> x * x).toList()
# ALTERNATIVAS (funcionan, pero NO son lo que pide el enunciado):
#   cuadrados = [x**2 for x in numeros]       # comprensión de lista
#   cuadrados = []                            # bucle for clásico
#   for x in numeros:
#       cuadrados.append(x**2)
```

03_funciones/14_orden_superior.py

```
"""
TEMA 5 - Ejercicio 14: Funciones de Orden Superior
==================================================
ENUNCIADO (literal del profesor):
    "Crea una función llamada aplicar_operacion que reciba tres parámetros:
    dos números y una función. aplicar_operacion debe ejecutar la función
    recibida pasándole los dos números como argumentos y retornar el
    resultado. Prueba tu función pasándole funciones creadas por ti para
    sumar y restar."

FORMA QUE EXIGE EL PROFESOR:
    - aplicar_operacion(a, b, funcion): el 3er parámetro es una FUNCIÓN.
    - Dentro: return funcion(a, b)
    - Probarla con funciones PROPIAS sumar y restar (definidas con def).

CONCEPTOS QUE PRACTICA:
    - En Python las funciones son valores: se pueden pasar como argumento.
    - Pasar la función SIN paréntesis (sumar), no llamarla (sumar()).

CÓMO EJECUTARLO:  python 14_orden_superior.py   (en Linux/Docker: python3 14_orden_superior.py)
"""


# ---------------------------------------------------------------
# PASO 1: Funciones "normales" que pasaremos como argumento
# ---------------------------------------------------------------
def sumar(a, b):
    """Devuelve a + b."""
    return a + b


def restar(a, b):
    """Devuelve a - b."""
    return a - b


# ---------------------------------------------------------------
# PASO 2: La función de orden superior
# ---------------------------------------------------------------
# "funcion" es un parámetro como otro cualquiera, pero lo que contiene
# es una función, así que podemos LLAMARLO con paréntesis: funcion(a, b).
def aplicar_operacion(a, b, funcion):
    """Ejecuta funcion(a, b) y devuelve su resultado."""
    return funcion(a, b)


# ---------------------------------------------------------------
# PASO 3: Pruebas
# ---------------------------------------------------------------
if __name__ == "__main__":
    # OJO: se pasa "sumar" SIN paréntesis -> pasamos la función, no su resultado
    print(f"10 + 3 = {aplicar_operacion(10, 3, sumar)}")
    print(f"10 - 3 = {aplicar_operacion(10, 3, restar)}")

    # Extra: también se le puede pasar una lambda (función anónima)
    print(f"10 x 3 = {aplicar_operacion(10, 3, lambda x, y: x * y)}")

# ---------------------------------------------------------------
# SALIDA ESPERADA:
# 10 + 3 = 13
# 10 - 3 = 7
# 10 x 3 = 30
# ---------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - aplicar_operacion(10, 3, sumar())  -> TypeError: sumar() se ejecutaría
#   sin argumentos. La función se pasa SIN ().
# - El orden de los parámetros importa: (a, b, funcion), la función la última
#   tal como dice el enunciado.
# EN JAVA SERÍA... int aplicarOperacion(int a, int b, BinaryOperator<Integer> f)
#   { return f.apply(a, b); }  — en Python no hace falta ninguna interfaz.
```

03_funciones/15_recursividad_factorial.py

```
"""
TEMA 5 - Ejercicio 15: Recursividad
===================================
ENUNCIADO (literal del profesor):
    "Escribe una función recursiva (una función que se llama a sí misma)
    llamada calcular_factorial que reciba un número entero positivo n y
    retorne su factorial. Recuerda establecer el caso base (el factorial de 0
    y 1 es 1) para evitar un bucle infinito."

FORMA QUE EXIGE EL PROFESOR:
    - Nombre exacto calcular_factorial (en tu repo la llamaste factorial: ¡ojo!).
    - RECURSIVA: debe llamarse a sí misma (nada de for/while).
    - Caso base: si n es 0 o 1 -> return 1.

CONCEPTOS QUE PRACTICA:
    - Caso base (cuándo PARAR) + caso recursivo (llamada con un problema más
      pequeño, n - 1).
    - Factorial: n! = n * (n-1) * ... * 1   y   0! = 1! = 1

CÓMO EJECUTARLO:  python 15_recursividad_factorial.py   (en Linux/Docker: python3 15_recursividad_factorial.py)
"""
```

```python
# ------------------------------------------------------------------
# PASO 1: Función recursiva
# ------------------------------------------------------------------
def calcular_factorial(n: int) -> int:
    """Devuelve el factorial de n (entero >= 0) de forma recursiva."""
    # CASO BASE: detiene las llamadas. Sin él, se llamaría sin fin hasta
    # RecursionError (maximum recursion depth exceeded).
    if n == 0 or n == 1:
        return 1
    # CASO RECURSIVO: n! = n * (n-1)!  -> cada llamada acerca n al caso base
    return n * calcular_factorial(n - 1)


# ------------------------------------------------------------------
# TRAZA DE calcular_factorial(4)  (cómo se ejecuta por dentro)
# ------------------------------------------------------------------
# BAJADA (cada llamada espera a la siguiente):
#   calcular_factorial(4) -> 4 * calcular_factorial(3)
#     calcular_factorial(3) -> 3 * calcular_factorial(2)
#       calcular_factorial(2) -> 2 * calcular_factorial(1)
#         calcular_factorial(1) -> CASO BASE -> devuelve 1
# SUBIDA (se resuelven de abajo arriba):
#       calcular_factorial(2) = 2 * 1  = 2
#     calcular_factorial(3) = 3 * 2  = 6
#   calcular_factorial(4) = 4 * 6  = 24
# ------------------------------------------------------------------


# ------------------------------------------------------------------
# PASO 2: Pruebas
# ------------------------------------------------------------------
if __name__ == "__main__":
    print(f"4! = {calcular_factorial(4)}")
    print(f"5! = {calcular_factorial(5)}")
    print(f"0! = {calcular_factorial(0)}")   # Caso base
    print(f"1! = {calcular_factorial(1)}")   # Caso base

# ------------------------------------------------------------------
# SALIDA ESPERADA:
# 4! = 24
# 5! = 120
# 0! = 1
# 1! = 1
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar el return en el caso recursivo (poner solo
#   n * calcular_factorial(n - 1)) -> la función devuelve None.
# - Llamar con n en vez de n - 1 -> nunca llega al caso base: RecursionError.
# - Con un negativo (-3) tampoco llega nunca al caso base -> RecursionError.
#   Si te pidieran controlarlo:  if n < 0: return None   (antes del caso base)
# ALTERNATIVA (funciona, pero NO es lo que pide: no es recursiva):
#   resultado = 1
#   for i in range(2, n + 1):
#       resultado *= i
#   return resultado
```

03_funciones/16_resumen_funciones.py

```python
"""
TEMA 5 - CHULETA EJECUTABLE DE FUNCIONES
========================================
Todo lo del tema en un solo archivo. Cada sección imprime un ejemplo.
Busca por número de sección (Ctrl+F "== 7" por ejemplo).

 1. def y llamada               10. Ámbito (local / global)
 2. Parámetros posicionales     11. Type hints
 3. Argumentos nombrados        12. Docstring
 4. Valores por defecto         13. lambda + map / filter / sorted(key=)
 5. return vs print (None)      14. Función como argumento
 6. Retorno múltiple            15. Decorador
 7. *args                       16. Recursividad
 8. **kwargs                    17. ERROR: parámetro mutable por defecto
 9. Orden correcto de parámetros

CÓMO EJECUTARLO:  python 16_resumen_funciones.py   (en Linux/Docker: python3 16_resumen_funciones.py)
"""

# ============================================================================
# 1. def Y LLAMADA
# ============================================================================
def saludar():
    """Definir NO ejecuta. Se ejecuta al llamarla con ()."""
    print("Hola")


# ============================================================================
# 2-4. POSICIONALES, NOMBRADOS Y POR DEFECTO
# ============================================================================
def presentar(nombre, edad, ciudad="Tenerife"):   # ciudad es opcional
    """Los parámetros con valor por defecto van SIEMPRE al final."""
    print(f"{nombre}, {edad} años, de {ciudad}")


# ============================================================================
# 5. return vs print
# ============================================================================
def doble_con_return(x):
    """Devuelve el doble de x."""
    return x * 2          # DEVUELVE: el valor se puede guardar y reutilizar
```

```python
def doble_con_print(x):
    """Imprime el doble de x (no devuelve nada -> None)."""
    print(x * 2)          # Solo MUESTRA. Sin return la función devuelve None


# ============================================================================
# 6. RETORNO MÚLTIPLE (devuelve una tupla)
# ============================================================================
def min_max(lista):
    """Devuelve la tupla (mínimo, máximo) de la lista."""
    return min(lista), max(lista)     # equivale a return (min(...), max(...))


# ============================================================================
# 7. *args -> tupla con todos los posicionales sobrantes
# ============================================================================
def sumar_todo(*args):
    """Devuelve la suma de todos los números recibidos."""
    return sum(args)


# ============================================================================
# 8. **kwargs -> diccionario con todos los nombrados sobrantes
# ============================================================================
def mostrar_datos(**kwargs):
    """Imprime cada pareja clave: valor recibida."""
    for clave, valor in kwargs.items():
        print(f"  {clave}: {valor}")


# ============================================================================
# 9. ORDEN CORRECTO: normales, por defecto, *args, **kwargs
# ============================================================================
def orden_completo(a, b=2, *args, **kwargs):
    """Muestra qué valor llega a cada tipo de parámetro."""
    print(f"  a={a} b={b} args={args} kwargs={kwargs}")


# ============================================================================
# 10. ÁMBITO: local vs global
# ============================================================================
contador = 0              # Variable GLOBAL (fuera de cualquier función)


def incrementar():
    """Suma 1 a la variable global contador."""
    global contador       # Sin esto, "contador += 1" da UnboundLocalError
    contador += 1


def solo_local():
    """Crea una variable local con el mismo nombre que la global."""
    contador = 100        # Esto crea una variable LOCAL nueva; la global no cambia
    return contador


# ============================================================================
# 11-12. TYPE HINTS Y DOCSTRING
# ============================================================================
def area_rectangulo(ancho: float, alto: float) -> float:
    """
    Calcula el área de un rectángulo.

    Args:
        ancho (float): base del rectángulo.
        alto (float): altura del rectángulo.

    Returns:
        float: ancho * alto.
    """
    return ancho * alto   # Los hints son informativos: Python NO los comprueba


# ============================================================================
# 14. FUNCIÓN COMO ARGUMENTO (orden superior)
# ============================================================================
def aplicar(funcion, valor):
    """Llama a funcion(valor) y devuelve su resultado."""
    return funcion(valor)     # Se pasa la función SIN () y aquí se llama


# ============================================================================
# 15. DECORADOR
# ============================================================================
def con_marco(funcion):
    """Decorador: imprime un mensaje antes y otro después."""
    def envoltura(*args, **kwargs):
        print("  >>> antes")
        resultado = funcion(*args, **kwargs)
        print("  <<< después")
        return resultado
    return envoltura          # SIN paréntesis


@con_marco                    # = decorada = con_marco(decorada)
def decorada():
    """Función de ejemplo a la que se aplica el decorador."""
    print("  dentro de la función")


# ============================================================================
```

```python
# 16. RECURSIVIDAD (caso base + llamada más pequeña)
# ============================================================================
def factorial(n):
    """Devuelve n! de forma recursiva."""
    if n <= 1:                # CASO BASE: para la recursión
        return 1
    return n * factorial(n - 1)


# ============================================================================
# 17. ERROR TÍPICO: lista (mutable) como valor por defecto
# ============================================================================
# MAL: la lista por defecto se crea UNA sola vez y se comparte entre llamadas.
def anadir_mal(elemento, lista=[]):
    """Ejemplo de lo que NO hay que hacer."""
    lista.append(elemento)
    return lista


# BIEN: usar None como valor por defecto y crear la lista dentro.
def anadir_bien(elemento, lista=None):
    """Añade elemento a lista (crea una lista nueva si no se pasa)."""
    if lista is None:
        lista = []
    lista.append(elemento)
    return lista


if __name__ == "__main__":
    print("== 1. def y llamada")
    saludar()

    print("== 2-4. Posicionales, nombrados, por defecto")
    presentar("Laura", 26)                          # posicionales (por orden)
    presentar(edad=30, nombre="Ana")                # nombrados (da igual el orden)
    presentar("Luis", 40, "Madrid")                 # sustituye el valor por defecto

    print("== 5. return vs print")
    a = doble_con_return(5)
    print("  con return ->", a)                     # 10
    b = doble_con_print(5)                          # imprime 10...
    print("  con print  ->", b)                     # ...pero b vale None

    print("== 6. Retorno múltiple")
    minimo, maximo = min_max([3, 9, 1, 7])          # desempaquetado
    print(f"  min={minimo} max={maximo}")

    print("== 7. *args")
    print("  ", sumar_todo(1, 2, 3, 4))            # args = (1, 2, 3, 4)

    print("== 8. **kwargs")
    mostrar_datos(nombre="Laura", ciudad="Tenerife")

    print("== 9. Orden de parámetros")
    orden_completo(1)
    orden_completo(1, 5, 6, 7, x=8)

    print("== 10. Ámbito")
    incrementar()
    incrementar()
    print("  global contador =", contador)        # 2
    print("  local  contador =", solo_local())    # 100
    print("  global sigue    =", contador)        # 2

    print("== 11-12. Type hints y docstring")
    print("  área:", area_rectangulo(3, 4.5))
    print("  doc :", area_rectangulo.__doc__.strip().splitlines()[0])

    print("== 13. lambda con map / filter / sorted")
    nums = [5, 2, 8, 1]
    print("  map    :", list(map(lambda x: x * 10, nums)))        # transforma
    print("  filter :", list(filter(lambda x: x > 2, nums)))      # filtra
    palabras = ["kiwi", "uva", "manzana"]
    print("  sorted :", sorted(palabras, key=lambda p: len(p)))   # ordena por longitud
    alumnos = [("Ana", 7), ("Luis", 9), ("Eva", 5)]
    print("  sorted :", sorted(alumnos, key=lambda t: t[1], reverse=True))

    print("== 14. Función como argumento")
    print("  ", aplicar(abs, -7), aplicar(lambda x: x ** 2, 4))

    print("== 15. Decorador")
    decorada()

    print("== 16. Recursividad")
    print("  5! =", factorial(5))

    print("== 17. Parámetro mutable por defecto")
    print("  mal :", anadir_mal(1), anadir_mal(2))     # [1, 2] [1, 2] ¡se acumula!
    print("  bien:", anadir_bien(1), anadir_bien(2))   # [1] [2]

# ----------------------------------------------------------------------------
# SALIDA ESPERADA:
# == 1. def y llamada
# Hola
# == 2-4. Posicionales, nombrados, por defecto
# Laura, 26 años, de Tenerife
# Ana, 30 años, de Tenerife
# Luis, 40 años, de Madrid
# == 5. return vs print
#   con return -> 10
# 10
#   con print  -> None
```

```
# == 6. Retorno múltiple
#   min=1 max=9
# == 7. *args
#    10
# == 8. **kwargs
#   nombre: Laura
#   ciudad: Tenerife
# == 9. Orden de parámetros
#   a=1 b=2 args=() kwargs={}
#   a=1 b=5 args=(6, 7) kwargs={'x': 8}
# == 10. Ámbito
#   global contador = 2
#   local  contador = 100
#   global sigue    = 2
# == 11-12. Type hints y docstring
#   área: 13.5
#   doc : Calcula el área de un rectángulo.
# == 13. lambda con map / filter / sorted
#   map    : [50, 20, 80, 10]
#   filter : [5, 8]
#   sorted : ['uva', 'kiwi', 'manzana']
#   sorted : [('Luis', 9), ('Ana', 7), ('Eva', 5)]
# == 14. Función como argumento
#    7 16
# == 15. Decorador
#   >>> antes
#   dentro de la función
#   <<< después
# == 16. Recursividad
#   5! = 120
# == 17. Parámetro mutable por defecto
#   mal : [1, 2] [1, 2]
#   bien: [1] [2]
# ------------------------------------------------------------------------------
# RECORDATORIOS RÁPIDOS:
# - Sin return explícito, toda función devuelve None.
# - Pasar una función como dato: SIN paréntesis (sumar). Llamarla: CON (sumar()).
# - lambda solo admite UNA expresión, sin return ni varias líneas.
# - map/filter devuelven objetos perezosos: envuélvelos en list().
# - sorted() devuelve lista NUEVA; lista.sort() ordena la original y devuelve None.
# - En "mal" salen dos [1, 2] porque las dos llamadas devuelven la MISMA lista.
# ------------------------------------------------------------------------------
```

6 · Clases, listas de objetos, diccionarios, comprensiones y lambda

04_clases_listas_diccionarios_lambda/00_INDICE.md

Tema 6: Clases, listas de objetos, diccionarios, comprensiones y lambda

Es el tema que más pesa en el examen. Si vas con prisa, empieza por `01` (la práctica evaluada) y `09` (la chuleta).

Archivo	Qué pide	Forma exigida	Conceptos clave
`01_teletubbies.py`	Práctica evaluada "La Galaxia de los Teletubbies": clase `PersonajeTeletubby`, pruebas en el main y cuestión teórica	Atributo de clase `saludo_oficial`; privados `__nivel_hambre=50` y `__nivel_felicidad=100`; `ver_estado()` hace **return** de un str; `__str__` / `__repr__` como en el ejemplo; pruebas en `if __name__ == "__main__":` en el orden pedido	`self`, `__init__`, clase vs instancia, name mangling, getter, `__str__` / `__repr__`. Al final, la respuesta teórica lista para copiar
`02_teletubbies_ampliado.py`	La misma clase con getters y setter con validación, más una lista de Teletubbies	Filtrar con comprensión de listas; ordenar con `sorted(key=lambda t: ...)`; recorrer diccionarios con `.items()`	Lista de objetos, buscar, filtrar, ordenar, `max` con key, diccionario nombre -> objeto, comprensión de diccionario
`03_clases_basico.py`	Teoría: la clase mínima	PascalCase, docstrings, pruebas en el main	Constructor, atributos de clase/instancia, métodos, `__str__` / `__repr__`, error de la lista como atributo de clase (Dog)
`04_encapsulamiento_getters_setters.py`	Teoría: público, `_protegido`, `__privado`	Getters y setters como **métodos** `get_` / `set_` (la forma de los apuntes); `@property` solo si lo piden	Name mangling, validación en el setter, `@property` como alternativa
`05_herencia_super.py`	Teoría: herencia y `Cyborg(Humano, Robot)`	`class Hija(Padre):`, `super().__init__(...)`, `**kwargs` y `Cyborg.mro()`	Override, ampliar con `super()`, polimorfismo, `isinstance`, MRO
`06_listas_de_objetos.py`	Teoría: CRUD sobre una lista de `Producto`	for / `enumerate(start=1)`, comprensión para filtrar, `key=lambda`	Añadir, listar, buscar, modificar, eliminar, contar, sumar, `max` / `min` con key, borrar sin romper el for
`07_diccionarios.py`	Teoría: diccionarios	`.items()` / `.values()` para recorrer; `.get()` para acceder sin riesgo	`[]` vs `get`, `del` / `pop`, `in`, diccionario de listas, lista de diccionarios, contar apariciones
`08_comprensiones_y_lambda.py`	Teoría: comprensiones y lambda	Comprensión para filtrar/transformar; `list(map(lambda x: x**2, numeros))`	Comprensiones con if / if-else / de diccionario; `map`, `filter`, `sorted`, `max` / `min` con key; siempre al lado del for equivalente
`09_resumen_poo.py`	Chuleta mínima de todo lo anterior	(Resumen)	Para copiar y adaptar rápido

Ejecutar: `python NN_nombre.py` (en Linux/Docker: `python3 NN_nombre.py`). Ninguno usa `input()`.

04_clases_listas_diccionarios_lambda/01_teletubbies.py

```
"""
TEMA 6 - Ejercicio 1: Práctica evaluada "La Galaxia de los Teletubbies"
======================================================================
ENUNCIADO (literal del profesor):
    Objetivo: Diseñar e implementar en Python una clase que represente a un
    Teletubby, aplicando los conceptos fundamentales de la Programación
    Orientada a Objetos (POO): atributos de clase, atributos de instancia
    (públicos y privados), métodos de instancia y encapsulamiento.

    Requisitos de la Clase (PersonajeTeletubby):
    - Atributo de clase: Crea una variable de clase llamada saludo_oficial con
      el valor '¡Tubby abrazo!' (compartida por todos los personajes).
    - Atributos de instancia (en el constructor __init__):
        Públicos: nombre (cadena), color (cadena), antena (cadena),
                  objeto_favorito (cadena).
        Privados (__): __nivel_hambre (entero, por defecto 50),
                       __nivel_felicidad (entero, por defecto 100).
    - Métodos de instancia:
        saludar(): Muestra por pantalla una presentación con su nombre, color
                   y forma de antena.
        jugar(): Imprime un mensaje indicando que el Teletubby está jugando
                 con su objeto favorito. Incrementa __nivel_hambre en 20
                 unidades y __nivel_felicidad en 10.
        comer_tubby_papilla(): Imprime un mensaje de alimentación. Reduce
                 __nivel_hambre a 0 e incrementa __nivel_felicidad en 20.
        ver_estado(): Método tipo Getter que devuelve una cadena de texto
                 informando sobre el estado actual de los niveles de
                 felicidad y hambre del personaje.
    - Métodos Mágicos (Dunder Methods):
        __str__(self): Devuelve una cadena amigable e informal para el usuario
                 (ej. "Teletubby Tinky Winky (Morado) - Feliz: 100").
        __repr__(self): Devuelve una cadena técnica que representa cómo se
                 instanció el objeto
                 (ej. PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso')).

    Bloque Principal (main): En la sección if __name__ == "__main__":
    realiza las siguientes pruebas:
    1. Instancia al menos dos objetos de la clase PersonajeTeletubby con
       características distintas (por ejemplo, Tinky Winky y Po).
    2. Haz que ambos personajes ejecuten sus métodos saludar() y muestren por
       consola el saludo_oficial de la clase.
    3. Invoca el método jugar() en ambos personajes.
    4. Muestra el estado interno de cada uno usando ver_estado().
    5. Haz que solo uno de los Teletubbies coma usando comer_tubby_papilla().
    6. Muestra de nuevo el estado final de ambos para comprobar que solo ha
       variado el hambre del que ha comido.

    Cuestión Teórica: ¿Qué ocurriría si intentaras modificar po.__nivel_hambre
```

```
    directamente desde el bloque principal? Justifica por qué se utilizan
    atributos privados en este diseño.

FORMA QUE EXIGE EL PROFESOR:
    - Clase llamada EXACTAMENTE PersonajeTeletubby (PascalCase).
    - saludo_oficial como ATRIBUTO DE CLASE (fuera del __init__).
    - Privados con DOBLE guion bajo: self.__nivel_hambre, self.__nivel_felicidad
      (con valores 50 y 100 por defecto).
    - ver_estado() tiene que DEVOLVER (return) un string, NO imprimirlo.
      Quien imprime es el main: print(tinky.ver_estado()).
    - __str__ y __repr__ con el formato de los ejemplos del enunciado.
    - Las pruebas dentro de if __name__ == "__main__": y en el ORDEN pedido.
    - Docstrings con triple comilla en la clase y en cada método.

CONCEPTOS QUE PRACTICA:
    - class, __init__ (constructor), self.
    - Atributo de clase vs atributo de instancia.
    - Encapsulamiento: atributos privados (__) y name mangling.
    - Métodos de instancia; getter (return) frente a print.
    - __str__ (para el usuario) y __repr__ (para el programador).

CÓMO EJECUTARLO:  python 01_teletubbies.py   (en Linux/Docker: python3 01_teletubbies.py)
"""


# ------------------------------------------------------------
# PASO 1: DEFINIR LA CLASE
# ------------------------------------------------------------
# Una CLASE es el "molde" (plantilla). Un OBJETO (o instancia) es cada cosa
# concreta que fabricas con ese molde: tinky_winky y po son dos objetos
# distintos hechos con el mismo molde PersonajeTeletubby.
# EN JAVA SERÍA: public class PersonajeTeletubby { ... }  (aquí no hay llaves,
# manda la indentación de 4 espacios, y no se declaran tipos).
class PersonajeTeletubby:
    """Representa a un personaje Teletubby con sus niveles de hambre y felicidad."""

    # ------------------------------------------------------------
    # ATRIBUTO DE CLASE
    # ------------------------------------------------------------
    # Se escribe DENTRO de la clase pero FUERA de cualquier método (sin self).
    # Pertenece a la CLASE, no a cada objeto: hay UNA sola copia compartida
    # por todos los Teletubbies. Se lee así:
    #     PersonajeTeletubby.saludo_oficial   (forma recomendada)
    #     tinky_winky.saludo_oficial          (también funciona: si el objeto
    #                                          no lo tiene, Python lo busca
    #                                          en la clase)
    # EN JAVA SERÍA: public static String saludoOficial = "¡Tubby abrazo!";
    saludo_oficial = "¡Tubby abrazo!"

    # ------------------------------------------------------------
    # CONSTRUCTOR: __init__
    # ------------------------------------------------------------
    # __init__ se ejecuta SOLO cuando creas un objeto:
    #     po = PersonajeTeletubby("Po", "Rojo", "Círculo", "Patinete")
    # Python crea el objeto vacío y llama a __init__ pasándole ese objeto
    # como primer argumento (self). Por eso tú escribes 4 argumentos al crear
    # el objeto, pero el método tiene 5 parámetros (self + 4).
    #
    # ¿QUÉ ES self? Es "el objeto con el que estoy trabajando ahora mismo".
    # Equivale al "this" de Java, pero en Python es OBLIGATORIO escribirlo:
    #   - como primer parámetro de cada método de instancia, y
    #   - delante de cada atributo: self.nombre (si escribes solo "nombre"
    #     sería una variable local que desaparece al terminar el método).
    #
    # Los privados NO se piden como parámetro: el enunciado dice "por defecto
    # 50" y "por defecto 100", así que todos los Teletubbies nacen con esos
    # valores.
    def __init__(self, nombre, color, antena, objeto_favorito):
        """Crea un Teletubby con sus datos públicos y sus niveles privados por defecto."""
        # ATRIBUTOS DE INSTANCIA PÚBLICOS: cada objeto tiene su propia copia.
        # Se pueden leer y cambiar desde fuera: tinky_winky.nombre
        self.nombre = nombre
        self.color = color
        self.antena = antena
        self.objeto_favorito = objeto_favorito

        # ATRIBUTOS DE INSTANCIA PRIVADOS: empiezan por DOS guiones bajos.
        # Solo se deben usar DENTRO de la clase (desde los métodos).
        # Python les aplica "name mangling": internamente los renombra a
        # _PersonajeTeletubby__nivel_hambre y _PersonajeTeletubby__nivel_felicidad,
        # así que desde fuera "po.__nivel_hambre" NO los encuentra.
        # EN JAVA SERÍA: private int nivelHambre = 50;
        self.__nivel_hambre = 50
        self.__nivel_felicidad = 100

    # ------------------------------------------------------------
    # MÉTODOS DE INSTANCIA
    # ------------------------------------------------------------
    # Son funciones definidas dentro de la clase. Siempre reciben self para
    # poder leer/modificar los atributos de ESE objeto concreto.
    # Se llaman con el punto: po.saludar()  (Python pone self = po).

    def saludar(self):
        """Muestra por pantalla una presentación con nombre, color y forma de antena."""
        # El enunciado dice "muestra por pantalla" -> aquí SÍ va print.
        print(f"¡Hola! Soy {self.nombre}, soy de color {self.color} "
              f"y mi antena tiene forma de {self.antena}.")

    def jugar(self):
        """Juega con su objeto favorito: +20 de hambre y +10 de felicidad."""
        print(f"{self.nombre} está jugando con su {self.objeto_favorito}.")
        # Dentro de la clase SÍ podemos usar los privados con self.__
```

```
        self.__nivel_hambre += 20       # igual que: self.__nivel_hambre = self.__nivel_hambre + 20
        self.__nivel_felicidad += 10

    def comer_tubby_papilla(self):
        """Come Tubby Papilla: el hambre baja a 0 y la felicidad sube 20."""
        print(f"{self.nombre} se está comiendo su Tubby Papilla. ¡Ñam, ñam!")
        self.__nivel_hambre = 0         # "reduce A 0" -> se asigna 0 (no se resta)
        self.__nivel_felicidad += 20

    def ver_estado(self):
        """Getter: DEVUELVE un texto con los niveles de felicidad y hambre."""
        # ¿POR QUÉ UN GETTER? Los atributos privados no se leen desde fuera.
        # La forma controlada de consultarlos es un método público de la
        # clase que los devuelve. Así la clase decide QUÉ se enseña y CÓMO.
        #
        # OJO: return, NO print. El enunciado dice "devuelve una cadena".
        # Con return el main puede imprimirlo, guardarlo en una variable,
        # compararlo... Con print el método devolvería None.
        return (f"Estado de {self.nombre} -> Felicidad: {self.__nivel_felicidad}"
                f" | Hambre: {self.__nivel_hambre}")

    # ------------------------------------------------------------
    # MÉTODOS MÁGICOS (dunder = "double underscore")
    # ------------------------------------------------------------
    # Python los llama SOLO en ciertos momentos; tú no los llamas a mano.

    def __str__(self):
        """Texto amigable para el usuario. Lo usan print(objeto) y str(objeto)."""
        # Formato del ejemplo: Teletubby Tinky Winky (Morado) - Feliz: 100
        # EN JAVA SERÍA: public String toString()
        return f"Teletubby {self.nombre} ({self.color}) - Feliz: {self.__nivel_felicidad}"

    def __repr__(self):
        """Texto técnico: cómo se creó el objeto. Lo usan repr(objeto) y print(lista)."""
        # Formato del ejemplo:
        #   PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso')
        # Las comillas simples van DENTRO del f-string (que usa dobles).
        return (f"PersonajeTeletubby('{self.nombre}', '{self.color}', "
                f"'{self.antena}', '{self.objeto_favorito}')")


# ------------------------------------------------------------------
# PASO 2: BLOQUE PRINCIPAL CON LAS PRUEBAS (en el orden del enunciado)
# ------------------------------------------------------------------
# if __name__ == "__main__": significa "ejecuta esto SOLO si lanzo este
# archivo directamente" (python 01_teletubbies.py). Si otro archivo hace
# "import", la clase se carga pero las pruebas NO se ejecutan.
# EN JAVA SERÍA: public static void main(String[] args)
if __name__ == "__main__":

    # 1. Instanciar al menos dos objetos con características distintas.
    #    Al escribir NombreClase(...) se llama automáticamente a __init__.
    print("=== 1. Creamos los Teletubbies ===")
    tinky_winky = PersonajeTeletubby("Tinky Winky", "Morado", "Triángulo", "Bolso")
    po = PersonajeTeletubby("Po", "Rojo", "Círculo", "Patinete")
    print(tinky_winky)          # print usa __str__
    print(po)
    print(repr(tinky_winky))    # repr() usa __repr__
    print(repr(po))

    # 2. Ambos saludan y muestran el saludo_oficial DE LA CLASE.
    print("\n=== 2. Saludos ===")
    tinky_winky.saludar()
    print(PersonajeTeletubby.saludo_oficial)    # leído desde la clase
    po.saludar()
    print(po.saludo_oficial)                    # leído desde el objeto: es el mismo valor compartido

    # 3. Ambos juegan.
    print("\n=== 3. A jugar ===")
    tinky_winky.jugar()
    po.jugar()

    # 4. Estado de cada uno con ver_estado(). Como ver_estado() DEVUELVE el
    #    texto, aquí hay que meterlo en un print.
    print("\n=== 4. Estado después de jugar ===")
    print(tinky_winky.ver_estado())
    print(po.ver_estado())

    # 5. Solo uno come (Po).
    print("\n=== 5. Solo Po come ===")
    po.comer_tubby_papilla()

    # 6. Estado final de ambos: solo cambia el hambre (y la felicidad) de Po;
    #    Tinky Winky sigue igual porque cada objeto tiene SUS PROPIOS atributos.
    print("\n=== 6. Estado final ===")
    print(tinky_winky.ver_estado())
    print(po.ver_estado())

    # Extra (no lo pide, pero muestra __repr__ en acción): al imprimir una
    # LISTA de objetos, Python usa __repr__ de cada elemento.
    print("\n=== Extra: lista de Teletubbies (usa __repr__) ===")
    print([tinky_winky, po])

    # ------------------------------------------------------------
    # PASO 3: DEMOSTRACIÓN DE LA CUESTIÓN TEÓRICA
    # ------------------------------------------------------------
    print("\n=== Cuestión teórica: ¿y si toco po.__nivel_hambre? ===")

    # a) LEER desde fuera -> AttributeError (no existe con ese nombre).
    try:
        print(po.__nivel_hambre)
    except AttributeError as error:
```

```
        print("Al leer po.__nivel_hambre -> AttributeError:", error)

    # b) ASIGNAR desde fuera -> NO da error, pero crea un atributo NUEVO y
    #    público llamado literalmente "__nivel_hambre" (fuera de la clase no
    #    hay name mangling). El privado de verdad NO cambia.
    po.__nivel_hambre = 999
    print("He hecho po.__nivel_hambre = 999")
    print("po.__nivel_hambre (atributo nuevo):", po.__nivel_hambre)
    print("ver_estado() sigue igual ->", po.ver_estado())

    # c) Dónde está realmente el privado: vars() muestra el diccionario de
    #    atributos del objeto. Se ve el nombre "mangleado" y el nuevo falso.
    print("Atributos reales de po:", vars(po))

    # d) El nombre real es _PersonajeTeletubby__nivel_hambre. Se PUEDE
    #    acceder con él (Python no lo prohíbe del todo), pero es mala
    #    práctica: te saltas el encapsulamiento a propósito.
    print("po._PersonajeTeletubby__nivel_hambre =", po._PersonajeTeletubby__nivel_hambre)


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === 1. Creamos los Teletubbies ===
# Teletubby Tinky Winky (Morado) - Feliz: 100
# Teletubby Po (Rojo) - Feliz: 100
# PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso')
# PersonajeTeletubby('Po', 'Rojo', 'Círculo', 'Patinete')
#
# === 2. Saludos ===
# ¡Hola! Soy Tinky Winky, soy de color Morado y mi antena tiene forma de Triángulo.
# ¡Tubby abrazo!
# ¡Hola! Soy Po, soy de color Rojo y mi antena tiene forma de Círculo.
# ¡Tubby abrazo!
#
# === 3. A jugar ===
# Tinky Winky está jugando con su Bolso.
# Po está jugando con su Patinete.
#
# === 4. Estado después de jugar ===
# Estado de Tinky Winky -> Felicidad: 110 | Hambre: 70
# Estado de Po -> Felicidad: 110 | Hambre: 70
#
# === 5. Solo Po come ===
# Po se está comiendo su Tubby Papilla. ¡Ñam, ñam!
#
# === 6. Estado final ===
# Estado de Tinky Winky -> Felicidad: 110 | Hambre: 70
# Estado de Po -> Felicidad: 130 | Hambre: 0
#
# === Extra: lista de Teletubbies (usa __repr__) ===
# [PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso'), PersonajeTeletubby('Po', 'Rojo', 'Círculo', 'Patinete')]
#
# === Cuestión teórica: ¿y si toco po.__nivel_hambre? ===
# Al leer po.__nivel_hambre -> AttributeError: 'PersonajeTeletubby' object has no attribute '__nivel_hambre'
# He hecho po.__nivel_hambre = 999
# po.__nivel_hambre (atributo nuevo): 999
# ver_estado() sigue igual -> Estado de Po -> Felicidad: 130 | Hambre: 0
# Atributos reales de po: {'nombre': 'Po', 'color': 'Rojo', 'antena': 'Círculo', 'objeto_favorito': 'Patinete', '_PersonajeTeletubby__nivel_hambre': 0,
'_PersonajeTeletubby__nivel_felicidad': 130, '__nivel_hambre': 999}
# po._PersonajeTeletubby__nivel_hambre = 0
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar self en la definición: def jugar():  -> TypeError al llamar po.jugar()
#   ("takes 0 positional arguments but 1 was given").
# - Escribir nivel_hambre += 20 sin self.__ -> UnboundLocalError / variable local.
# - Un solo guion bajo (_nivel_hambre) NO es privado para Python: es solo
#   una convención "no lo toques". El enunciado pide DOS (__).
# - ver_estado() con print en vez de return -> print(po.ver_estado())
#   imprimiría el texto y después "None". Pierdes puntos: pide "devuelve".
# - Poner saludo_oficial dentro de __init__ como self.saludo_oficial -> sería
#   de instancia, no de clase. El enunciado pide variable DE CLASE.
# - __str__ y __repr__ deben hacer return de un str; si haces print dentro,
#   da TypeError ("__str__ returned non-string").
# - Olvidar los paréntesis: po.jugar (sin ()) no ejecuta nada.
# - En comer_tubby_papilla, "reduce a 0" es = 0, no -= 0 ni -= 20.
# - El enunciado no pone límite a la felicidad, así que puede superar 100.
#
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
# - Poner los privados como parámetros con valor por defecto:
#     def __init__(self, nombre, color, antena, objeto_favorito,
#                  nivel_hambre=50, nivel_felicidad=100):
#   Es válido, pero el enunciado solo pide los 4 públicos en el constructor.
# - @property en lugar de ver_estado(): es más "pythónico", pero el
#   enunciado pide un MÉTODO tipo getter llamado ver_estado().
# - En los apuntes aparece otra versión de __str__:
#     f"Teletubby {self.nombre} (Color: {self.color}) - Estado actual: Hambre {...}"
#   Aquí se ha usado el formato del EJEMPLO del enunciado ("- Feliz: 100").
#   Si en el examen te dan otro ejemplo, copia EXACTAMENTE ese formato.
# ------------------------------------------------------------------


# ================================================================
# RESPUESTA A LA CUESTIÓN TEÓRICA (lista para copiar)
# ================================================================
# ¿Qué ocurriría si intentaras modificar po.__nivel_hambre directamente
# desde el bloque principal? Justifica por qué se utilizan atributos privados.
#
# Al definir self.__nivel_hambre dentro de la clase, Python aplica el
# mecanismo de "name mangling": el atributo se guarda internamente con el
# nombre _PersonajeTeletubby__nivel_hambre. Por eso, desde el bloque
# principal:
#
```

```
#   - Si intento LEER po.__nivel_hambre, se produce un AttributeError,
#     porque el objeto no tiene ningún atributo con ese nombre exacto.
#
#   - Si intento ASIGNAR po.__nivel_hambre = 999, no da error, pero no se
#     modifica el atributo privado: fuera de la clase no hay name mangling,
#     así que Python crea un atributo NUEVO y público llamado __nivel_hambre
#     en ese objeto. El valor real (_PersonajeTeletubby__nivel_hambre) sigue
#     intacto, y ver_estado() seguirá mostrando el hambre verdadera.
#
#   - Técnicamente se podría acceder con po._PersonajeTeletubby__nivel_hambre,
#     porque en Python la privacidad no es absoluta como en Java, pero hacerlo
#     es una mala práctica que rompe el encapsulamiento.
#
# Se usan atributos privados para aplicar el ENCAPSULAMIENTO: los niveles de
# hambre y felicidad son el estado interno del personaje y solo deben cambiar
# a través de sus métodos (jugar() suma 20 de hambre y 10 de felicidad,
# comer_tubby_papilla() pone el hambre a 0 y suma 20 de felicidad). Así se
# evita que desde fuera se pongan valores incoherentes (por ejemplo, hambre
# negativa o un texto), se protege la lógica del objeto y, si algún día
# cambia la forma de calcular estos niveles, solo hay que tocar la clase.
# Para consultarlos desde fuera se usa el método getter ver_estado(), que
# devuelve la información de forma controlada sin permitir modificarla.
# ============================================================
```

04_clases_listas_diccionarios_lambda/02_teletubbies_ampliado.py

```
"""
TEMA 6 - Ejercicio 2: Teletubbies ampliado (lista de objetos, diccionarios y lambda)
================================================================================
ENUNCIADO (ampliación para practicar el tema 6 completo; no es literal del profesor):
    Partiendo de la clase PersonajeTeletubby de la práctica evaluada:
    - Añade getters (get_nivel_hambre, get_nivel_felicidad) y un setter con
      validación (set_nivel_hambre: solo enteros entre 0 y 100).
    - Crea una LISTA con los cuatro Teletubbies (lista de objetos).
    - Recórrela con for, busca uno por nombre, filtra con una comprensión de
      listas los que tienen hambre > 50 y ordénalos con sorted(key=lambda ...).
    - Crea un diccionario nombre -> objeto y otro {nombre: color} con una
      comprensión de diccionario.

FORMA QUE EXIGE EL PROFESOR:
    - Clase PersonajeTeletubby igual que en la práctica (atributo de clase,
      privados con __, ver_estado() que DEVUELVE un str, __str__ y __repr__).
    - Para filtrar/transformar en una línea: COMPRENSIÓN DE LISTAS.
    - Para ordenar por un atributo: sorted(lista, key=lambda t: t.atributo).
    - Recorrer diccionarios con .items() / .values().
    - Pruebas dentro de if __name__ == "__main__":

CONCEPTOS QUE PRACTICA:
    - Lista de objetos: añadir, recorrer, buscar, filtrar, ordenar.
    - Getters y setters con validación (los privados solo se tocan desde la clase).
    - Diccionario de objetos y comprensión de diccionario.
    - lambda como "key" de sorted y max.

CÓMO EJECUTARLO:  python 02_teletubbies_ampliado.py   (en Linux/Docker: python3 02_teletubbies_ampliado.py)
"""


# ------------------------------------------------------------------
# PASO 1: LA CLASE (la misma de la práctica + getters y setter)
# ------------------------------------------------------------------
class PersonajeTeletubby:
    """Representa a un personaje Teletubby con sus niveles de hambre y felicidad."""

    saludo_oficial = "¡Tubby abrazo!"       # atributo de CLASE (compartido)

    def __init__(self, nombre, color, antena, objeto_favorito):
        """Crea un Teletubby con datos públicos y niveles privados por defecto."""
        self.nombre = nombre                # públicos
        self.color = color
        self.antena = antena
        self.objeto_favorito = objeto_favorito
        self.__nivel_hambre = 50            # privados (name mangling)
        self.__nivel_felicidad = 100

    # ---- Métodos de la práctica ----
    def saludar(self):
        """Muestra una presentación con nombre, color y forma de antena."""
        print(f"¡Hola! Soy {self.nombre}, soy de color {self.color} "
              f"y mi antena tiene forma de {self.antena}.")

    def jugar(self):
        """Juega con su objeto favorito: +20 de hambre y +10 de felicidad."""
        print(f"{self.nombre} está jugando con su {self.objeto_favorito}.")
        self.__nivel_hambre += 20
        self.__nivel_felicidad += 10

    def comer_tubby_papilla(self):
        """Come Tubby Papilla: hambre a 0 y +20 de felicidad."""
        print(f"{self.nombre} se está comiendo su Tubby Papilla. ¡Ñam, ñam!")
        self.__nivel_hambre = 0
        self.__nivel_felicidad += 20

    def ver_estado(self):
        """Getter: devuelve un texto con los niveles de felicidad y hambre."""
        return (f"Estado de {self.nombre} -> Felicidad: {self.__nivel_felicidad}"
                f" | Hambre: {self.__nivel_hambre}")

    # ---- GETTERS nuevos ----
    # Un getter DEVUELVE el valor de un privado. Hacen falta porque desde
    # fuera no se puede escribir t.__nivel_hambre (AttributeError) y para
```

```python
    # filtrar u ordenar por hambre necesitamos el NÚMERO, no un texto.
    def get_nivel_hambre(self):
        """Devuelve el nivel de hambre (int)."""
        return self.__nivel_hambre

    def get_nivel_felicidad(self):
        """Devuelve el nivel de felicidad (int)."""
        return self.__nivel_felicidad

    # ---- SETTER con validación ----
    # Un setter CAMBIA un privado, pero solo si el valor es correcto.
    # Esa es la gracia del encapsulamiento: la clase controla qué entra.
    def set_nivel_hambre(self, nuevo_nivel):
        """Cambia el nivel de hambre si es un entero entre 0 y 100.

        Devuelve True si lo cambia y False si el valor no es válido.
        """
        # isinstance(x, int) comprueba que sea un entero.
        # 0 <= x <= 100 es una comparación encadenada (en Java: x >= 0 && x <= 100).
        if isinstance(nuevo_nivel, int) and 0 <= nuevo_nivel <= 100:
            self.__nivel_hambre = nuevo_nivel
            return True
        print(f"Valor no válido para el hambre de {self.nombre}: {nuevo_nivel!r} "
              f"(debe ser un entero de 0 a 100).")
        return False

    # ---- Métodos mágicos ----
    def __str__(self):
        """Texto amigable para el usuario (print)."""
        return f"Teletubby {self.nombre} ({self.color}) - Feliz: {self.__nivel_felicidad}"

    def __repr__(self):
        """Texto técnico: cómo se creó el objeto (repr y print de listas)."""
        return (f"PersonajeTeletubby('{self.nombre}', '{self.color}', "
                f"'{self.antena}', '{self.objeto_favorito}')")


# ------------------------------------------------------------------
# PASO 2: FUNCIÓN DE BÚSQUEDA EN UNA LISTA DE OBJETOS
# ------------------------------------------------------------------
def buscar_por_nombre(lista, nombre):
    """Devuelve el Teletubby cuyo nombre coincide (sin mirar mayúsculas) o None."""
    for teletubby in lista:
        # .lower() en los dos lados para que "po" encuentre a "Po".
        if teletubby.nombre.lower() == nombre.lower():
            return teletubby        # en cuanto lo encuentra, sale de la función
    return None                     # si el for termina sin encontrarlo


# ------------------------------------------------------------------
# PASO 3: PRUEBAS
# ------------------------------------------------------------------
if __name__ == "__main__":

    # 3.1 LISTA DE OBJETOS: cada elemento es un objeto PersonajeTeletubby.
    print("=== Lista de Teletubbies ===")
    teletubbies = [
        PersonajeTeletubby("Tinky Winky", "Morado", "Triángulo", "Bolso"),
        PersonajeTeletubby("Dipsy", "Verde", "Palo", "Sombrero"),
        PersonajeTeletubby("Laa-Laa", "Amarillo", "Rizo", "Pelota"),
    ]
    # append() añade un objeto al final de la lista.
    teletubbies.append(PersonajeTeletubby("Po", "Rojo", "Círculo", "Patinete"))
    print(teletubbies)              # print de una lista -> usa __repr__ de cada uno
    print("Hay", len(teletubbies), "Teletubbies")

    # 3.2 RECORRER con for (cada vuelta, t es UN objeto -> t.nombre, t.saludar()...)
    print("\n=== Recorrer con for (usa __str__) ===")
    for t in teletubbies:
        print(t)                    # print de UN objeto -> usa __str__

    # enumerate(..., start=1) da posición y objeto a la vez (forma pedida en bucles).
    print("\n=== Recorrer con enumerate ===")
    for posicion, t in enumerate(teletubbies, start=1):
        print(f"{posicion}. {t.nombre} - antena: {t.antena}")

    # 3.3 Modificar estados para tener datos variados.
    print("\n=== Jugamos un poco ===")
    teletubbies[0].jugar()          # Tinky Winky: hambre 70
    teletubbies[0].jugar()          # Tinky Winky: hambre 90
    teletubbies[1].jugar()          # Dipsy: hambre 70
    teletubbies[3].comer_tubby_papilla()    # Po: hambre 0
    for t in teletubbies:
        print(t.ver_estado())

    # 3.4 GETTERS y SETTER con validación.
    print("\n=== Getters y setter ===")
    laa_laa = teletubbies[2]
    print("Hambre de Laa-Laa (getter):", laa_laa.get_nivel_hambre())
    laa_laa.set_nivel_hambre(60)            # válido -> cambia
    print("Tras set_nivel_hambre(60):", laa_laa.get_nivel_hambre())
    laa_laa.set_nivel_hambre(150)           # fuera de rango -> no cambia
    laa_laa.set_nivel_hambre("mucha")       # no es int -> no cambia
    print("Sigue en:", laa_laa.get_nivel_hambre())

    # 3.5 BUSCAR por nombre.
    print("\n=== Buscar por nombre ===")
    encontrado = buscar_por_nombre(teletubbies, "dipsy")
    if encontrado:                  # None se evalúa como False
        print("Encontrado:", encontrado)
    else:
        print("No encontrado")
```

```python
    if buscar_por_nombre(teletubbies, "Noo-Noo") is None:
        print("Noo-Noo no está en la lista")

    # 3.6 FILTRAR con comprensión de listas: los que tienen hambre > 50.
    #     Forma: [expresión for elemento in lista if condición]
    #     OJO: hay que usar el getter; t.__nivel_hambre daría AttributeError.
    print("\n=== Filtrar: hambre > 50 ===")
    hambrientos = [t for t in teletubbies if t.get_nivel_hambre() > 50]
    print(hambrientos)
    # Solo los nombres (la expresión puede ser cualquier cosa, no solo t):
    nombres_hambrientos = [t.nombre for t in hambrientos]
    print(nombres_hambrientos)
    # El for equivalente (funciona, pero el profesor pide comprensión para esto):
    #   hambrientos = []
    #   for t in teletubbies:
    #       if t.get_nivel_hambre() > 50:
    #           hambrientos.append(t)

    # 3.7 ORDENAR con sorted(key=lambda ...).
    #     key recibe una función que dice POR QUÉ ordenar. La lambda recibe un
    #     Teletubby (t) y devuelve el dato por el que se compara.
    #     sorted() devuelve una lista NUEVA; la original no cambia.
    print("\n=== Ordenar ===")
    por_hambre = sorted(teletubbies, key=lambda t: t.get_nivel_hambre())
    print("Por hambre (menor a mayor):", [t.nombre for t in por_hambre])
    por_hambre_desc = sorted(teletubbies, key=lambda t: t.get_nivel_hambre(), reverse=True)
    print("Por hambre (mayor a menor):", [t.nombre for t in por_hambre_desc])
    por_nombre = sorted(teletubbies, key=lambda t: t.nombre)
    print("Por nombre:", [t.nombre for t in por_nombre])

    # max / min con key: el MÁS hambriento y el MÁS feliz.
    mas_hambriento = max(teletubbies, key=lambda t: t.get_nivel_hambre())
    mas_feliz = max(teletubbies, key=lambda t: t.get_nivel_felicidad())
    print("Más hambriento:", mas_hambriento.nombre)
    print("Más feliz:", mas_feliz.nombre)

    # 3.8 DICCIONARIO nombre -> objeto (buscar por clave es directo, sin for).
    print("\n=== Diccionario nombre -> objeto ===")
    por_nombre_dic = {}
    for t in teletubbies:
        por_nombre_dic[t.nombre] = t       # añadir clave: valor
    print(por_nombre_dic["Po"])            # acceso directo por clave (__str__)
    print(por_nombre_dic.get("Noo-Noo", "No existe"))   # get evita KeyError
    # Recorrer con .items(): clave y valor a la vez.
    for nombre, t in por_nombre_dic.items():
        print(f"{nombre}: {t.get_nivel_hambre()} de hambre")

    # 3.9 COMPRENSIÓN DE DICCIONARIO: {clave: valor for elemento in lista}
    print("\n=== Comprensión de diccionario ===")
    colores = {t.nombre: t.color for t in teletubbies}
    print(colores)
    # Con if: solo los que tienen hambre > 50, nombre -> hambre.
    hambre_alta = {t.nombre: t.get_nivel_hambre() for t in teletubbies if t.get_nivel_hambre() > 50}
    print(hambre_alta)

    # 3.10 Contar y sumar con la lista de objetos.
    print("\n=== Contar y sumar ===")
    total_hambre = sum(t.get_nivel_hambre() for t in teletubbies)
    print("Hambre total:", total_hambre)
    print("Media de hambre:", total_hambre / len(teletubbies))
    print("Cuántos tienen hambre > 50:", len(hambrientos))


# ----------------------------------------------------------------
# SALIDA ESPERADA:
# === Lista de Teletubbies ===
# [PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso'), PersonajeTeletubby('Dipsy', 'Verde', 'Palo', 'Sombrero'), PersonajeTeletubby('Laa-Laa', 'Amarillo', 'Rizo',
'Pelota'), PersonajeTeletubby('Po', 'Rojo', 'Círculo', 'Patinete')]
# Hay 4 Teletubbies
#
# === Recorrer con for (usa __str__) ===
# Teletubby Tinky Winky (Morado) - Feliz: 100
# Teletubby Dipsy (Verde) - Feliz: 100
# Teletubby Laa-Laa (Amarillo) - Feliz: 100
# Teletubby Po (Rojo) - Feliz: 100
#
# === Recorrer con enumerate ===
# 1. Tinky Winky - antena: Triángulo
# 2. Dipsy - antena: Palo
# 3. Laa-Laa - antena: Rizo
# 4. Po - antena: Círculo
#
# === Jugamos un poco ===
# Tinky Winky está jugando con su Bolso.
# Tinky Winky está jugando con su Bolso.
# Dipsy está jugando con su Sombrero.
# Po se está comiendo su Tubby Papilla. ¡Ñam, ñam!
# Estado de Tinky Winky -> Felicidad: 120 | Hambre: 90
# Estado de Dipsy -> Felicidad: 110 | Hambre: 70
# Estado de Laa-Laa -> Felicidad: 100 | Hambre: 50
# Estado de Po -> Felicidad: 120 | Hambre: 0
#
# === Getters y setter ===
# Hambre de Laa-Laa (getter): 50
# Tras set_nivel_hambre(60): 60
# Valor no válido para el hambre de Laa-Laa: 150 (debe ser un entero de 0 a 100).
# Valor no válido para el hambre de Laa-Laa: 'mucha' (debe ser un entero de 0 a 100).
# Sigue en: 60
#
# === Buscar por nombre ===
# Encontrado: Teletubby Dipsy (Verde) - Feliz: 110
# Noo-Noo no está en la lista
```

```
#
# === Filtrar: hambre > 50 ===
# [PersonajeTeletubby('Tinky Winky', 'Morado', 'Triángulo', 'Bolso'), PersonajeTeletubby('Dipsy', 'Verde', 'Palo', 'Sombrero'), PersonajeTeletubby('Laa-Laa', 'Amarillo', 'Rizo',
'Pelota')]
# ['Tinky Winky', 'Dipsy', 'Laa-Laa']
#
# === Ordenar ===
# Por hambre (menor a mayor): ['Po', 'Laa-Laa', 'Dipsy', 'Tinky Winky']
# Por hambre (mayor a menor): ['Tinky Winky', 'Dipsy', 'Laa-Laa', 'Po']
# Por nombre: ['Dipsy', 'Laa-Laa', 'Po', 'Tinky Winky']
# Más hambriento: Tinky Winky
# Más feliz: Tinky Winky
#
# === Diccionario nombre -> objeto ===
# Teletubby Po (Rojo) - Feliz: 120
# No existe
# Tinky Winky: 90 de hambre
# Dipsy: 70 de hambre
# Laa-Laa: 60 de hambre
# Po: 0 de hambre
#
# === Comprensión de diccionario ===
# {'Tinky Winky': 'Morado', 'Dipsy': 'Verde', 'Laa-Laa': 'Amarillo', 'Po': 'Rojo'}
# {'Tinky Winky': 90, 'Dipsy': 70, 'Laa-Laa': 60}
#
# === Contar y sumar ===
# Hambre total: 220
# Media de hambre: 55.0
# Cuántos tienen hambre > 50: 3
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Filtrar u ordenar con t.__nivel_hambre desde fuera -> AttributeError.
#   Usa el getter: t.get_nivel_hambre()  (¡con paréntesis!).
# - key=lambda t: t.get_nivel_hambre   (sin paréntesis) compara MÉTODOS, no
#   números -> TypeError. Lleva ().
# - sorted() devuelve una lista nueva; lista.sort() ordena la misma lista y
#   devuelve None (no hagas lista = lista.sort()).
# - Empate en max(): devuelve el PRIMERO que encuentra con el valor máximo
#   (Tinky Winky y Po tienen felicidad 120 -> sale Tinky Winky).
# - Si dos objetos tienen el mismo nombre, en el diccionario nombre -> objeto
#   el segundo SOBRESCRIBE al primero (las claves no se repiten).
#
# ALTERNATIVA (funciona, pero NO es lo que pide el tema):
# - Filtrar con filter(lambda t: t.get_nivel_hambre() > 50, teletubbies)
#   -> hay que envolverlo en list(...). Para filtrar el profesor pide
#   comprensión de listas.
# - Buscar con next((t for t in teletubbies if t.nombre == "Po"), None):
#   correcto pero menos legible que el for con return.
# ------------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/03_clases_basico.py

```
"""
TEMA 6 - Teoría-práctica 3: Clases básicas
==========================================
ENUNCIADO: Archivo de consulta (no es un ejercicio del profesor). Muestra con
un ejemplo sencillo (CuentaBancaria y el Dog de los apuntes) todo lo básico:
clase, constructor, self, atributos de clase y de instancia, métodos,
__str__ y __repr__.

FORMA QUE EXIGE EL PROFESOR:
    - Clases en PascalCase (CuentaBancaria), métodos y atributos en snake_case.
    - Docstring con triple comilla en la clase y en cada método.
    - Pruebas dentro de if __name__ == "__main__":

CONCEPTOS QUE PRACTICA:
    - class, __init__, self, instanciar objetos.
    - Atributo de clase (compartido) vs atributo de instancia (de cada objeto).
    - Métodos de instancia que modifican el objeto o devuelven datos.
    - __str__ (usuario, print) vs __repr__ (programador, listas, repr()).

CÓMO EJECUTARLO:  python 03_clases_basico.py   (en Linux/Docker: python3 03_clases_basico.py)
"""


# ------------------------------------------------------------------
# PASO 1: LA CLASE MÍNIMA
# ------------------------------------------------------------------
# class Nombre:  -> define un tipo nuevo. Dentro van atributos y métodos.
class CuentaBancaria:
    """Cuenta bancaria sencilla con titular y saldo."""

    # ATRIBUTO DE CLASE: uno solo para todas las cuentas.
    # Útil para constantes o contadores comunes.
    banco = "Banco Python"
    total_cuentas = 0           # contador de cuentas creadas

    # CONSTRUCTOR: se ejecuta al hacer CuentaBancaria("Ana", 100).
    # saldo=0 es un parámetro con valor por defecto (opcional al crear).
    def __init__(self, titular, saldo=0):
        """Crea la cuenta con su titular y su saldo inicial."""
        self.titular = titular  # ATRIBUTO DE INSTANCIA: cada cuenta el suyo
        self.saldo = saldo
        # Para cambiar un atributo de CLASE se usa el nombre de la clase.
        # Si hicieras self.total_cuentas += 1 crearías uno de instancia.
        CuentaBancaria.total_cuentas += 1

    # MÉTODO que MODIFICA el objeto.
    def ingresar(self, cantidad):
        """Suma la cantidad al saldo si es positiva."""
```

```python
        if cantidad > 0:
            self.saldo += cantidad
        else:
            print("La cantidad debe ser positiva.")

    # MÉTODO que DEVUELVE un dato (return), útil para usarlo fuera.
    def retirar(self, cantidad):
        """Resta la cantidad si hay saldo suficiente. Devuelve True/False."""
        if cantidad <= self.saldo:
            self.saldo -= cantidad
            return True
        return False

    # __str__: lo llama print(objeto) y str(objeto). Para el USUARIO.
    def __str__(self):
        """Texto legible para el usuario."""
        return f"Cuenta de {self.titular}: {self.saldo} €"

    # __repr__: lo llama repr(objeto) y print(lista_de_objetos). Para el
    # PROGRAMADOR: idealmente, el código para volver a crear el objeto.
    def __repr__(self):
        """Texto técnico que muestra cómo se creó el objeto."""
        return f"CuentaBancaria('{self.titular}', {self.saldo})"


# ---------------------------------------------------------------
# PASO 2: EJEMPLO DE LOS APUNTES (Dog) y el ERROR CLÁSICO
# ---------------------------------------------------------------
class Dog:
    """Perro de los apuntes: kind es de clase, name y tricks de instancia."""

    kind = "canine"             # de clase: todos los perros son 'canine'

    def __init__(self, name):
        """Crea el perro con su nombre y su lista de trucos vacía."""
        self.name = name        # de instancia: cada perro su nombre
        self.tricks = []        # de instancia: cada perro SU PROPIA lista

    def add_trick(self, trick):
        """Añade un truco a la lista de este perro."""
        self.tricks.append(trick)


class DogMal:
    """ERROR CLÁSICO: lista mutable como atributo de CLASE (la comparten todos)."""

    tricks = []                 # MAL: una sola lista para TODOS los perros

    def __init__(self, name):
        """Crea el perro (pero tricks no es suyo, es de la clase)."""
        self.name = name

    def add_trick(self, trick):
        """Añade el truco... a la lista compartida."""
        self.tricks.append(trick)


# ---------------------------------------------------------------
# PASO 3: PRUEBAS
# ---------------------------------------------------------------
if __name__ == "__main__":

    print("=== Crear objetos (instanciar) ===")
    # No se escribe "new" como en Java: se llama a la clase como a una función.
    cuenta_ana = CuentaBancaria("Ana", 100)
    cuenta_luis = CuentaBancaria("Luis")       # saldo por defecto: 0
    print(cuenta_ana)                          # __str__
    print(cuenta_luis)
    print(repr(cuenta_ana))                    # __repr__
    print([cuenta_ana, cuenta_luis])           # lista -> __repr__ de cada uno

    print("\n=== Usar métodos ===")
    cuenta_ana.ingresar(50)
    cuenta_luis.ingresar(-10)                  # mensaje de error
    if cuenta_ana.retirar(500):                # usamos lo que devuelve
        print("Retirada hecha")
    else:
        print("Saldo insuficiente")
    print(cuenta_ana)

    print("\n=== Atributos de instancia: cada objeto el suyo ===")
    cuenta_luis.saldo = 20                     # público: se puede cambiar
    print(cuenta_ana.saldo, cuenta_luis.saldo)

    print("\n=== Atributos de clase: compartidos ===")
    print(CuentaBancaria.banco)                # desde la clase
    print(cuenta_ana.banco, cuenta_luis.banco) # desde los objetos (mismo valor)
    print("Cuentas creadas:", CuentaBancaria.total_cuentas)
    CuentaBancaria.banco = "Nuevo Banco"       # cambia para TODOS
    print(cuenta_ana.banco, "|", cuenta_luis.banco)
    # Si se asigna desde un objeto, se crea uno de INSTANCIA que tapa al de clase:
    cuenta_ana.banco = "Banco de Ana"
    print(cuenta_ana.banco, "|", cuenta_luis.banco, "|", CuentaBancaria.banco)

    print("\n=== vars(): atributos de instancia de un objeto ===")
    print(vars(cuenta_luis))

    print("\n=== Dog (bien) vs DogMal (lista de clase) ===")
    d = Dog("Fido")
    e = Dog("Buddy")
    d.add_trick("rodar")
    e.add_trick("hacerse el muerto")
```

```
    print(d.name, d.kind, d.tricks)
    print(e.name, e.kind, e.tricks)
    m1 = DogMal("Rex")
    m2 = DogMal("Toby")
    m1.add_trick("rodar")
    m2.add_trick("sentarse")
    print(m1.name, m1.tricks)                # ¡tiene también el truco de Toby!
    print(m2.name, m2.tricks)


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === Crear objetos (instanciar) ===
# Cuenta de Ana: 100 €
# Cuenta de Luis: 0 €
# CuentaBancaria('Ana', 100)
# [CuentaBancaria('Ana', 100), CuentaBancaria('Luis', 0)]
#
# === Usar métodos ===
# La cantidad debe ser positiva.
# Saldo insuficiente
# Cuenta de Ana: 150 €
#
# === Atributos de instancia: cada objeto el suyo ===
# 150 20
#
# === Atributos de clase: compartidos ===
# Banco Python
# Banco Python Banco Python
# Cuentas creadas: 2
# Nuevo Banco | Nuevo Banco
# Banco de Ana | Nuevo Banco | Nuevo Banco
#
# === vars(): atributos de instancia de un objeto ===
# {'titular': 'Luis', 'saldo': 20}
#
# === Dog (bien) vs DogMal (lista de clase) ===
# Fido canine ['rodar']
# Buddy canine ['hacerse el muerto']
# Rex ['rodar', 'sentarse']
# Toby ['rodar', 'sentarse']
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar self como primer parámetro -> TypeError al llamar al método.
# - Olvidar self. delante del atributo -> se crea una variable local y el
#   objeto no guarda nada.
# - __init__ con UN guion bajo a cada lado (_init_) -> no es el constructor;
#   al crear el objeto con argumentos da TypeError.
# - Listas/diccionarios como atributo de CLASE -> los comparten todos (DogMal).
#   Créalos en __init__ con self.lista = [].
# - __str__ / __repr__ deben DEVOLVER un str (return), no imprimir.
# EN JAVA SERÍA: constructor = __init__, this = self, static = atributo de
# clase, toString() = __str__. No hay new ni tipos declarados.
# ------------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/04_encapsulamiento_getters_setters.py

```
"""
TEMA 6 - Teoría-práctica 4: Encapsulamiento, getters y setters
==============================================================
ENUNCIADO: Archivo de consulta. Muestra atributos públicos, "protegidos" (_)
y privados (__), el name mangling, getters y setters con validación (la forma
de los apuntes) y @property como alternativa más moderna.

FORMA QUE EXIGE EL PROFESOR:
    - Privados con DOBLE guion bajo: self.__atributo (name mangling).
    - Getters y setters como MÉTODOS normales (como ver_estado() de la
      práctica de los Teletubbies): get_x() devuelve, set_x(valor) valida y
      cambia. Es lo que aparece en los apuntes.
    - @property: funciona y es muy pythónico, pero NO es lo que muestran los
      apuntes. Úsalo solo si el enunciado lo pide.

CONCEPTOS QUE PRACTICA:
    - Público / _protegido (convención) / __privado (name mangling).
    - Getter = método que DEVUELVE un privado.
    - Setter = método que VALIDA y cambia un privado.
    - @property y @x.setter.

CÓMO EJECUTARLO:  python 04_encapsulamiento_getters_setters.py   (en Linux/Docker: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: LOS TRES NIVELES DE "VISIBILIDAD"
# ------------------------------------------------------------------
# En Python NO existen public/private como palabras clave. Se usan nombres:
#   nombre       -> público: se usa libremente desde fuera.
#   _nombre      -> "protegido": SOLO una convención ("no lo toques").
#                   Python no hace nada especial; se puede leer desde fuera.
#   __nombre     -> privado: Python lo renombra a _Clase__nombre (name
#                   mangling), así que obj.__nombre desde fuera no existe.
class Alumno:
    """Alumno con nota privada, accesible mediante getter y setter (forma de los apuntes)."""

    def __init__(self, nombre, nota):
        """Crea el alumno. La nota pasa por el setter para validarla."""
        self.nombre = nombre        # público
        self._curso = "2º DAW"      # protegido (convención)
        self.__nota = 0             # privado: valor seguro inicial
        self.set_nota(nota)         # validamos también al crear
```

```python
    # GETTER: devuelve el valor privado. Sin él, no podríamos leer la nota.
    def get_nota(self):
        """Devuelve la nota del alumno."""
        return self.__nota

    # SETTER: solo cambia la nota si es válida. Esta es la ventaja de tener
    # el atributo privado: NADIE puede poner una nota de 15 o "hola".
    def set_nota(self, nueva_nota):
        """Cambia la nota si es un número entre 0 y 10."""
        if isinstance(nueva_nota, (int, float)) and 0 <= nueva_nota <= 10:
            self.__nota = nueva_nota
        else:
            print(f"Nota no válida: {nueva_nota!r}. Se mantiene {self.__nota}.")

    # Un método puede usar el privado para CALCULAR algo (getter "derivado").
    def esta_aprobado(self):
        """Devuelve True si la nota es 5 o más."""
        return self.__nota >= 5

    def __str__(self):
        """Texto para el usuario."""
        return f"{self.nombre} ({self._curso}): {self.__nota}"


# ------------------------------------------------------------------
# PASO 2: ALTERNATIVA CON @property (NO es la forma de los apuntes)
# ------------------------------------------------------------------
# @property convierte un método en algo que se USA COMO UN ATRIBUTO:
#     alumno.nota        -> llama al getter (sin paréntesis)
#     alumno.nota = 7    -> llama al setter (con su validación)
# Por dentro sigue habiendo un privado (self.__nota).
class AlumnoProperty:
    """Igual que Alumno, pero con @property en vez de get_/set_."""

    def __init__(self, nombre, nota):
        """Crea el alumno; la asignación self.nota = nota pasa por el setter."""
        self.nombre = nombre
        self.__nota = 0
        self.nota = nota            # OJO: esto llama al setter de abajo

    @property
    def nota(self):
        """Getter: se usa como alumno.nota (sin paréntesis)."""
        return self.__nota

    @nota.setter
    def nota(self, nueva_nota):
        """Setter: se usa como alumno.nota = valor."""
        if isinstance(nueva_nota, (int, float)) and 0 <= nueva_nota <= 10:
            self.__nota = nueva_nota
        else:
            print(f"Nota no válida: {nueva_nota!r}. Se mantiene {self.__nota}.")


# ------------------------------------------------------------------
# PASO 3: PRUEBAS
# ------------------------------------------------------------------
if __name__ == "__main__":

    print("=== Getters y setters (forma de los apuntes) ===")
    ana = Alumno("Ana", 8)
    print(ana)
    print("Nota con getter:", ana.get_nota())
    ana.set_nota(9.5)                   # válida
    print("Tras set_nota(9.5):", ana.get_nota())
    ana.set_nota(15)                    # no válida
    ana.set_nota("diez")                # no válida
    print("¿Aprobada?", ana.esta_aprobado())

    print("\n=== Validación también en el constructor ===")
    luis = Alumno("Luis", -3)           # no válida -> se queda en 0
    print(luis, "| ¿Aprobado?", luis.esta_aprobado())

    print("\n=== Protegido (_): se puede leer, es solo convención ===")
    print(ana._curso)

    print("\n=== Privado (__): name mangling ===")
    try:
        print(ana.__nota)
    except AttributeError as error:
        print("AttributeError:", error)
    ana.__nota = 0                      # crea un atributo NUEVO, no toca el privado
    print("Tras ana.__nota = 0, get_nota() sigue dando:", ana.get_nota())
    print(vars(ana))
    print("Nombre real:", ana._Alumno__nota, "(se puede, pero NO se debe)")

    print("\n=== Alternativa con @property ===")
    eva = AlumnoProperty("Eva", 6)
    print("eva.nota:", eva.nota)        # getter sin paréntesis
    eva.nota = 7                        # setter con validación
    print("eva.nota:", eva.nota)
    eva.nota = 11                       # rechazado por el setter
    print("eva.nota:", eva.nota)


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === Getters y setters (forma de los apuntes) ===
# Ana (2º DAW): 8
# Nota con getter: 8
# Tras set_nota(9.5): 9.5
```

```
# Nota no válida: 15. Se mantiene 9.5.
# Nota no válida: 'diez'. Se mantiene 9.5.
# ¿Aprobada? True
#
# === Validación también en el constructor ===
# Nota no válida: -3. Se mantiene 0.
# Luis (2º DAW): 0 | ¿Aprobado? False
#
# === Protegido (_): se puede leer, es solo convención ===
# 2º DAW
#
# === Privado (__): name mangling ===
# AttributeError: 'Alumno' object has no attribute '__nota'
# Tras ana.__nota = 0, get_nota() sigue dando: 9.5
# {'nombre': 'Ana', '_curso': '2º DAW', '_Alumno__nota': 9.5, '__nota': 0}
# Nombre real: 9.5 (se puede, pero NO se debe)
#
# === Alternativa con @property ===
# eva.nota: 6
# eva.nota: 7
# Nota no válida: 11. Se mantiene 7.
# eva.nota: 7
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Getter con print en vez de return -> no puedes usar el valor fuera.
# - Llamar al getter sin paréntesis: ana.get_nota  -> muestra
#   <bound method ...>, no la nota. Con @property es al revés: eva.nota()
#   da TypeError porque ya devuelve el número.
# - En @property, si el atributo interno se llama igual que la propiedad
#   (self.nota = ... dentro del setter) -> recursión infinita. Por eso el
#   interno es self.__nota.
# - Un solo guion bajo NO protege nada: es una convención.
# ¿POR QUÉ ENCAPSULAR? La clase controla sus datos: valida lo que entra,
# evita estados incoherentes y puedes cambiar la implementación interna sin
# romper el código que usa la clase.
# EN JAVA SERÍA: private double nota; public double getNota(); public void setNota(double n)
# ----------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/05_herencia_super.py

```
"""
TEMA 6 - Teoría-práctica 5: Herencia, super() y herencia múltiple (MRO)
======================================================================
ENUNCIADO (parte final, literal del profesor - ejercicio integrador):
    "Implementar la herencia múltiple cooperativa utilizando **kwargs, lo que
    permite que super() propague la llamada a través de toda la cadena del MRO
    (Cyborg -> Humano -> Robot) sin perder los argumentos. Definir la clase
    Humano con ejecutar_tarea(...), la clase Robot con ejecutar_tarea(...) y
    la subclase Cyborg(Humano, Robot) probando la ejecución en cascada con
    super(), llamadas explícitas aisladas (Humano.ejecutar_tarea(self, ...)) y
    la consulta del orden MRO mediante Cyborg.mro()."

FORMA QUE EXIGE EL PROFESOR:
    - Herencia: class Hija(Padre):
    - Reutilizar el constructor del padre con super().__init__(...).
    - Herencia múltiple cooperativa: cada clase llama a super() pasando
      **kwargs; consultar el orden con Cyborg.mro().

CONCEPTOS QUE PRACTICA:
    - Herencia simple, super().__init__, añadir atributos propios.
    - Sobrescribir métodos (override) y ampliar el del padre con super().
    - isinstance() / issubclass().
    - Herencia múltiple, MRO (de izquierda a derecha), **kwargs.

CÓMO EJECUTARLO:  python 05_herencia_super.py   (en Linux/Docker: python3 05_herencia_super.py)
"""


# ----------------------------------------------------------------
# PASO 1: HERENCIA SIMPLE
# ----------------------------------------------------------------
# La clase PADRE (base) tiene lo común a todos.
class Animal:
    """Clase base: todo animal tiene nombre y edad, y puede hablar."""

    def __init__(self, nombre, edad):
        """Guarda nombre y edad."""
        self.nombre = nombre
        self.edad = edad

    def hablar(self):
        """Sonido genérico (las hijas lo sobrescriben)."""
        return "..."

    def presentarse(self):
        """Devuelve una presentación usando hablar() (polimorfismo)."""
        # self.hablar() llamará a la versión de la clase HIJA si la tiene.
        return f"Soy {self.nombre}, tengo {self.edad} años y digo {self.hablar()}"

    def __str__(self):
        """Texto para el usuario."""
        return f"{type(self).__name__}: {self.nombre}"


# class Hija(Padre): -> Perro HEREDA todo lo de Animal.
# EN JAVA SERÍA: class Perro extends Animal
class Perro(Animal):
    """Perro: añade raza y sobrescribe hablar()."""

    def __init__(self, nombre, edad, raza):
```

```python
        """Reutiliza el constructor de Animal y añade la raza."""
        # super().__init__(...) llama al __init__ del PADRE para que guarde
        # nombre y edad. Sin esta línea, el perro NO tendría self.nombre.
        # (En Java: super(nombre, edad);) Aquí NO se pasa self.
        super().__init__(nombre, edad)
        self.raza = raza                    # atributo propio de Perro

    # SOBRESCRIBIR (override): mismo nombre que en el padre -> manda este.
    def hablar(self):
        """Sonido del perro."""
        return "¡Guau!"


class Gato(Animal):
    """Gato: no tiene __init__ propio -> usa el de Animal directamente."""

    def hablar(self):
        """Sonido del gato."""
        return "¡Miau!"

    # AMPLIAR un método del padre: llamo a su versión con super() y añado algo.
    def presentarse(self):
        """Presentación del padre + una frase extra."""
        return super().presentarse() + " (y paso de ti)"


# ------------------------------------------------------------------
# PASO 2: HERENCIA MÚLTIPLE COOPERATIVA (ejemplo de los apuntes)
# ------------------------------------------------------------------
# class Cyborg(Humano, Robot): hereda de DOS clases.
# MRO (Method Resolution Order) = orden en que Python busca un método:
#   Cyborg -> Humano -> Robot -> object   (de izquierda a derecha)
# super() NO significa "mi padre", sino "la SIGUIENTE clase en el MRO".
# Por eso, dentro de Humano, super() apunta a Robot cuando el objeto es un Cyborg.
# **kwargs recoge argumentos con nombre extra y los pasa a lo largo de la
# cadena sin perderlos.
class Humano:
    """Parte humana del Cyborg."""

    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        """Ejecuta la tarea y, si hay otra clase después en el MRO, le pasa la llamada."""
        print(f"[Humano] Intentando '{tarea}' con prioridad {prioridad}.")
        # hasattr comprueba si la siguiente clase del MRO tiene ese método.
        # (Si el objeto es un Humano solo, la siguiente es object y no lo tiene.)
        if hasattr(super(), 'ejecutar_tarea'):
            super().ejecutar_tarea(tarea, prioridad, **kwargs)


class Robot:
    """Parte robótica del Cyborg (última de la cadena)."""

    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        """Procesa la tarea; muestra los kwargs que han llegado hasta aquí."""
        print(f"[Robot] Procesando '{tarea}' a nivel {prioridad}.")
        if kwargs:
            print(f"[Robot] Datos extra recibidos: {kwargs}")


class Cyborg(Humano, Robot):
    """Hereda de Humano y de Robot; super() recorre toda la cadena."""

    def ejecutar_tarea(self, tarea, prioridad, **kwargs):
        """Inicia la directiva y delega en la siguiente clase del MRO (Humano)."""
        print("[Cyborg] Iniciando directiva principal...")
        super().ejecutar_tarea(tarea, prioridad, **kwargs)


# ------------------------------------------------------------------
# PASO 3: PRUEBAS
# ------------------------------------------------------------------
if __name__ == "__main__":

    print("=== Herencia simple ===")
    toby = Perro("Toby", 3, "Beagle")
    misi = Gato("Misi", 5)                  # usa el __init__ de Animal
    print(toby, "| raza:", toby.raza)
    print(misi)
    print(toby.presentarse())               # método heredado + hablar() sobrescrito
    print(misi.presentarse())               # método ampliado con super()

    print("\n=== Polimorfismo con una lista de objetos ===")
    animales = [toby, misi, Animal("Genérico", 1)]
    for a in animales:
        print(f"{a.nombre}: {a.hablar()}")  # cada uno usa SU versión

    print("\n=== isinstance / issubclass ===")
    print(isinstance(toby, Perro), isinstance(toby, Animal), isinstance(toby, Gato))
    print(issubclass(Perro, Animal))

    print("\n=== Herencia múltiple: ejecución en cascada con super() ===")
    c = Cyborg()
    c.ejecutar_tarea("Reparar nave", 1, zona="motor")

    print("\n=== Llamadas explícitas aisladas ===")
    # Llamar al método de UNA clase concreta: Clase.metodo(self, ...)
    Robot.ejecutar_tarea(c, "Escanear", 2)  # solo Robot
    print("--")
    Humano.ejecutar_tarea(Humano(), "Leer", 3)   # Humano solo: no hay Robot después

    print("\n=== Orden MRO ===")
    print(Cyborg.mro())
    print([clase.__name__ for clase in Cyborg.mro()])
```

```
# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === Herencia simple ===
# Perro: Toby | raza: Beagle
# Gato: Misi
# Soy Toby, tengo 3 años y digo ¡Guau!
# Soy Misi, tengo 5 años y digo ¡Miau! (y paso de ti)
#
# === Polimorfismo con una lista de objetos ===
# Toby: ¡Guau!
# Misi: ¡Miau!
# Genérico: ...
#
# === isinstance / issubclass ===
# True True False
# True
#
# === Herencia múltiple: ejecución en cascada con super() ===
# [Cyborg] Iniciando directiva principal...
# [Humano] Intentando 'Reparar nave' con prioridad 1.
# [Robot] Procesando 'Reparar nave' a nivel 1.
# [Robot] Datos extra recibidos: {'zona': 'motor'}
#
# === Llamadas explícitas aisladas ===
# [Robot] Procesando 'Escanear' a nivel 2.
# --
# [Humano] Intentando 'Leer' con prioridad 3.
#
# === Orden MRO ===
# [<class '__main__.Cyborg'>, <class '__main__.Humano'>, <class '__main__.Robot'>, <class 'object'>]
# ['Cyborg', 'Humano', 'Robot', 'object']
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar super().__init__(...) en la hija -> AttributeError al usar
#   self.nombre (el padre nunca lo guardó).
# - Escribir super().__init__(self, nombre, edad) -> sobra self: TypeError.
# - Si la hija no define __init__, se usa el del padre (caso Gato).
# - En herencia múltiple el ORDEN de los padres importa:
#   Cyborg(Humano, Robot) busca primero en Humano. Cyborg(Robot, Humano)
#   llamaría primero a Robot (y como Robot no llama a super(), Humano no
#   se ejecutaría).
# - Si quitas **kwargs de alguna clase de la cadena, el argumento zona="motor"
#   provoca TypeError (unexpected keyword argument).
# EN JAVA SERÍA: extends + super(...). Java NO permite herencia múltiple de
# clases (usa interfaces); Python sí, y resuelve el orden con el MRO.
# ------------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/06_listas_de_objetos.py

```
"""
TEMA 6 - Teoría-práctica 6: CRUD sobre una lista de objetos
============================================================
ENUNCIADO: Archivo de consulta. Gestiona una lista de objetos Producto con
las operaciones típicas de examen: añadir (Create), listar y buscar (Read),
modificar (Update), eliminar (Delete), contar, sumar y sacar el máximo y el
mínimo con max/min(key=lambda).

FORMA QUE EXIGE EL PROFESOR:
    - Los objetos se guardan en una lista: inventario = [Producto(...), ...]
    - Recorrer con for (o enumerate(..., start=1) si se muestra numerado).
    - Filtrar/transformar en una línea con comprensión de listas.
    - Ordenar / máximo por un atributo con key=lambda p: p.atributo.
    - Funciones con docstring y pruebas en if __name__ == "__main__":

CONCEPTOS QUE PRACTICA:
    - Lista de objetos y acceso a sus atributos dentro de un for.
    - Funciones que reciben la lista y la modifican o devuelven resultados.
    - remove(), del, pop(); no borrar mientras recorres la lista.
    - sum(), len(), max()/min() con key, sorted() con key.

CÓMO EJECUTARLO:  python 06_listas_de_objetos.py   (en Linux/Docker: python3 06_listas_de_objetos.py)
"""


# ------------------------------------------------------------------
# PASO 1: LA CLASE DE LOS OBJETOS QUE VAMOS A GUARDAR
# ------------------------------------------------------------------
class Producto:
    """Producto de una tienda con nombre, precio, categoría y stock."""

    def __init__(self, nombre, precio, categoria, stock):
        """Crea el producto."""
        self.nombre = nombre
        self.precio = precio
        self.categoria = categoria
        self.stock = stock

    def valor_stock(self):
        """Devuelve precio * stock (dinero que hay en el almacén de este producto)."""
        return self.precio * self.stock

    def __str__(self):
        """Texto para el usuario."""
        return f"{self.nombre} - {self.precio} € ({self.categoria}, stock {self.stock})"

    def __repr__(self):
        """Texto técnico."""
        return f"Producto('{self.nombre}', {self.precio}, '{self.categoria}', {self.stock})"
```

```python
# ------------------------------------------------------------------
# PASO 2: FUNCIONES CRUD
# ------------------------------------------------------------------
# Las listas son MUTABLES: si una función hace lista.append(...) o
# lista.remove(...), el cambio se ve fuera (no hace falta devolver la lista).

def anadir_producto(lista, producto):
    """CREATE: añade el producto al final de la lista."""
    lista.append(producto)


def listar_productos(lista):
    """READ: muestra todos los productos numerados."""
    if not lista:                           # lista vacía -> False
        print("No hay productos.")
        return
    for numero, p in enumerate(lista, start=1):
        print(f"{numero}. {p}")


def buscar_producto(lista, nombre):
    """READ: devuelve el producto con ese nombre (sin mirar mayúsculas) o None."""
    for p in lista:
        if p.nombre.lower() == nombre.lower():
            return p
    return None


def modificar_precio(lista, nombre, nuevo_precio):
    """UPDATE: cambia el precio del producto. Devuelve True si lo encontró."""
    p = buscar_producto(lista, nombre)      # reutilizamos la búsqueda
    if p is None:
        return False
    p.precio = nuevo_precio                 # modificamos EL OBJETO que está en la lista
    return True


def eliminar_producto(lista, nombre):
    """DELETE: quita el producto de la lista. Devuelve True si lo encontró."""
    p = buscar_producto(lista, nombre)
    if p is None:
        return False
    lista.remove(p)                         # remove borra la PRIMERA coincidencia
    return True


# ------------------------------------------------------------------
# PASO 3: PRUEBAS
# ------------------------------------------------------------------
if __name__ == "__main__":

    inventario = []                         # empezamos con la lista vacía

    print("=== CREATE: añadir ===")
    anadir_producto(inventario, Producto("Portátil", 800, "electrónica", 5))
    anadir_producto(inventario, Producto("Ratón", 25, "electrónica", 50))
    anadir_producto(inventario, Producto("Mesa", 200, "muebles", 10))
    anadir_producto(inventario, Producto("Silla", 90, "muebles", 0))
    anadir_producto(inventario, Producto("Teclado", 45, "electrónica", 20))
    print("Productos añadidos:", len(inventario))

    print("\n=== READ: listar ===")
    listar_productos(inventario)

    print("\n=== READ: buscar ===")
    encontrado = buscar_producto(inventario, "mesa")
    print("Buscar 'mesa':", encontrado)
    print("Buscar 'Lámpara':", buscar_producto(inventario, "Lámpara"))

    print("\n=== UPDATE: modificar ===")
    if modificar_precio(inventario, "Ratón", 19.99):
        print("Precio cambiado:", buscar_producto(inventario, "Ratón"))
    if not modificar_precio(inventario, "Lámpara", 30):
        print("No se puede modificar 'Lámpara': no existe")
    # También se puede modificar directamente recorriendo:
    for p in inventario:
        if p.categoria == "muebles":
            p.stock += 5                    # reponemos todos los muebles

    print("\n=== DELETE: eliminar ===")
    print("Eliminar 'Teclado':", eliminar_producto(inventario, "Teclado"))
    print("Eliminar 'Lámpara':", eliminar_producto(inventario, "Lámpara"))
    # Otras formas de borrar por POSICIÓN:
    #   del inventario[0]          -> borra el primero
    #   ultimo = inventario.pop()  -> quita el último y lo devuelve
    listar_productos(inventario)

    print("\n=== Contar ===")
    print("Total de productos:", len(inventario))
    # Contar los que cumplen algo: con contador o con len() de una comprensión.
    contador = 0
    for p in inventario:
        if p.categoria == "electrónica":
            contador += 1
    print("De electrónica (for):", contador)
    print("De electrónica (comprensión):", len([p for p in inventario if p.categoria == "electrónica"]))

    print("\n=== Sumar ===")
    total = 0
    for p in inventario:
```

```
        total += p.valor_stock()
    print("Valor total del almacén (for):", total)
    print("Valor total del almacén (sum):", sum(p.valor_stock() for p in inventario))
    print("Unidades totales:", sum(p.stock for p in inventario))

    print("\n=== Máximo y mínimo con key=lambda ===")
    # max(lista) sin key no sabe comparar objetos -> TypeError.
    # key=lambda p: p.precio  -> "compara los productos por su precio".
    # Devuelve el OBJETO entero, no el número.
    mas_caro = max(inventario, key=lambda p: p.precio)
    mas_barato = min(inventario, key=lambda p: p.precio)
    print("Más caro:", mas_caro)
    print("Más barato:", mas_barato)
    print("Precio máximo (solo el número):", max(p.precio for p in inventario))

    print("\n=== Filtrar y ordenar ===")
    disponibles = [p for p in inventario if p.stock > 0]
    print("Disponibles:", [p.nombre for p in disponibles])
    ordenados = sorted(inventario, key=lambda p: p.precio, reverse=True)
    print("Por precio (mayor a menor):", [p.nombre for p in ordenados])

    print("\n=== Borrar varios cumpliendo una condición (forma segura) ===")
    # NO borres con remove dentro de "for p in inventario" (se salta elementos).
    # Crea una lista nueva con los que QUIERES CONSERVAR:
    inventario = [p for p in inventario if p.precio >= 50]
    print(inventario)


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === CREATE: añadir ===
# Productos añadidos: 5
#
# === READ: listar ===
# 1. Portátil - 800 € (electrónica, stock 5)
# 2. Ratón - 25 € (electrónica, stock 50)
# 3. Mesa - 200 € (muebles, stock 10)
# 4. Silla - 90 € (muebles, stock 0)
# 5. Teclado - 45 € (electrónica, stock 20)
#
# === READ: buscar ===
# Buscar 'mesa': Mesa - 200 € (muebles, stock 10)
# Buscar 'Lámpara': None
#
# === UPDATE: modificar ===
# Precio cambiado: Ratón - 19.99 € (electrónica, stock 50)
# No se puede modificar 'Lámpara': no existe
#
# === DELETE: eliminar ===
# Eliminar 'Teclado': True
# Eliminar 'Lámpara': False
# 1. Portátil - 800 € (electrónica, stock 5)
# 2. Ratón - 19.99 € (electrónica, stock 50)
# 3. Mesa - 200 € (muebles, stock 15)
# 4. Silla - 90 € (muebles, stock 5)
#
# === Contar ===
# Total de productos: 4
# De electrónica (for): 2
# De electrónica (comprensión): 2
#
# === Sumar ===
# Valor total del almacén (for): 8449.5
# Valor total del almacén (sum): 8449.5
# Unidades totales: 75
#
# === Máximo y mínimo con key=lambda ===
# Más caro: Portátil - 800 € (electrónica, stock 5)
# Más barato: Ratón - 19.99 € (electrónica, stock 50)
# Precio máximo (solo el número): 800
#
# === Filtrar y ordenar ===
# Disponibles: ['Portátil', 'Ratón', 'Mesa', 'Silla']
# Por precio (mayor a menor): ['Portátil', 'Mesa', 'Silla', 'Ratón']
#
# === Borrar varios cumpliendo una condición (forma segura) ===
# [Producto('Portátil', 800, 'electrónica', 5), Producto('Mesa', 200, 'muebles', 15), Producto('Silla', 90, 'muebles', 5)]
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - max(inventario) sin key -> TypeError ('>' not supported between Producto).
# - Confundir el objeto con un atributo: print(mas_caro) muestra el producto;
#   mas_caro.precio es el número.
# - remove() de algo que no está -> ValueError. Por eso primero se busca.
# - Borrar con remove DENTRO de un for sobre la misma lista se salta
#   elementos. Usa una comprensión (o recorre una copia: for p in lista[:]).
# - Comparar texto sin .lower(): "mesa" != "Mesa".
# EN JAVA SERÍA: Arraylist<Producto>; aquí la lista no tiene tipo y se
# recorre con "for p in lista" (como el for-each de Java).
# ------------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/07_diccionarios.py

```
"""
TEMA 6 - Teoría-práctica 7: Diccionarios
========================================
ENUNCIADO: Archivo de consulta. Todo lo que se hace con diccionarios en el
examen: crear, acceder ([] vs get), añadir, modificar, borrar (del / pop),
comprobar con in, recorrer (items / keys / values), diccionario de listas,
lista de diccionarios y contar apariciones.
```

```
FORMA QUE EXIGE EL PROFESOR:
    - Recorrer diccionarios con .items() (clave y valor) o .values().
    - Acceso seguro con .get(clave, valor_por_defecto) cuando la clave puede
      no existir (evita KeyError).

CONCEPTOS QUE PRACTICA:
    - Pares clave: valor; las claves no se repiten.
    - dic[clave] (lanza KeyError si no existe) vs dic.get(clave).
    - del, pop, in, len, items, keys, values.
    - Estructuras anidadas: dict de listas y lista de dicts.
    - Contar apariciones (con for + get y con dict).

CÓMO EJECUTARLO:  python 07_diccionarios.py   (en Linux/Docker: python3 07_diccionarios.py)
"""

# ------------------------------------------------------------------
# PASO 1: CREAR Y ACCEDER
# ------------------------------------------------------------------
# Un diccionario guarda pares clave: valor entre llaves.
# EN JAVA SERÍA: HashMap<String, Object>  (pero aquí mantiene el orden de inserción).
pelicula = {
    "titulo": "Inception",
    "year": 2010,
    "director": "Nolan",
    "rating": 8.8,
}
vacio = {}                                  # diccionario vacío (dict() también vale)

print("=== Crear y acceder ===")
print(pelicula)
print(pelicula["titulo"])                   # [] -> si la clave no existe: KeyError
print(pelicula.get("genero"))               # get -> si no existe: None
print(pelicula.get("genero", "n/a"))        # get con valor por defecto
try:
    print(pelicula["genero"])
except KeyError as error:
    print("KeyError con []:", error)
print("Número de claves:", len(pelicula), "| vacío:", len(vacio))

# ------------------------------------------------------------------
# PASO 2: AÑADIR, MODIFICAR, BORRAR
# ------------------------------------------------------------------
print("\n=== Añadir y modificar (misma sintaxis) ===")
pelicula["genero"] = "Ciencia ficción"      # la clave NO existía -> se AÑADE
pelicula["rating"] = 9.0                    # la clave SÍ existía -> se MODIFICA
print(pelicula)

print("\n=== Borrar ===")
del pelicula["year"]                        # borra la clave (KeyError si no existe)
director = pelicula.pop("director")         # borra y DEVUELVE el valor
print("Valor quitado con pop:", director)
print(pelicula.pop("oscar", "no tenía"))    # pop con defecto: no da error
print(pelicula)

# ------------------------------------------------------------------
# PASO 3: COMPROBAR CON in (mira las CLAVES, no los valores)
# ------------------------------------------------------------------
print("\n=== in ===")
print("titulo" in pelicula)                 # True
print("Inception" in pelicula)              # False: es un valor, no una clave
print("Inception" in pelicula.values())     # True: buscando en los valores
if "year" not in pelicula:
    print("Ya no tiene 'year'")

# ------------------------------------------------------------------
# PASO 4: RECORRER
# ------------------------------------------------------------------
precios = {"portátil": 800, "móvil": 500, "tableta": 300}

print("\n=== Recorrer ===")
for producto in precios:                    # por defecto recorre las CLAVES
    print("Clave:", producto)
for producto in precios.keys():             # igual que lo anterior, más explícito
    print("keys():", producto)
total = 0
for precio in precios.values():             # solo los VALORES
    total += precio
print("Total con values():", total)
for producto, precio in precios.items():    # CLAVE y VALOR a la vez (forma pedida)
    print(f"{producto}: {precio} €")

# Modificar todos los valores recorriendo:
for producto in precios:
    precios[producto] = precios[producto] * 0.9     # 10 % de descuento
print("Con descuento:", precios)

# ------------------------------------------------------------------
# PASO 5: DICCIONARIO DE LISTAS (valor = lista)
# ------------------------------------------------------------------
print("\n=== Diccionario de listas ===")
notas = {
    "Ana": [7, 8, 9],
    "Luis": [5, 4, 6],
}
notas["Eva"] = []                           # nuevo alumno sin notas
notas["Eva"].append(10)                     # añadir a la LISTA de Eva
notas["Ana"].append(10)
for alumno, lista_notas in notas.items():
    media = sum(lista_notas) / len(lista_notas)
    print(f"{alumno}: {lista_notas} -> media {media:.2f}")

# ------------------------------------------------------------------
```

```python
# PASO 6: LISTA DE DICCIONARIOS (cada elemento = un "registro")
# ----------------------------------------------------------------
print("\n=== Lista de diccionarios ===")
productos = [
    {"name": "Laptop", "price": 800, "category": "electronics", "stock": 5},
    {"name": "Mouse", "price": 25, "category": "electronics", "stock": 50},
    {"name": "Desk", "price": 200, "category": "furniture", "stock": 10},
]
# Filtrar con for (ejemplo de los apuntes):
search_category = "electronics"
available_products = []
for product in productos:
    if product["category"] == search_category and product["stock"] > 0:
        available_products.append(product)
print("Disponibles:", [p["name"] for p in available_products])
# Ordenar por una clave con lambda (como en los apuntes con peliculas.sort):
productos.sort(key=lambda p: p["price"], reverse=True)
print("Por precio desc:", [p["name"] for p in productos])
# El más barato:
barato = min(productos, key=lambda p: p["price"])
print("Más barato:", barato["name"], barato["price"])

# ----------------------------------------------------------------
# PASO 7: CONTAR APARICIONES
# ----------------------------------------------------------------
print("\n=== Contar apariciones ===")
frutas = ["manzana", "pera", "manzana", "uva", "pera", "manzana"]
# Forma 1: if/else
conteo = {}
for fruta in frutas:
    if fruta in conteo:
        conteo[fruta] += 1                  # ya estaba: sumamos 1
    else:
        conteo[fruta] = 1                   # primera vez: empieza en 1
print(conteo)
# Forma 2: get con 0 por defecto (más corta)
conteo2 = {}
for fruta in frutas:
    conteo2[fruta] = conteo2.get(fruta, 0) + 1
print(conteo2)
# Contar letras de una palabra:
letras = {}
for letra in "banana":
    letras[letra] = letras.get(letra, 0) + 1
print(letras)
# La más repetida: max sobre las claves usando el valor como criterio.
print("Fruta más repetida:", max(conteo, key=lambda f: conteo[f]))


# ----------------------------------------------------------------
# SALIDA ESPERADA:
# === Crear y acceder ===
# {'titulo': 'Inception', 'year': 2010, 'director': 'Nolan', 'rating': 8.8}
# Inception
# None
# n/a
# KeyError con []: 'genero'
# Número de claves: 4 | vacio: 0
#
# === Añadir y modificar (misma sintaxis) ===
# {'titulo': 'Inception', 'year': 2010, 'director': 'Nolan', 'rating': 9.0, 'genero': 'Ciencia ficción'}
#
# === Borrar ===
# Valor quitado con pop: Nolan
# no tenia
# {'titulo': 'Inception', 'rating': 9.0, 'genero': 'Ciencia ficción'}
#
# === in ===
# True
# False
# True
# Ya no tiene 'year'
#
# === Recorrer ===
# Clave: portátil
# Clave: móvil
# Clave: tableta
# keys(): portátil
# keys(): móvil
# keys(): tableta
# Total con values(): 1600
# portátil: 800 €
# móvil: 500 €
# tableta: 300 €
# Con descuento: {'portátil': 720.0, 'móvil': 450.0, 'tableta': 270.0}
#
# === Diccionario de listas ===
# Ana: [7, 8, 9, 10] -> media 8.50
# Luis: [5, 4, 6] -> media 5.00
# Eva: [10] -> media 10.00
#
# === Lista de diccionarios ===
# Disponibles: ['Laptop', 'Mouse']
# Por precio desc: ['Laptop', 'Desk', 'Mouse']
# Más barato: Mouse 25
#
# === Contar apariciones ===
# {'manzana': 3, 'pera': 2, 'uva': 1}
# {'manzana': 3, 'pera': 2, 'uva': 1}
# {'b': 1, 'a': 3, 'n': 2}
# Fruta más repetida: manzana
# ----------------------------------------------------------------
```

```
# OJO / ERRORES TÍPICOS:
# - dic["clave"] con una clave que no existe -> KeyError. Usa get() o "in".
# - "valor" in dic busca en las CLAVES; para valores: in dic.values().
# - for k, v in dic:  (sin .items()) recorre solo las CLAVES e intenta
#   partir cada una en dos -> normalmente ValueError. Lleva .items().
# - No añadas ni borres claves MIENTRAS recorres el mismo diccionario
#   (RuntimeError). Cambiar el VALOR de claves existentes sí se puede.
# - Las claves deben ser inmutables (str, int, tupla); una lista no vale.
# - Asignar a una clave que ya existe la SOBRESCRIBE (no hay duplicados).
# ------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/08_comprensiones_y_lambda.py

```
"""
TEMA 6 - Teoría-práctica 8: Comprensiones y lambda
==================================================
ENUNCIADO: Archivo de consulta. Comprensiones de listas (básica, con if, con
if-else) y de diccionario; lambda con map, filter, sorted, max y min. Cada
ejemplo va comparado con el for equivalente.

FORMA QUE EXIGE EL PROFESOR:
    - Filtrados y transformaciones "en una sola línea" -> COMPRENSIÓN DE LISTAS.
    - Elevar al cuadrado con map + lambda (Funciones ej. 13):
          cuadrados = list(map(lambda x: x**2, numeros))
    - Ordenar por un criterio: sorted(..., key=lambda ...) o lista.sort(key=...).
    - Si el enunciado dice "con map" o "con lambda", úsalos aunque una
      comprensión también funcione (y viceversa).

CONCEPTOS QUE PRACTICA:
    - [expresión for x in iterable]
    - [expresión for x in iterable if condición]       (FILTRA)
    - [valor_si for x in iterable ... ] con if-else ANTES del for (TRANSFORMA)
    - {clave: valor for x in iterable}
    - lambda parámetros: expresión
    - map, filter, sorted, max, min con key.

CÓMO EJECUTARLO:  python 08_comprensiones_y_lambda.py   (en Linux/Docker: python3 08_comprensiones_y_lambda.py)
"""

numeros = [1, 2, 3, 4, 5, 6]

# ------------------------------------------------------------
# PASO 1: COMPRENSIÓN DE LISTAS BÁSICA
# ------------------------------------------------------------
# Se lee: "dame n ** 3 PARA CADA n EN numeros".
print("=== 1. Comprensión básica: cubos (Bucles ej. 7) ===")
# Con for:
cubos_for = []
for n in numeros:
    cubos_for.append(n ** 3)
# Con comprensión (lo mismo en una línea):
cubos = [n ** 3 for n in numeros]
print(cubos_for)
print(cubos)

# La expresión puede ser cualquier cosa: métodos, f-strings...
palabras = ["sol", "murciélago", "mar", "elefante", "luz"]
print([p.upper() for p in palabras])
print([len(p) for p in palabras])

# ------------------------------------------------------------
# PASO 2: COMPRENSIÓN CON if (FILTRAR) -> el if va AL FINAL
# ------------------------------------------------------------
print("\n=== 2. Con if (filtra): pares y palabras largas ===")
# Con for:
pares_for = []
for n in numeros:
    if n % 2 == 0:
        pares_for.append(n)
# Con comprensión:
pares = [n for n in numeros if n % 2 == 0]
print(pares_for)
print(pares)
# Bucles ej. 8: palabras con más de 3 letras
largas = [p for p in palabras if len(p) > 3]
print(largas)

# ------------------------------------------------------------
# PASO 3: COMPRENSIÓN CON if-else (TRANSFORMAR) -> va ANTES del for
# ------------------------------------------------------------
# Aquí NO se quita ningún elemento: a cada uno se le da un valor u otro.
print("\n=== 3. Con if-else (transforma) ===")
# Con for:
tipos_for = []
for n in numeros:
    if n % 2 == 0:
        tipos_for.append("par")
    else:
        tipos_for.append("impar")
# Con comprensión:
tipos = ["par" if n % 2 == 0 else "impar" for n in numeros]
print(tipos_for)
print(tipos)
# Otro: los negativos se convierten en 0
valores = [3, -1, 7, -5, 0]
print([v if v > 0 else 0 for v in valores])

# ------------------------------------------------------------
# PASO 4: COMPRENSIÓN DE DICCIONARIO
# ------------------------------------------------------------
```

```python
print("\n=== 4. Comprensión de diccionario ===")
# Con for:
cuadrados_for = {}
for n in range(5):
    cuadrados_for[n] = n ** 2
# Con comprensión: {clave: valor for ...}
cuadrados_dict = {n: n ** 2 for n in range(5)}
print(cuadrados_for)
print(cuadrados_dict)
# Palabra -> longitud, solo las largas (con if al final, igual que en listas)
print({p: len(p) for p in palabras if len(p) > 3})
# Transformar un diccionario existente con .items(): precios con IVA
precios = {"pan": 1.0, "leche": 2.0, "café": 4.0}
print({prod: round(precio * 1.21, 2) for prod, precio in precios.items()})

# ------------------------------------------------------------------
# PASO 5: LAMBDA = FUNCIÓN ANÓNIMA DE UNA LÍNEA
# ------------------------------------------------------------------
# Lambda parámetros: expresión   -> devuelve la expresión (sin escribir return).
print("\n=== 5. lambda ===")


def cuadrado(x):
    """Función normal equivalente a la lambda de abajo."""
    return x ** 2


cuadrado_lambda = lambda x: x ** 2      # (en la práctica no se guarda en variable)
suma = lambda a, b: a + b
print(cuadrado(4), cuadrado_lambda(4), suma(2, 3))

# ------------------------------------------------------------------
# PASO 6: map(función, lista) -> aplica la función a CADA elemento
# ------------------------------------------------------------------
print("\n=== 6. map + lambda (Funciones ej. 13) ===")
numeros_map = [1, 2, 3, 4, 5]
# Con for:
cuadrados_for = []
for x in numeros_map:
    cuadrados_for.append(x ** 2)
# Con map + lambda (FORMA EXIGIDA en el ej. 13). map devuelve un objeto
# "perezoso": hay que convertirlo con list() para verlo.
cuadrados = list(map(lambda x: x ** 2, numeros_map))
# Con comprensión (equivalente):
cuadrados_comp = [x ** 2 for x in numeros_map]
print(cuadrados_for)
print("Cuadrados: ", cuadrados)
print(cuadrados_comp)
print(type(map(lambda x: x ** 2, numeros_map)).__name__)  # sin list() es un objeto 'map', no una lista
print(list(map(str.upper, ["a", "b"])))                 # map con función ya existente

# ------------------------------------------------------------------
# PASO 7: filter(función, lista) -> se queda con los que devuelven True
# ------------------------------------------------------------------
print("\n=== 7. filter + lambda ===")
# Con for:
impares_for = []
for n in numeros:
    if n % 2 != 0:
        impares_for.append(n)
# Con filter + lambda:
impares = list(filter(lambda n: n % 2 != 0, numeros))
# Con comprensión:
impares_comp = [n for n in numeros if n % 2 != 0]
print(impares_for, impares, impares_comp)

# ------------------------------------------------------------------
# PASO 8: sorted / sort con key=Lambda
# ------------------------------------------------------------------
# key = función que dice POR QUÉ dato comparar cada elemento.
print("\n=== 8. sorted con key ===")
print(sorted(palabras))                             # alfabético
print(sorted(palabras, key=lambda p: len(p)))       # por longitud
print(sorted(palabras, key=len, reverse=True))      # igual, con función existente, desc
alumnos = [("Ana", 8.5), ("Luis", 6.0), ("Eva", 9.2)]
print(sorted(alumnos, key=lambda a: a[1], reverse=True))   # por nota desc (posición 1)
peliculas = [{"titulo": "A", "rating": 8}, {"titulo": "B", "rating": 9}]
peliculas.sort(key=lambda p: p["rating"], reverse=True)    # sort cambia la MISMA lista
print(peliculas)
# Con for habría que programar el algoritmo de ordenación a mano: por eso se usa sorted.

# ------------------------------------------------------------------
# PASO 9: max / min con key
# ------------------------------------------------------------------
print("\n=== 9. max / min con key ===")
# Con for:
mejor = alumnos[0]
for a in alumnos:
    if a[1] > mejor[1]:
        mejor = a
print("Mejor (for):", mejor)
print("Mejor (max):", max(alumnos, key=lambda a: a[1]))
print("Peor (min):", min(alumnos, key=lambda a: a[1]))
print("Palabra más larga:", max(palabras, key=len))


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# === 1. Comprensión básica: cubos (Bucles ej. 7) ===
# [1, 8, 27, 64, 125, 216]
# [1, 8, 27, 64, 125, 216]
# ['SOL', 'MURCIÉLAGO', 'MAR', 'ELEFANTE', 'LUZ']
```

```
# [3, 10, 3, 8, 3]
#
# === 2. Con if (filtra): pares y palabras largas ===
# [2, 4, 6]
# [2, 4, 6]
# ['murciélago', 'elefante']
#
# === 3. Con if-else (transforma) ===
# ['impar', 'par', 'impar', 'par', 'impar', 'par']
# ['impar', 'par', 'impar', 'par', 'impar', 'par']
# [3, 0, 7, 0, 0]
#
# === 4. Comprensión de diccionario ===
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
# {'murciélago': 10, 'elefante': 8}
# {'pan': 1.21, 'leche': 2.42, 'café': 4.84}
#
# === 5. lambda ===
# 16 16 5
#
# === 6. map + lambda (Funciones ej. 13) ===
# [1, 4, 9, 16, 25]
# Cuadrados:  [1, 4, 9, 16, 25]
# [1, 4, 9, 16, 25]
# map
# ['A', 'B']
#
# === 7. filter + lambda ===
# [1, 3, 5] [1, 3, 5] [1, 3, 5]
#
# === 8. sorted con key ===
# ['elefante', 'luz', 'mar', 'murciélago', 'sol']
# ['sol', 'mar', 'luz', 'elefante', 'murciélago']
# ['murciélago', 'elefante', 'sol', 'mar', 'luz']
# [('Eva', 9.2), ('Ana', 8.5), ('Luis', 6.0)]
# [{'titulo': 'B', 'rating': 9}, {'titulo': 'A', 'rating': 8}]
#
# === 9. max / min con key ===
# Mejor (for): ('Eva', 9.2)
# Mejor (max): ('Eva', 9.2)
# Peor (min): ('Luis', 6.0)
# Palabra más larga: murciélago
# ------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - if para FILTRAR va al FINAL: [x for x in l if cond]
#   if-else para TRANSFORMAR va DELANTE: [a if cond else b for x in l]
#   [x for x in l if cond else 0] -> SyntaxError.
# - map y filter devuelven un objeto, no una lista: envuélvelos en list().
# - lista.sort() devuelve None: no hagas nueva = lista.sort().
# - key=len (sin paréntesis): se pasa la función, no se llama.
# - Una lambda solo admite UNA expresión (sin return, sin varias líneas).
# - En sorted con empates se mantiene el orden original ('sol', 'mar', 'luz').
# EN JAVA SERÍA: lambda = x -> x * x ; map/filter = stream().map().filter().
# ALTERNATIVA: casi todo map/filter se puede hacer con comprensión (y al
# revés). Funciona igual, pero usa la forma que nombre el enunciado.
# ------------------------------------------------------------
```

04_clases_listas_diccionarios_lambda/09_resumen_poo.py

```
"""
TEMA 6 - 9: CHULETA EJECUTABLE (clases, listas de objetos, dicts, comprensiones, lambda)
================================================================================
Lo mínimo de cada cosa, para copiar y adaptar en el examen.
Detalle y explicaciones: archivos 01 a 08 de esta carpeta.

CÓMO EJECUTARLO:  python 09_resumen_poo.py   (en Linux/Docker: python3 09_resumen_poo.py)
"""


# ==================== CLASE COMPLETA ====================
class Personaje:
    """Plantilla mínima: clase, instancia, privado, getter, setter, dunder."""

    especie = "Teletubby"                       # atributo de CLASE (compartido)

    def __init__(self, nombre, energia=50):     # constructor; self = el objeto
        """Crea el personaje."""
        self.nombre = nombre                    # público (de instancia)
        self.__energia = energia                # privado -> _Personaje__energia

    def get_energia(self):                      # GETTER: return, no print
        """Devuelve la energía."""
        return self.__energia

    def set_energia(self, valor):               # SETTER con validación
        """Cambia la energía si está entre 0 y 100."""
        if 0 <= valor <= 100:
            self.__energia = valor
        else:
            print("Valor no válido")

    def jugar(self):                            # método que modifica el estado
        """Gasta 10 de energía."""
        self.__energia -= 10

    def __str__(self):                          # print(obj) -> usuario
        """Texto para el usuario."""
        return f"{self.nombre} ({self.__energia})"
```

```python
    def __repr__(self):                         # repr(obj) / print(lista) -> técnico
        """Texto técnico."""
        return f"Personaje('{self.nombre}', {self.__energia})"


# ===================== HERENCIA =====================
class Heroe(Personaje):                         # Heroe hereda de Personaje
    """Personaje con poder."""

    def __init__(self, nombre, poder, energia=50):
        """Reutiliza el constructor del padre."""
        super().__init__(nombre, energia)       # SIN self
        self.poder = poder

    def jugar(self):                            # sobrescribe y amplía
        """Juega como el padre y además usa su poder."""
        super().jugar()
        print(f"{self.nombre} usa {self.poder}")


if __name__ == "__main__":
    # ---- Objetos ----
    a = Personaje("Po", 80)
    b = Heroe("Dipsy", "rayo")
    print(a, "|", repr(a), "|", Personaje.especie)
    a.jugar()
    b.jugar()
    a.set_energia(500)                          # rechazado
    print(a.get_energia(), b.get_energia())
    try:
        a.__energia                             # privado desde fuera
    except AttributeError:
        print("AttributeError: el privado no se ve desde fuera")

    # ---- Lista de objetos ----
    grupo = [a, b, Personaje("Laa-Laa", 90)]
    grupo.append(Personaje("Tinky", 30))
    print(grupo)                                # usa __repr__
    for p in grupo:
        print(p.nombre, p.get_energia())
    encontrado = None
    for p in grupo:                             # buscar
        if p.nombre == "Laa-Laa":
            encontrado = p
            break
    print("Buscado:", encontrado)
    grupo.remove(encontrado)                    # eliminar
    print("Quedan:", len(grupo))
    print("Suma energía:", sum(p.get_energia() for p in grupo))
    print("Más energía:", max(grupo, key=lambda p: p.get_energia()))
    print("Ordenados:", sorted(grupo, key=lambda p: p.get_energia()))

    # ---- Diccionarios ----
    d = {"Po": "Rojo", "Dipsy": "Verde"}
    d["Tinky"] = "Morado"                       # añadir / modificar
    print(d["Po"], d.get("Noo", "no está"), "Po" in d)
    del d["Dipsy"]                              # borrar (o d.pop("Dipsy"))
    for clave, valor in d.items():
        print(clave, "->", valor)
    conteo = {}
    for letra in "papa":                        # contar apariciones
        conteo[letra] = conteo.get(letra, 0) + 1
    print(conteo)

    # ---- Comprensiones ----
    nums = [1, 2, 3, 4, 5]
    print([n * 2 for n in nums])                            # transformar
    print([n for n in nums if n % 2 == 0])                  # filtrar (if al final)
    print(["par" if n % 2 == 0 else "impar" for n in nums]) # if-else delante
    print({p.nombre: p.get_energia() for p in grupo})       # de diccionario

    # ---- Lambda ----
    print(list(map(lambda x: x ** 2, nums)))                # aplicar a todos
    print(list(filter(lambda x: x > 2, nums)))              # filtrar
    print(sorted(["sol", "mar", "elefante"], key=lambda s: len(s), reverse=True))


# ------------------------------------------------------------------
# SALIDA ESPERADA:
# Po (80) | Personaje('Po', 80) | Teletubby
# Dipsy usa rayo
# Valor no válido
# 70 40
# AttributeError: el privado no se ve desde fuera
# [Personaje('Po', 70), Personaje('Dipsy', 40), Personaje('Laa-Laa', 90), Personaje('Tinky', 30)]
# Po 70
# Dipsy 40
# Laa-Laa 90
# Tinky 30
# Buscado: Laa-Laa (90)
# Quedan: 3
# Suma energía: 140
# Más energía: Po (70)
# Ordenados: [Personaje('Tinky', 30), Personaje('Dipsy', 40), Personaje('Po', 70)]
# Rojo no está True
# Po -> Rojo
# Tinky -> Morado
# {'p': 2, 'a': 2}
# [2, 4, 6, 8, 10]
# [2, 4]
# ['impar', 'par', 'impar', 'par', 'impar']
# {'Po': 70, 'Dipsy': 40, 'Tinky': 30}
```

```
# [1, 4, 9, 16, 25]
# [3, 4, 5]
# ['elefante', 'sol', 'mar']
# ----------------------------------------------------------------
# RECORDATORIOS RÁPIDOS:
# - self primero en cada método; self.atributo para guardar en el objeto.
# - Atributo de clase: fuera de __init__; se cambia con Clase.atributo.
# - __privado: AttributeError al leer desde fuera; asignar crea otro atributo.
# - Getter = return. print(obj.ver_estado()).
# - __str__ -> print(obj); __repr__ -> repr(obj) y print(lista).
# - super().__init__(...) sin self.
# - dic[k] da KeyError si no existe; dic.get(k, defecto) no.
# - for k, v in dic.items():
# - max/min/sorted con key=lambda x: x.atributo  (devuelven el objeto).
# - map/filter -> list(...).  lista.sort() devuelve None.
# ----------------------------------------------------------------
```

Variantes tipo examen (todo mezclado)

05_variantes_tipo_examen/00_INDICE.md

05 - Variantes tipo examen (modelo "La Galaxia de los Teletubbies")

Seis ejercicios integradores ORIGINALES, de dificultad igual o mayor que la práctica de los Teletubbies.

Cada archivo trae el enunciado con requisitos numerados, la solución comentada, la **SALIDA ESPERADA** real,

la respuesta a la cuestión teórica, errores típicos y la alternativa que "funciona pero no es lo que pide".

Todos se ejecutan sin pedir datos: `python 0N_nombre.py` (en Linux: `python3`).

Los menús con `while` + `input()` (variantes 01 y 05) están en una función aparte; para probarlos,

descomenta la última línea del bloque `if __name__ == "__main__":`.

Ejercicios

Archivo	Qué pide	Forma exigida	Conceptos clave
`01_tamagotchi_mascotas.py`	Clase `MascotaVirtual` con niveles privados y un refugio (lista)	`*args` en `alimentar`, `1 <= minutos <= 60`, match-case con guardas, getter que devuelve string	Contador de clase, método privado `__limitar`, menú `while`
`02_biblioteca_prestamos.py`	Clases `Libro`, `Socio` y `Biblioteca` con préstamos	`**kwargs` en `alta_libro`, `for ... else`, cláusulas de guarda, `filter` / `sorted` + lambda	Dict de objetos, composición de clases, `dict.get`
`03_tienda_carrito.py`	Inventario `Producto` y `Carrito` con ticket, IVA y cupones	Atributo de clase `IVA`, match-case con guarda sobre texto (`"VIP" if total >= 100`), `*args`, `map` / `filter` / `sorted`	Dict con valor `[objeto, cantidad]`, tuplas de retorno, formato `:.2f`
`04_gestor_alumnos_notas.py`	Alumnos con notas en un dict privado, medias y calificaciones	match-case con guardas para la calificación, `**kwargs` en `crear_alumno`, type hints, comprensiones	Comprensión de diccionario, copia de dict, `while` con límite
`05_cuenta_bancaria.py`	Cuenta con saldo privado, historial y transferencias	Atributos de clase, método privado `__registrar`, `*args` y `**kwargs`, `0 < importe <= limite`	Lista de diccionarios, `max(key=lambda)`, interés compuesto con `while`, cajero `while True` + `break`
`06_videojuego_herencia.py`	Clase base `Personaje` y dos hijas, `Guerrero` y `Mago`, con combate por turnos	`class Hija(Padre)`, `super().__init__`, sobrescritura con `super().metodo()`, match-case sobre texto en la fábrica, `if/elif` + `isinstance()` en `describir`	Polimorfismo, name mangling en herencia, `isinstance`, `type(obj).__name__`

Qué concepto aparece en qué variante (para buscar rápido un ejemplo)

Busca el nombre de la función o método (Ctrl+F) dentro del archivo.

Concepto	01 Tamagotchi	02 Biblioteca	03 Tienda	04 Notas	05 Banco	06 Videojuego
Atributo de clase	`nombre_app`	`max_prestamos`	`IVA`	`nota_aprobado`	`comision_retirada`	`mundo`
Contador de clase (`Clase.x += 1`)	`total_mascotas`	`total_libros`	-	-	`num_cuentas`	`total_personajes`
Atributos privados `__`	`__hambre` …	`__disponible`, `__catalogo`	`__stock`, `__lineas`	`__notas` (dict)	`__saldo`, `__movimientos`	`__vida`, `__mana`
Método privado `__metodo`	`__limitar`	-	-	-	`__registrar`	-
Getter que devuelve string	`ver_estado`	`ver_prestamos`	`ver_stock`	`ver_boletin`	`consultar_saldo`	`ver_estado`
Getter que devuelve una copia	-	-	-	`get_notas`	`get_movimientos`	-
`__str__` y `__repr__`	sí	sí (2 clases)	sí	sí	sí	sí (con `super().__str__()`)
match-case con guardas	`estado_animo`	`epoca`	`ver_stock`, `descuento`	`calificacion`	`nivel_cliente`	(comentada en `describir`)
match-case con texto / menú	`menu_interactivo`	-	`descuento`	-	`cajero_interactivo`	`crear_personaje`
match-case con `\`	`` ` `` (o)	-	-	-	-	`nivel_cliente`
`isinstance()` para saber la clase	-	-	-	-	-	`describir`, main
Comparación encadenada	`jugar`	`epoca`	`ver_stock`, `agregar`	`poner_nota`	`ingresar`, `nivel_cliente`	-
Cláusulas de guarda (return temprano)	`alimentar`, `jugar`	`prestar`	`agregar`	`poner_nota`, `media`	`retirar`, `transferir`	`curar`
Ternario `x if c else y`	-	`__str__` de Libro	-	`ver_boletin`	`transferir`	`combate`
`while`	`menu_interactivo`	-	-	`dieces_necesarios`	`anios_para_objetivo`, `cajero_interactivo`	`combate`
`for ... else`	-	`buscar_libro`	-	-	-	-
`enumerate(start=1)`	main	main	`ticket`, main	main	main	main
Recorrer dict con `.items()`	main	main	`ticket`, main	`suspensas`, `crear_alumno`	main	main
Contar con `dict.get(k, 0) + 1`	-	`libros_por_epoca`	-	-	`resumen_por_tipo`	`contar_por_clase`

`setdefault(k, []).append(...)`	-	-	`inventario_por_categoria`	`medias_por_asignatura`	-	-
Comprensión de listas	`mascotas_que_necesitan_cuidado`	`disponibles`	`subtotal`	`suspensas`, `aprobados`	`ingresar_varios`	`vivos`
Comprensión de diccionario	(comentada en `resumen_refugio`)	-	-	`medias_por_asignatura`	-	-
`sorted` + lambda	ranking en main	`ordenados_por_anio`	`mas_caros`	`ranking`	ranking en main	`clasificacion`
`filter` + lambda	-	`libros_de`	`bajo_stock`	-	`ingresos_grandes`	-
`map` + lambda	-	-	`precios_con_iva`	`nombres_en_mayusculas`	`importes`	-
`max(key=lambda)`	-	-	-	-	`cuenta_mas_rica`	-
`*args`	`alimentar`	-	`agregar_varios`	-	`ingresar_varios`	-
`**kwargs`	-	`alta_libro` (`Libro(**datos)`)	-	`crear_alumno`	`transferir`	`crear_personaje`
Type hints	-	-	-	todas las funciones	-	-
Lista de objetos	`refugio`	`__catalogo`	`inventario`	`grupo`	`cuentas`	`personajes`
Lista de diccionarios	-	-	-	-	`__movimientos`	-
Herencia + `super()`	-	-	-	-	-	`Guerrero`, `Mago`
Polimorfismo	-	-	-	-	-	`atacar`
Varias clases que colaboran	-	`Biblioteca` / `Socio` / `Libro`	`Carrito` / `Producto`	-	`transferir` entre cuentas	`atacar` entre personajes
Demostrar `AttributeError` con privado	final del main	-	-	-	-	(explicado en la cuestión teórica)
Menú `while` + `input` (opcional)	`menu_interactivo`	-	-	-	`cajero_interactivo`	-

Cuestiones teóricas resueltas (al final de cada archivo)

- **01**: para qué sirve `*args` y por qué un getter hace `return` y no `print`.
- **02**: diferencia entre `for-else` y un `else` dentro del `for`; por qué el catálogo es privado.
- **03**: atributo de clase frente a atributo de instancia (qué pasa si cambia el IVA).
- **04**: por qué devolver una copia del diccionario privado.
- **05**: por qué `__saldo` es privado y no tiene `set_saldo()`.
- **06**: name mangling en herencia (la hija no ve `self.__vida`) y qué hace `super()`.

05_variantes_tipo_examen/01_tamagotchi_mascotas.py

```
"""
TEMA 6 (integrador) - Variante 01: "El refugio de mascotas virtuales" (tipo Tamagotchi)
================================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Diseñar una clase que represente una mascota virtual y gestionar
    un refugio (lista de objetos) aplicando POO, condicionales, bucles,
    funciones, diccionarios, comprensiones de listas y lambda.

    Requisitos de la clase MascotaVirtual:
    1. Atributo de clase: nombre_app = "TamaPy" y un contador total_mascotas
       que se incremente cada vez que se crea una mascota.
    2. Atributos de instancia:
       - Públicos: nombre (str), especie (str).
       - Privados (__): __hambre (int, por defecto 50), __energia (int, por
         defecto 80), __felicidad (int, por defecto 60).
       Los niveles siempre deben quedar entre 0 y 100.
    3. alimentar(*comidas): recibe un número variable de comidas (*args).
       Cada comida reduce __hambre en 15. Devuelve (return) un mensaje.
    4. jugar(minutos): solo acepta minutos entre 1 y 60 usando una
       COMPARACIÓN ENCADENADA. Si no es válido, devuelve un mensaje de error
       (return temprano). Si es válido: felicidad += minutos // 2,
       energia -= minutos, hambre += minutos // 3.
    5. dormir(): energía a 100 y hambre += 10. Devuelve un mensaje.
    6. ver_estado(): GETTER que DEVUELVE un string con los tres niveles.
    7. estado_animo(): usa MATCH-CASE con GUARDAS para devolver:
       "Hambrienta" si hambre >= 80, "Agotada" si energía <= 20,
       "Feliz" si felicidad >= 70, "Normal" en cualquier otro caso (case _).
    8. get_felicidad(): getter que devuelve el entero (se usará con lambda).
    9. __str__ (amigable) y __repr__ (técnico, como se instanció).

    Funciones fuera de la clase:
    10. resumen_refugio(mascotas): devuelve un DICCIONARIO {nombre: ánimo}.
    11. mascotas_que_necesitan_cuidado(mascotas): devuelve con una
        COMPRENSIÓN DE LISTAS los nombres de las que NO están "Feliz" ni "Normal".

    Bloque principal (if __name__ == "__main__"):
    12. Crea al menos 3 mascotas en una lista y recórrela con enumerate(start=1).
    13. Haz que interactúen (alimentar con varias comidas, jugar, dormir,
        y un jugar con minutos inválidos).
    14. Recorre el diccionario del resumen con .items().
    15. Ordena las mascotas por felicidad (de mayor a menor) con sorted + lambda.
    16. Muestra total_mascotas y el atributo de clase.
    17. (Opcional) Menú interactivo con while + input en una función aparte.

    Cuestión teórica: ¿Por qué alimentar() usa *args? ¿Qué diferencia hay
    entre que ver_estado() devuelva el texto o lo imprima?

FORMA QUE EXIGE EL PROFESOR:
    - Atributo de clase + contador de clase (MascotaVirtual.total_mascotas += 1).
    - Atributos privados con doble guion bajo (__).
```

```
    - Getter que DEVUELVE string (return, no print).
    - *args en alimentar.
    - Comparación encadenada 1 <= minutos <= 60.
    - match-case con guardas (case _ if ...) y case _ final.
    - Diccionario recorrido con .items(), enumerate(start=1),
      comprensión de listas, sorted con key=lambda.

CONCEPTOS QUE PRACTICA:
    - Clases, atributos de clase/instancia, privados, __str__/__repr__
    - *args, return temprano, comparación encadenada, match-case
    - Lista de objetos, diccionario, comprensión, lambda, while (menú)

CÓMO EJECUTARLO:  python 01_tamagotchi_mascotas.py   (en Linux/Docker: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: La clase. Atributos de CLASE arriba (compartidos por todas
# las mascotas) y atributos de INSTANCIA en __init__ (uno por mascota).
# ------------------------------------------------------------------
class MascotaVirtual:
    """Representa una mascota virtual con niveles de hambre, energía y felicidad."""

    # Atributos de clase: se escriben FUERA de __init__ y SIN self.
    nombre_app = "TamaPy"
    total_mascotas = 0  # contador compartido por toda la clase

    def __init__(self, nombre, especie, hambre=50, energia=80, felicidad=60):
        """Constructor: guarda los datos públicos y los niveles privados."""
        # Públicos: se pueden leer/cambiar desde fuera (mascota.nombre).
        self.nombre = nombre
        self.especie = especie
        # Privados: el doble guion bajo activa el "name mangling"
        # (Python lo renombra a _MascotaVirtual__hambre).
        # Pasan por __limitar para que también al crearla queden entre 0 y 100.
        self.__hambre = self.__limitar(hambre)
        self.__energia = self.__limitar(energia)
        self.__felicidad = self.__limitar(felicidad)
        # Para tocar el contador de CLASE se usa el nombre de la clase.
        # OJO: self.total_mascotas += 1 crearía un atributo de instancia nuevo.
        MascotaVirtual.total_mascotas += 1

    # Método auxiliar privado: mantiene un valor entre 0 y 100.
    # Se llama __limitar para que no se use desde fuera de la clase.
    def __limitar(self, valor):
        """Devuelve el valor recortado al rango 0-100."""
        return max(0, min(100, valor))

    def alimentar(self, *comidas):
        """Da de comer. *comidas recoge todas las comidas en una TUPLA."""
        # Cláusula de guarda: si no pasan ninguna comida, salimos ya.
        if not comidas:
            return f"{self.nombre} no ha recibido nada de comer."
        # len(comidas) = cuántos argumentos llegaron.
        self.__hambre = self.__limitar(self.__hambre - 15 * len(comidas))
        # ", ".join(...) une la tupla de strings con comas.
        return f"{self.nombre} come: {', '.join(comidas)}. Hambre: {self.__hambre}"

    def jugar(self, minutos):
        """Juega unos minutos (solo entre 1 y 60)."""
        # Comparación ENCADENADA: equivale a 1 <= minutos and minutos <= 60.
        # Usamos "not (...)" para hacer el return temprano del caso inválido.
        if not (1 <= minutos <= 60):
            return f"Error: {minutos} minutos no es válido (1-60)."
        self.__felicidad = self.__limitar(self.__felicidad + minutos // 2)
        self.__energia = self.__limitar(self.__energia - minutos)
        self.__hambre = self.__limitar(self.__hambre + minutos // 3)
        return f"{self.nombre} ha jugado {minutos} min."

    def dormir(self):
        """Recupera toda la energía, pero se despierta con algo de hambre."""
        self.__energia = 100
        self.__hambre = self.__limitar(self.__hambre + 10)
        return f"{self.nombre} ha dormido. Energía al máximo."

    def ver_estado(self):
        """GETTER: devuelve (no imprime) un texto con el estado interno."""
        return (f"Estado de {self.nombre} -> Hambre: {self.__hambre} | "
                f"Energía: {self.__energia} | Felicidad: {self.__felicidad}")

    def get_felicidad(self):
        """Getter numérico: necesario para ordenar desde fuera con lambda."""
        return self.__felicidad

    def estado_animo(self):
        """Devuelve el ánimo usando match-case con guardas."""
        # Truco típico: hacemos match sobre algo cualquiera (aquí self) y
        # decidimos con las GUARDAS "if". El orden de los case importa:
        # se ejecuta el PRIMERO que cumpla.
        match self:
            case _ if self.__hambre >= 80:
                return "Hambrienta"
            case _ if self.__energia <= 20:
                return "Agotada"
            case _ if self.__felicidad >= 70:
                return "Feliz"
            case _:  # comodín: equivale al "else"/"default"
                return "Normal"

    def __str__(self):
        """Texto amigable: lo usa print(mascota) y str(mascota)."""
        return f"{self.especie} {self.nombre} - Ánimo: {self.estado_animo()}"
```

```python
    def __repr__(self):
        """Texto técnico: cómo se instanció. Lo usa print(lista_de_mascotas)."""
        return (f"MascotaVirtual('{self.nombre}', '{self.especie}', "
                f"{self.__hambre}, {self.__energia}, {self.__felicidad})")


# ------------------------------------------------------------------
# PASO 2: Funciones que trabajan con la LISTA de objetos.
# Devuelven datos (return): quien las llama decide si los imprime.
# ------------------------------------------------------------------
def resumen_refugio(mascotas):
    """Devuelve un diccionario {nombre: ánimo} de todas las mascotas."""
    resumen = {}
    for mascota in mascotas:
        resumen[mascota.nombre] = mascota.estado_animo()
    return resumen
    # Versión en una línea (comprensión de diccionario):
    # return {m.nombre: m.estado_animo() for m in mascotas}


def mascotas_que_necesitan_cuidado(mascotas):
    """Devuelve (comprensión de listas) los nombres que necesitan atención."""
    # Estructura: [EXPRESIÓN for ELEMENTO in LISTA if CONDICIÓN]
    return [m.nombre for m in mascotas
            if m.estado_animo() not in ("Feliz", "Normal")]


# ------------------------------------------------------------------
# PASO 3 (opcional): menú interactivo con while + input.
# Está en una función aparte para que el main se ejecute sin pedir datos.
# Para probarlo, descomenta la llamada al final del main.
# ------------------------------------------------------------------
def menu_interactivo(mascota):
    """Menú con while: se repite hasta que el usuario elige salir."""
    opcion = ""
    while opcion != "0":
        print("\n1) Alimentar  2) Jugar  3) Dormir  4) Ver estado  0) Salir")
        opcion = input("Elige opción: ")
        match opcion:
            case "1":
                print(mascota.alimentar("pienso"))
            case "2":
                minutos = int(input("¿Cuántos minutos? "))
                print(mascota.jugar(minutos))
            case "3":
                print(mascota.dormir())
            case "4":
                print(mascota.ver_estado())
            case "0":
                print("¡Hasta pronto!")
            case _:
                print("Opción no válida.")


# ------------------------------------------------------------------
# PASO 4: Bloque principal con datos de prueba fijos.
# ------------------------------------------------------------------
if __name__ == "__main__":
    print(f"=== Bienvenida a {MascotaVirtual.nombre_app} ===")

    # Lista de objetos: cada elemento es una instancia.
    refugio = [
        MascotaVirtual("Pelusa", "Gato"),
        MascotaVirtual("Rex", "Dinosaurio", hambre=70, energia=30),
        MascotaVirtual("Burbuja", "Pez", felicidad=90),
    ]

    # enumerate(..., start=1) da el número de orden empezando en 1.
    print("\n--- Mascotas del refugio ---")
    for numero, mascota in enumerate(refugio, start=1):
        print(f"{numero}. {mascota}")  # print llama a __str__

    print("\n--- Interacciones ---")
    pelusa, rex, burbuja = refugio  # desempaquetado de la lista
    print(pelusa.alimentar("pienso", "atún"))     # *args: 2 comidas
    print(pelusa.jugar(40))
    print(rex.jugar(25))
    print(rex.jugar(90))                          # inválido -> mensaje de error
    print(burbuja.alimentar())                    # *args vacío
    print(burbuja.dormir())

    print("\n--- Estados (getter que devuelve string) ---")
    for mascota in refugio:
        print(mascota.ver_estado())

    print("\n--- Resumen del refugio (dict con .items()) ---")
    for nombre, animo in resumen_refugio(refugio).items():
        print(f"{nombre}: {animo}")

    print("\nNecesitan cuidado:", mascotas_que_necesitan_cuidado(refugio))

    # sorted NO modifica la lista original; key=lambda dice POR QUÉ ordenar.
    # reverse=True -> de mayor a menor.
    ranking = sorted(refugio, key=lambda m: m.get_felicidad(), reverse=True)
    print("\n--- Ranking de felicidad ---")
    for posicion, mascota in enumerate(ranking, start=1):
        print(f"{posicion}º {mascota.nombre} ({mascota.get_felicidad()})")

    print("\nRepr de la lista:", refugio)  # print de una lista usa __repr__
    print(f"Total de mascotas creadas: {MascotaVirtual.total_mascotas}")
```

```
    # Cuestión típica: acceder al privado desde fuera da AttributeError.
    try:
        print(pelusa.__hambre)
    except AttributeError as error:
        print("AttributeError:", error)

    # menu_interactivo(pelusa)   # <- descomenta para probar el menú con input()

# ------------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# === Bienvenida a TamaPy ===
#
# --- Mascotas del refugio ---
# 1. Gato Pelusa - Ánimo: Normal
# 2. Dinosaurio Rex - Ánimo: Normal
# 3. Pez Burbuja - Ánimo: Feliz
#
# --- Interacciones ---
# Pelusa come: pienso, atún. Hambre: 20
# Pelusa ha jugado 40 min.
# Rex ha jugado 25 min.
# Error: 90 minutos no es válido (1-60).
# Burbuja no ha recibido nada de comer.
# Burbuja ha dormido. Energía al máximo.
#
# --- Estados (getter que devuelve string) ---
# Estado de Pelusa -> Hambre: 33 | Energía: 40 | Felicidad: 80
# Estado de Rex -> Hambre: 78 | Energía: 5 | Felicidad: 72
# Estado de Burbuja -> Hambre: 60 | Energía: 100 | Felicidad: 90
#
# --- Resumen del refugio (dict con .items()) ---
# Pelusa: Feliz
# Rex: Agotada
# Burbuja: Feliz
#
# Necesitan cuidado: ['Rex']
#
# --- Ranking de felicidad ---
# 1º Burbuja (90)
# 2º Pelusa (80)
# 3º Rex (72)
#
# Repr de la lista: [MascotaVirtual('Pelusa', 'Gato', 33, 40, 80), MascotaVirtual('Rex', 'Dinosaurio', 78, 5, 72), MascotaVirtual('Burbuja', 'Pez', 60, 100, 90)]
# Total de mascotas creadas: 3
# AttributeError: 'MascotaVirtual' object has no attribute '__hambre'
# ------------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - *args permite llamar a alimentar() con 0, 1 o muchas comidas; dentro
#   llegan como una tupla (comidas). Sin *args habría que pasar una lista.
# - Si ver_estado() hace return, el texto se puede guardar, comparar o
#   imprimir donde quieras. Si hiciera print, devolvería None y
#   print(m.ver_estado()) mostraría el texto y luego "None".
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar self en los métodos: def jugar(minutos) -> TypeError al llamarlo.
# - Escribir self.total_mascotas += 1: NO cambia el contador de la clase.
# - En match-case, "case _:" (sin if) tiene que ir el ÚLTIMO: si lo pones
#   antes que otros, Python da SyntaxError ("wildcard makes remaining
#   patterns unreachable"). Con guarda ("case _ if ...") sí puede ir antes.
# - Desde fuera NO existe mascota.__hambre (name mangling); existe
#   mascota._MascotaVirtual__hambre, pero usarlo rompe el encapsulamiento.
# EN JAVA SERÍA... private int hambre; y static int totalMascotas;
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if 1 <= minutos and minutos <= 60  -> pide comparación ENCADENADA.
#   un if/elif/else en estado_animo    -> pide match-case con guardas.
# ------------------------------------------------------------------
```

05_variantes_tipo_examen/02_biblioteca_prestamos.py

```
"""
TEMA 6 (integrador) - Variante 02: "La biblioteca del instituto"
================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Gestionar los libros y los préstamos de una biblioteca usando
    dos clases que colaboran, una lista de objetos y un diccionario de socios.

    Requisitos de la clase Libro:
    1. Atributo de clase: total_libros (contador de libros creados).
    2. Atributos públicos: isbn, titulo, autor, anio (int).
       Atributo privado: __disponible (bool, True al crearse).
    3. esta_disponible(): getter que devuelve el booleano.
       marcar_prestado() y marcar_devuelto(): cambian __disponible.
    4. epoca(): MATCH-CASE con GUARDAS sobre el año:
       antes de 1900 -> "Clásico"; de 1900 a 1999 (COMPARACIÓN ENCADENADA)
       -> "Siglo XX"; 2000 en adelante -> "Contemporáneo".
    5. __str__ (amigable con estado) y __repr__ (técnico).

    Requisitos de la clase Socio:
    6. Atributo de clase: max_prestamos = 2.
    7. Atributos: nombre (público), __libros (lista privada de ISBN).
    8. puede_pedir(): devuelve True si aún no ha llegado al máximo.
       ver_prestamos(): GETTER que DEVUELVE un string con sus préstamos.

    Requisitos de la clase Biblioteca:
    9. Atributos: __catalogo (lista de Libro) y __socios (dict nombre -> Socio).
    10. alta_libro(**datos): crea el Libro con **kwargs y lo añade.
    11. buscar_libro(isbn): recorre con for ... ELSE (el else se ejecuta si
        el bucle no encontró nada y no hubo break). Devuelve el libro o None.
    12. prestar(isbn, nombre_socio): CLÁUSULAS DE GUARDA (return temprano)
```

```
        para: socio inexistente, libro inexistente, libro no disponible,
        socio con el máximo. Devuelve SIEMPRE un string (no print).
    13. devolver(isbn, nombre_socio): devuelve un string.
    14. disponibles(): COMPRENSIÓN DE LISTAS con los títulos disponibles.
    15. libros_de(autor): FILTER + LAMBDA.
    16. ordenados_por_anio(): SORTED + LAMBDA.
    17. libros_por_epoca(): DICCIONARIO {época: número de libros}.

    Bloque principal: alta de 5 libros con **kwargs, 2 socios, préstamos
    correctos y erróneos (todos los casos de guarda), devolución, listados
    con enumerate(start=1) y recorrido del diccionario con .items().

    Cuestión teórica: ¿Qué diferencia hay entre for-else y poner el else
    dentro del for? ¿Por qué __catalogo es privado?

FORMA QUE EXIGE EL PROFESOR:
    - **kwargs en alta_libro, for-else en buscar_libro.
    - Cláusulas de guarda con return temprano en prestar.
    - match-case con guardas + comparación encadenada en epoca.
    - Comprensión de listas, filter+lambda, sorted+lambda, dict con .items().

CONCEPTOS QUE PRACTICA:
    - Varias clases que colaboran (composición), atributos privados
    - Diccionario de objetos, lista de objetos, for-else, **kwargs

CÓMO EJECUTARLO:  python 02_biblioteca_prestamos.py   (en Linux: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: Clase Libro.
# ------------------------------------------------------------------
class Libro:
    """Un libro del catálogo."""

    total_libros = 0  # atributo de clase (contador)

    def __init__(self, isbn, titulo, autor, anio):
        """Guarda los datos del libro. Al crearse está disponible."""
        self.isbn = isbn
        self.titulo = titulo
        self.autor = autor
        self.anio = anio
        self.__disponible = True  # privado: solo lo cambian los métodos
        Libro.total_libros += 1

    def esta_disponible(self):
        """Getter: devuelve True si se puede prestar."""
        return self.__disponible

    def marcar_prestado(self):
        """Pone el libro como prestado."""
        self.__disponible = False

    def marcar_devuelto(self):
        """Pone el libro como disponible otra vez."""
        self.__disponible = True

    def epoca(self):
        """Clasifica el libro por su año con match-case y guardas."""
        # Aquí hacemos match sobre el propio año: "case a if ..." captura el
        # valor en la variable a y luego comprueba la guarda.
        match self.anio:
            case a if a < 1900:
                return "Clásico"
            case a if 1900 <= a <= 1999:   # comparación encadenada
                return "Siglo XX"
            case _:
                return "Contemporáneo"

    def __str__(self):
        """Texto amigable para el usuario."""
        # Expresión condicional (ternario de Python): VALOR_SI if COND else VALOR_NO
        estado = "disponible" if self.__disponible else "PRESTADO"
        return f"'{self.titulo}' de {self.autor} ({self.anio}) - {estado}"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"Libro('{self.isbn}', '{self.titulo}', '{self.autor}', {self.anio})"


# ------------------------------------------------------------------
# PASO 2: Clase Socio.
# ------------------------------------------------------------------
class Socio:
    """Un socio que puede tener libros prestados."""

    max_prestamos = 2  # atributo de clase: el límite es igual para todos

    def __init__(self, nombre):
        """Crea el socio sin préstamos."""
        self.nombre = nombre
        # Lista privada creada DENTRO de __init__: cada socio tiene la suya.
        # (Nunca pongas def __init__(self, nombre, libros=[]): se comparte.)
        self.__libros = []

    def puede_pedir(self):
        """Devuelve True si no ha llegado al máximo de préstamos."""
        return len(self.__libros) < Socio.max_prestamos

    def anotar(self, isbn):
        """Apunta un préstamo."""
```

```
        self.__libros.append(isbn)

    def tiene(self, isbn):
        """Devuelve True si el socio tiene ese libro."""
        return isbn in self.__libros

    def quitar(self, isbn):
        """Borra un préstamo."""
        self.__libros.remove(isbn)

    def ver_prestamos(self):
        """GETTER que devuelve un string con los préstamos del socio."""
        if not self.__libros:
            return f"{self.nombre} no tiene préstamos."
        return f"{self.nombre} tiene: {', '.join(self.__libros)}"

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return f"Socio {self.nombre} ({len(self.__libros)}/{Socio.max_prestamos})"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"Socio('{self.nombre}')"


# ---------------------------------------------------------------
# PASO 3: Clase Biblioteca: contiene la lista de libros y el dict de socios.
# ---------------------------------------------------------------
class Biblioteca:
    """Gestiona el catálogo y los préstamos."""

    def __init__(self, nombre):
        """Constructor: guarda los datos iniciales del objeto."""
        self.nombre = nombre
        self.__catalogo = []   # lista de objetos Libro
        self.__socios = {}     # diccionario: clave nombre -> valor objeto Socio

    def alta_libro(self, **datos):
        """Crea un libro a partir de argumentos con nombre (**kwargs)."""
        # datos es un diccionario: {'isbn': ..., 'titulo': ..., ...}
        # Libro(**datos) "desempaqueta" el dict en argumentos con nombre.
        libro = Libro(**datos)
        self.__catalogo.append(libro)
        return libro

    def alta_socio(self, nombre):
        """Añade un socio al diccionario."""
        self.__socios[nombre] = Socio(nombre)

    def obtener_socio(self, nombre):
        """Devuelve el socio o None (dict.get no da error si no existe)."""
        return self.__socios.get(nombre)

    def buscar_libro(self, isbn):
        """Busca un libro por ISBN con for-else."""
        for libro in self.__catalogo:
            if libro.isbn == isbn:
                encontrado = libro
                break             # al hacer break, el else NO se ejecuta
        else:
            encontrado = None     # solo llega aquí si el for terminó sin break
        return encontrado

    def prestar(self, isbn, nombre_socio):
        """Presta un libro. Usa cláusulas de guarda (return temprano)."""
        socio = self.obtener_socio(nombre_socio)
        # Cada "if ...: return" corta la función en cuanto algo falla.
        # Así evitamos if anidados y el caso bueno queda al final, limpio.
        if socio is None:
            return f"Error: el socio '{nombre_socio}' no existe."
        libro = self.buscar_libro(isbn)
        if libro is None:
            return f"Error: no hay ningún libro con ISBN {isbn}."
        if not libro.esta_disponible():
            return f"Error: '{libro.titulo}' ya está prestado."
        if not socio.puede_pedir():
            return f"Error: {socio.nombre} ya tiene {Socio.max_prestamos} libros."
        # Caso correcto:
        libro.marcar_prestado()
        socio.anotar(isbn)
        return f"Préstamo OK: '{libro.titulo}' para {socio.nombre}."

    def devolver(self, isbn, nombre_socio):
        """Devuelve un libro prestado."""
        socio = self.obtener_socio(nombre_socio)
        libro = self.buscar_libro(isbn)
        if socio is None or libro is None or not socio.tiene(isbn):
            return "Error: devolución no válida."
        socio.quitar(isbn)
        libro.marcar_devuelto()
        return f"Devolución OK: '{libro.titulo}'."

    def disponibles(self):
        """Comprensión de listas: títulos de los libros disponibles."""
        return [libro.titulo for libro in self.__catalogo if libro.esta_disponible()]

    def libros_de(self, autor):
        """filter + lambda: libros de un autor. filter devuelve un iterador,
        por eso lo convertimos con list()."""
        return list(filter(lambda libro: libro.autor == autor, self.__catalogo))

    def ordenados_por_anio(self):
```

```
        """sorted + lambda: copia del catálogo ordenada por año."""
        return sorted(self.__catalogo, key=lambda libro: libro.anio)

    def libros_por_epoca(self):
        """Diccionario {época: cantidad} contando con .get(clave, 0)."""
        conteo = {}
        for libro in self.__catalogo:
            epoca = libro.epoca()
            # get(epoca, 0): si la clave aún no existe empieza en 0.
            conteo[epoca] = conteo.get(epoca, 0) + 1
        return conteo


# ------------------------------------------------------------------
# PASO 4: Bloque principal con datos fijos (sin input).
# ------------------------------------------------------------------
if __name__ == "__main__":
    bib = Biblioteca("IES Las Galletas")

    # **kwargs: los argumentos van con nombre, en cualquier orden.
    bib.alta_libro(isbn="111", titulo="Don Quijote", autor="Cervantes", anio=1605)
    bib.alta_libro(isbn="222", titulo="Cien años de soledad",
                   autor="García Márquez", anio=1967)
    bib.alta_libro(isbn="333", titulo="Novelas ejemplares", autor="Cervantes", anio=1613)
    bib.alta_libro(titulo="Python para todos", autor="Severance",
                   anio=2016, isbn="444")
    bib.alta_libro(isbn="555", titulo="El túnel", autor="Sabato", anio=1948)

    bib.alta_socio("Ana")
    bib.alta_socio("Luis")

    print("--- Préstamos ---")
    print(bib.prestar("111", "Ana"))
    print(bib.prestar("444", "Ana"))
    print(bib.prestar("222", "Ana"))     # guarda: Ana ya tiene 2
    print(bib.prestar("111", "Luis"))    # guarda: ya prestado
    print(bib.prestar("999", "Luis"))    # guarda: ISBN inexistente
    print(bib.prestar("222", "Marta"))   # guarda: socio inexistente
    print(bib.prestar("222", "Luis"))

    print("\n--- Estado de los socios ---")
    for nombre in ("Ana", "Luis"):
        print(bib.obtener_socio(nombre).ver_prestamos())

    print("\n--- Devolución ---")
    print(bib.devolver("111", "Ana"))
    print(bib.devolver("111", "Ana"))    # ya no lo tiene
    print(bib.obtener_socio("Ana"))      # __str__ del socio

    print("\nDisponibles:", bib.disponibles())
    print("De Cervantes:", bib.libros_de("Cervantes"))   # lista -> __repr__

    print("\n--- Catálogo por año ---")
    for numero, libro in enumerate(bib.ordenados_por_anio(), start=1):
        print(f"{numero}. {libro} [{libro.epoca()}]")

    print("\n--- Libros por época ---")
    for epoca, cantidad in bib.libros_por_epoca().items():
        print(f"{epoca}: {cantidad}")

    print(f"\nTotal de libros creados: {Libro.total_libros}")

# ------------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# --- Préstamos ---
# Préstamo OK: 'Don Quijote' para Ana.
# Préstamo OK: 'Python para todos' para Ana.
# Error: Ana ya tiene 2 libros.
# Error: 'Don Quijote' ya está prestado.
# Error: no hay ningún libro con ISBN 999.
# Error: el socio 'Marta' no existe.
# Préstamo OK: 'Cien años de soledad' para Luis.
#
# --- Estado de los socios ---
# Ana tiene: 111, 444
# Luis tiene: 222
#
# --- Devolución ---
# Devolución OK: 'Don Quijote'.
# Error: devolución no válida.
# Socio Ana (1/2)
#
# Disponibles: ['Don Quijote', 'Novelas ejemplares', 'El túnel']
# De Cervantes: [Libro('111', 'Don Quijote', 'Cervantes', 1605), Libro('333', 'Novelas ejemplares', 'Cervantes', 1613)]
#
# --- Catálogo por año ---
# 1. 'Don Quijote' de Cervantes (1605) - disponible [Clásico]
# 2. 'Novelas ejemplares' de Cervantes (1613) - disponible [Clásico]
# 3. 'El túnel' de Sabato (1948) - disponible [Siglo XX]
# 4. 'Cien años de soledad' de García Márquez (1967) - PRESTADO [Siglo XX]
# 5. 'Python para todos' de Severance (2016) - PRESTADO [Contemporáneo]
#
# --- Libros por época ---
# Clásico: 2
# Siglo XX: 2
# Contemporáneo: 1
#
# Total de libros creados: 5
# ------------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - El else del for-else va al nivel del for (no del if) y se ejecuta UNA
#   vez, solo si el bucle acabó sin break: perfecto para "no encontrado".
```

```
#   Un else dentro del for (pegado al if) se ejecutaría en cada vuelta.
# - __catalogo es privado para que nadie haga bib.__catalogo.append(...)
#   saltándose alta_libro(); toda modificación pasa por los métodos.
# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - filter() y map() devuelven iteradores: usa list(...) para verlos.
# - libro(**datos) solo funciona si las claves coinciden con los
#   parámetros de __init__ (isbn, titulo, autor, anio).
# - Usar socios["Marta"] con una clave inexistente da KeyError;
#   socios.get("Marta") devuelve None.
# EN JAVA SERÍA... un HashMap<String, Socio> y un ArrayList<Libro>.
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   prestar con if/else anidados en vez de cláusulas de guarda, o
#   buscar con una variable bandera "encontrado = False" en vez de for-else.
# ------------------------------------------------------------------
```

05_variantes_tipo_examen/03_tienda_carrito.py

```
"""
TEMA 6 (integrador) - Variante 03: "La tienda online y su carrito"
==================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Programar el inventario de una tienda y un carrito de la compra
    que calcule el ticket con IVA y descuentos.

    Requisitos de la clase Producto:
    1. Atributo de clase: IVA = 0.21 (compartido por todos los productos).
    2. Atributos públicos: nombre, precio (float, sin IVA), categoria.
       Atributo privado: __stock (int).
    3. get_stock(): getter que devuelve el entero.
       ver_stock(): GETTER que devuelve un STRING del tipo
       "Teclado: 3 uds (stock bajo)" usando MATCH-CASE con guardas:
       0 -> "agotado"; 1 a 5 -> "stock bajo" (COMPARACIÓN ENCADENADA);
       resto -> "disponible".
    4. retirar(cantidad): resta del stock solo si hay suficiente;
       devuelve True o False (return, no print).
    5. precio_con_iva(): devuelve el precio con IVA redondeado a 2 decimales.
    6. __str__ y __repr__.

    Requisitos de la clase Carrito:
    7. Atributo privado __lineas: DICCIONARIO {nombre_producto: [producto, cantidad]}.
    8. agregar(producto, cantidad=1): cantidad debe ser 1 <= cantidad <= 10
       (comparación encadenada). Si no hay stock, no se añade. Devuelve string.
    9. agregar_varios(*pares): recibe un número variable de tuplas
       (producto, cantidad) con *args y llama a agregar para cada una.
    10. subtotal(): suma con una COMPRENSIÓN (sum([...])) recorriendo el
        diccionario con .items() o .values().
    11. descuento(codigo): MATCH-CASE con guardas sobre el código:
        "VERANO" -> 10 %; "VIP" solo si subtotal >= 100 -> 20 %;
        "VIP" con subtotal menor -> 0 % (mensaje); cualquier otro -> 0 %.
        Devuelve una tupla (porcentaje, mensaje).
    12. ticket(codigo=None): devuelve un STRING multilínea con las líneas
        numeradas con enumerate(start=1), subtotal, descuento, total.

    Funciones fuera de las clases:
    13. precios_con_iva(productos): MAP + LAMBDA -> lista de (nombre, precio_iva).
    14. bajo_stock(productos): FILTER + LAMBDA -> productos con stock <= 5.
    15. mas_caros(productos, n): SORTED + LAMBDA (de mayor a menor), los n primeros.
    16. inventario_por_categoria(productos): DICCIONARIO {categoria: [nombres]}.

    Bloque principal: crea 5 productos, prueba todas las funciones y dos
    tickets (uno con "VIP" válido y otro con un código que no aplica).

    Cuestión teórica: ¿Por qué IVA es atributo de clase y no de instancia?
    ¿Qué pasaría si mañana el IVA sube al 0.23?

FORMA QUE EXIGE EL PROFESOR:
    - Atributo de clase IVA usado como Producto.IVA.
    - Atributo privado __stock con getter.
    - match-case con guardas (2 veces) y comparación encadenada.
    - *args en agregar_varios, dict recorrido con .items(),
      comprensión en subtotal, map/filter/sorted con lambda,
      enumerate(start=1), return en vez de print.

CONCEPTOS QUE PRACTICA:
    - Diccionario cuyo valor es una lista [objeto, cantidad]
    - Tuplas como valor de retorno múltiple, f-strings con :.2f

CÓMO EJECUTARLO:  python 03_tienda_carrito.py   (en Linux: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: Clase Producto.
# ------------------------------------------------------------------
class Producto:
    """Un producto del inventario."""

    IVA = 0.21  # atributo de clase: igual para todos los productos

    def __init__(self, nombre, precio, categoria, stock=0):
        """Constructor. stock es privado."""
        self.nombre = nombre
        self.precio = precio
        self.categoria = categoria
        self.__stock = stock

    def get_stock(self):
        """Getter numérico del stock."""
```

```
        return self.__stock

    def ver_stock(self):
        """GETTER que devuelve un string describiendo el stock."""
        match self.__stock:
            case 0:                          # patrón literal
                estado = "agotado"
            case s if 1 <= s <= 5:           # guarda + comparación encadenada
                estado = "stock bajo"
            case _:
                estado = "disponible"
        return f"{self.nombre}: {self.__stock} uds ({estado})"

    def retirar(self, cantidad):
        """Resta stock si hay suficiente. Devuelve True/False."""
        if cantidad > self.__stock:
            return False
        self.__stock -= cantidad
        return True

    def precio_con_iva(self):
        """Precio final con IVA. Se usa Producto.IVA (atributo de clase)."""
        return round(self.precio * (1 + Producto.IVA), 2)

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        # :.2f -> 2 decimales en el f-string
        return f"{self.nombre} ({self.categoria}) - {self.precio_con_iva():.2f} € IVA incl."

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return (f"Producto('{self.nombre}', {self.precio}, "
                f"'{self.categoria}', {self.__stock})")


# ------------------------------------------------------------------
# PASO 2: Clase Carrito.
# ------------------------------------------------------------------
class Carrito:
    """Carrito de la compra: guarda cada producto con su cantidad."""

    def __init__(self, cliente):
        """Constructor: guarda los datos iniciales del objeto."""
        self.cliente = cliente
        # Diccionario: clave = nombre del producto, valor = [producto, cantidad]
        self.__lineas = {}

    def agregar(self, producto, cantidad=1):
        """Añade un producto si la cantidad es válida y hay stock."""
        if not (1 <= cantidad <= 10):                # comparación encadenada
            return f"Error: cantidad {cantidad} no permitida (1-10)."
        if not producto.retirar(cantidad):
            return f"Error: no hay stock suficiente de {producto.nombre}."
        if producto.nombre in self.__lineas:
            # Ya estaba: sumamos a la cantidad (posición 1 de la lista).
            self.__lineas[producto.nombre][1] += cantidad
        else:
            self.__lineas[producto.nombre] = [producto, cantidad]
        return f"Añadido: {cantidad} x {producto.nombre}"

    def agregar_varios(self, *pares):
        """*args: cada argumento es una tupla (producto, cantidad).
        Devuelve una lista con el mensaje de cada intento."""
        mensajes = []
        for producto, cantidad in pares:   # desempaquetamos cada tupla
            mensajes.append(self.agregar(producto, cantidad))
        return mensajes

    def subtotal(self):
        """Suma de precio_con_iva * cantidad con una comprensión."""
        # .values() da cada [producto, cantidad]; lo desempaquetamos.
        return round(sum([prod.precio_con_iva() * cant
                          for prod, cant in self.__lineas.values()]), 2)

    def descuento(self, codigo):
        """Devuelve (porcentaje, mensaje) según el código. match-case con guardas."""
        total = self.subtotal()
        match codigo:
            case None:
                return 0, "Sin código"
            case "VERANO":
                return 10, "Código VERANO: 10 %"
            case "VIP" if total >= 100:       # guarda: solo si gasta 100 o más
                return 20, "Código VIP: 20 %"
            case "VIP":                        # llega aquí si la guarda falló
                return 0, "VIP requiere 100 € mínimo"
            case _:
                return 0, f"Código '{codigo}' no válido"

    def ticket(self, codigo=None):
        """Devuelve el ticket completo como string (NO lo imprime)."""
        lineas_texto = [f"TICKET de {self.cliente}"]
        # .items() da (clave, valor); el valor es [producto, cantidad].
        # enumerate añade el número delante: (1, ('Ratón', [prod, 2])).
        # Por eso desempaquetamos con paréntesis: num, (nombre, linea).
        # Formato: :<10 alinea a la izquierda en 10 huecos; :>7.2f a la
        # derecha en 7 huecos con 2 decimales (solo estética).
        for num, (nombre, linea) in enumerate(self.__lineas.items(), start=1):
            prod, cant = linea      # linea es la lista [producto, cantidad]
            importe = prod.precio_con_iva() * cant
            lineas_texto.append(f"{num}. {nombre:<10} {cant} x "
                                f"{prod.precio_con_iva():>7.2f} = {importe:>7.2f} €")
```

```python
        subtotal = self.subtotal()
        porcentaje, mensaje = self.descuento(codigo)   # desempaquetar la tupla
        rebaja = round(subtotal * porcentaje / 100, 2)
        lineas_texto.append(f"Subtotal: {subtotal:.2f} €")
        lineas_texto.append(f"{mensaje} -> -{rebaja:.2f} €")
        lineas_texto.append(f"TOTAL: {subtotal - rebaja:.2f} €")
        return "\n".join(lineas_texto)   # une la lista con saltos de línea


# ------------------------------------------------------------------
# PASO 3: Funciones con lambda sobre la lista de productos.
# ------------------------------------------------------------------
def precios_con_iva(productos):
    """map + lambda: transforma cada producto en (nombre, precio con IVA)."""
    return list(map(lambda p: (p.nombre, p.precio_con_iva()), productos))


def bajo_stock(productos):
    """filter + lambda: se queda con los que tienen 5 o menos."""
    return list(filter(lambda p: p.get_stock() <= 5, productos))


def mas_caros(productos, n):
    """sorted + lambda (reverse=True) y slicing [:n] para quedarnos con n."""
    return sorted(productos, key=lambda p: p.precio, reverse=True)[:n]


def inventario_por_categoria(productos):
    """Diccionario {categoria: [nombres]}."""
    categorias = {}
    for p in productos:
        # setdefault crea la lista vacía si la clave no existe y la devuelve.
        # Equivale a (forma más básica, también válida):
        #     if p.categoria not in categorias:
        #         categorias[p.categoria] = []
        #     categorias[p.categoria].append(p.nombre)
        categorias.setdefault(p.categoria, []).append(p.nombre)
    return categorias


# ------------------------------------------------------------------
# PASO 4: Bloque principal.
# ------------------------------------------------------------------
if __name__ == "__main__":
    inventario = [
        Producto("Portátil", 650.0, "informática", 4),
        Producto("Ratón", 12.5, "informática", 30),
        Producto("Teclado", 25.0, "informática", 3),
        Producto("Silla", 89.9, "mobiliario", 0),
        Producto("Lámpara", 19.99, "mobiliario", 12),
    ]

    print("--- Inventario ---")
    for num, p in enumerate(inventario, start=1):
        print(f"{num}. {p} | {p.ver_stock()}")

    print("\nPrecios con IVA (map):", precios_con_iva(inventario))
    print("Bajo stock (filter):", [p.nombre for p in bajo_stock(inventario)])
    print("Los 2 más caros (sorted):", mas_caros(inventario, 2))

    print("\n--- Por categoría (.items()) ---")
    for categoria, nombres in inventario_por_categoria(inventario).items():
        print(f"{categoria}: {', '.join(nombres)}")

    portatil, raton, teclado, silla, lampara = inventario

    print("\n--- Carrito de Ana ---")
    carrito_ana = Carrito("Ana")
    for mensaje in carrito_ana.agregar_varios((portatil, 1), (raton, 2),
                                              (silla, 1), (raton, 15)):
        print(mensaje)
    print(carrito_ana.ticket("VIP"))

    print("\n--- Carrito de Luis ---")
    carrito_luis = Carrito("Luis")
    print(carrito_luis.agregar(teclado, 2))
    print(carrito_luis.agregar(teclado, 2))     # solo queda 1
    print(carrito_luis.agregar(lampara))        # cantidad por defecto = 1
    print(carrito_luis.ticket("VIP"))

    print("\n--- Stock tras las compras ---")
    for p in inventario:
        print(p.ver_stock())

# ------------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# --- Inventario ---
# 1. Portátil (informática) - 786.50 € IVA incl. | Portátil: 4 uds (stock bajo)
# 2. Ratón (informática) - 15.12 € IVA incl. | Ratón: 30 uds (disponible)
# 3. Teclado (informática) - 30.25 € IVA incl. | Teclado: 3 uds (stock bajo)
# 4. Silla (mobiliario) - 108.78 € IVA incl. | Silla: 0 uds (agotado)
# 5. Lámpara (mobiliario) - 24.19 € IVA incl. | Lámpara: 12 uds (disponible)
#
# Precios con IVA (map): [('Portátil', 786.5), ('Ratón', 15.12), ('Teclado', 30.25), ('Silla', 108.78), ('Lámpara', 24.19)]
# Bajo stock (filter): ['Portátil', 'Teclado', 'Silla']
# Los 2 más caros (sorted): [Producto('Portátil', 650.0, 'informática', 4), Producto('Silla', 89.9, 'mobiliario', 0)]
#
# --- Por categoría (.items()) ---
# informática: Portátil, Ratón, Teclado
# mobiliario: Silla, Lámpara
#
# --- Carrito de Ana ---
```

```
# Añadido: 1 x Portátil
# Añadido: 2 x Ratón
# Error: no hay stock suficiente de Silla.
# Error: cantidad 15 no permitida (1-10).
# TICKET de Ana
# 1. Portátil   1 x  786.50 =  786.50 €
# 2. Ratón      2 x   15.12 =   30.24 €
# Subtotal: 816.74 €
# Código VIP: 20 % -> -163.35 €
# TOTAL: 653.39 €
#
# --- Carrito de Luis ---
# Añadido: 2 x Teclado
# Error: no hay stock suficiente de Teclado.
# Añadido: 1 x Lámpara
# TICKET de Luis
# 1. Teclado    2 x   30.25 =   60.50 €
# 2. Lámpara    1 x   24.19 =   24.19 €
# Subtotal: 84.69 €
# VIP requiere 100 € mínimo -> -0.00 €
# TOTAL: 84.69 €
#
# --- Stock tras las compras ---
# Portátil: 3 uds (stock bajo)
# Ratón: 28 uds (disponible)
# Teclado: 1 uds (stock bajo)
# Silla: 0 uds (agotado)
# Lámpara: 11 uds (disponible)
# ----------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - El IVA es el mismo para todos los productos: guardarlo una vez en la
#   clase evita repetirlo en cada objeto. Si sube al 23 %, basta con
#   Producto.IVA = 0.23 y TODOS los productos lo usan al instante.
# - Si se hiciera portatil.IVA = 0.23, solo ese objeto tendría un
#   atributo de instancia nuevo que "tapa" al de la clase.
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - En un patrón de match, "case VIP:" sin comillas NO compara con un
#   texto: captura cualquier valor en una variable llamada VIP (y si no
#   es el último case, Python da SyntaxError).
# - El orden importa: "case 'VIP' if ..." tiene que ir ANTES de "case 'VIP'".
# - round(x, 2) para dinero; en el f-string :.2f solo cambia cómo se ve.
# EN JAVA SERÍA... static final double IVA = 0.21;
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   subtotal con un for y un acumulador total += ... -> pide comprensión.
#   [p for p in productos if p.get_stock() <= 5]     -> pide filter+Lambda.
# ----------------------------------------------------------------
```

05_variantes_tipo_examen/04_gestor_alumnos_notas.py

```
"""
TEMA 6 (integrador) - Variante 04: "El cuaderno del profesor"
================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Gestionar las notas de un grupo de alumnos con una clase,
    una lista de objetos y diccionarios, calculando medias y calificaciones.

    Requisitos de la clase Alumno:
    1. Atributos de clase: nota_aprobado = 5.0 y centro = "IES Las Galletas".
    2. Atributos de instancia: nombre (público) y __notas (privado): un
       DICCIONARIO {asignatura: nota}.
    3. poner_nota(asignatura, nota): solo acepta 0 <= nota <= 10
       (COMPARACIÓN ENCADENADA). Devuelve True/False (return temprano).
    4. media(): devuelve la media redondeada a 2 decimales (0.0 si no hay
       notas). Debe usar sum() y len() sobre .values().
    5. calificacion(): MATCH-CASE con GUARDAS sobre la media:
       < 5 "Insuficiente", < 6 "Suficiente", < 7 "Bien",
       < 9 "Notable", resto "Sobresaliente".
    6. suspensas(): COMPRENSIÓN DE LISTAS con las asignaturas < nota_aprobado,
       recorriendo el diccionario con .items().
    7. ver_boletin(): GETTER que devuelve un STRING multilínea con cada
       asignatura y su nota, la media y la calificación.
    8. get_notas(): devuelve una COPIA del diccionario (no el original).
    9. __str__ y __repr__.

    Funciones (con type hints):
    10. crear_alumno(nombre: str, **notas: float) -> Alumno: usa **kwargs
        para recibir las notas como argumentos con nombre.
    11. aprobados(alumnos) -> list[str]: comprensión de listas con los
        nombres de los alumnos sin ninguna suspensa.
    12. ranking(alumnos): SORTED + LAMBDA por media de mayor a menor.
    13. medias_por_asignatura(alumnos) -> dict[str, float]: diccionario
        con la media de cada asignatura de todo el grupo.
    14. nombres_en_mayusculas(alumnos): MAP + LAMBDA.
    15. dieces_necesarios(alumno, objetivo) -> int: con un WHILE, calcula
        cuántos 10 adicionales necesitaría para alcanzar la media objetivo
        (sin modificar al alumno original). Si objetivo no está entre 0 y 10,
        devuelve -1.

    Bloque principal: crea 4 alumnos con **kwargs, muestra los boletines,
    aprobados, ranking con enumerate(start=1), medias por asignatura con
    .items(), una nota inválida y el cálculo de dieces necesarios.

    Cuestión teórica: ¿Por qué get_notas() devuelve una copia? ¿Qué pasa
    si devuelve self.__notas directamente?

FORMA QUE EXIGE EL PROFESOR:
    - match-case con guardas para la calificación (NO if/elif).
    - **kwargs en crear_alumno y type hints en las funciones.
```

```
    - Comprensiones para suspensas y aprobados, sorted/map con lambda.
    - while en dieces_necesarios, .items() al recorrer diccionarios.

CONCEPTOS QUE PRACTICA:
    - Diccionario como atributo privado, copia de diccionarios
    - Funciones que devuelven objetos, listas y diccionarios

CÓMO EJECUTARLO:  python 04_gestor_alumnos_notas.py   (en Linux: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: Clase Alumno.
# ------------------------------------------------------------------
class Alumno:
    """Un alumno con sus notas por asignatura."""

    nota_aprobado = 5.0
    centro = "IES Las Galletas"

    def __init__(self, nombre: str):
        """Crea el alumno con el diccionario de notas vacío."""
        self.nombre = nombre
        self.__notas = {}   # {asignatura: nota}

    def poner_nota(self, asignatura: str, nota: float) -> bool:
        """Guarda la nota si es válida (0-10). Devuelve True si la guardó."""
        if not (0 <= nota <= 10):    # comparación encadenada
            return False             # return temprano: nota inválida
        self.__notas[asignatura] = nota  # añade o sobrescribe la clave
        return True

    def media(self) -> float:
        """Media de las notas (0.0 si no tiene ninguna)."""
        if not self.__notas:          # diccionario vacío -> False
            return 0.0                # evita dividir entre 0
        return round(sum(self.__notas.values()) / len(self.__notas), 2)

    def calificacion(self) -> str:
        """Calificación textual con match-case y guardas."""
        # Las guardas se comprueban en orden: basta con "< 6" en la segunda
        # porque si fuera < 5 ya habría entrado en la primera.
        match self.media():
            case m if m < 5:
                return "Insuficiente"
            case m if m < 6:
                return "Suficiente"
            case m if m < 7:
                return "Bien"
            case m if m < 9:
                return "Notable"
            case _:
                return "Sobresaliente"

    def suspensas(self) -> list[str]:
        """Comprensión de listas recorriendo el diccionario con .items()."""
        return [asig for asig, nota in self.__notas.items()
                if nota < Alumno.nota_aprobado]

    def get_notas(self) -> dict[str, float]:
        """Devuelve una COPIA para que desde fuera no se pueda modificar el original."""
        return self.__notas.copy()   # también valdría dict(self.__notas)

    def ver_boletin(self) -> str:
        """GETTER: devuelve el boletín como string multilínea."""
        lineas = [f"Boletín de {self.nombre} ({Alumno.centro})"]
        for asignatura, nota in self.__notas.items():
            # Ternario para marcar las suspensas
            marca = " <- suspensa" if nota < Alumno.nota_aprobado else ""
            lineas.append(f"  {asignatura:<12}{nota:>5}{marca}")
        lineas.append(f"  Media: {self.media()} - {self.calificacion()}")
        return "\n".join(lineas)

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return f"{self.nombre}: media {self.media()} ({self.calificacion()})"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"Alumno('{self.nombre}')"


# ------------------------------------------------------------------
# PASO 2: Funciones sobre la lista de alumnos (con type hints).
# ------------------------------------------------------------------
def crear_alumno(nombre: str, **notas: float) -> Alumno:
    """Crea un alumno y le pone las notas recibidas con **kwargs.

    Ejemplo: crear_alumno("Ana", Mates=7, Lengua=8)
    -> notas = {'Mates': 7, 'Lengua': 8}
    """
    alumno = Alumno(nombre)
    for asignatura, nota in notas.items():   # notas es un diccionario
        alumno.poner_nota(asignatura, nota)
    return alumno


def aprobados(alumnos: list[Alumno]) -> list[str]:
    """Nombres de los alumnos sin suspensas (lista vacía = False)."""
    return [a.nombre for a in alumnos if not a.suspensas()]
```

```python
def ranking(alumnos: list[Alumno]) -> list[Alumno]:
    """Ordena por media de mayor a menor con sorted + lambda."""
    return sorted(alumnos, key=lambda a: a.media(), reverse=True)


def medias_por_asignatura(alumnos: list[Alumno]) -> dict[str, float]:
    """Diccionario {asignatura: media del grupo}."""
    acumulado = {}   # {asignatura: [nota1, nota2, ...]}
    for alumno in alumnos:
        for asignatura, nota in alumno.get_notas().items():
            acumulado.setdefault(asignatura, []).append(nota)
    # Comprensión de DICCIONARIO: {clave: valor for ... in ...}
    return {asig: round(sum(notas) / len(notas), 2)
            for asig, notas in acumulado.items()}


def nombres_en_mayusculas(alumnos: list[Alumno]) -> list[str]:
    """map + lambda: transforma cada alumno en su nombre en mayúsculas."""
    return list(map(lambda a: a.nombre.upper(), alumnos))


def dieces_necesarios(alumno: Alumno, objetivo: float) -> int:
    """Cuántos 10 extra harían falta para llegar a la media objetivo."""
    if not (0 <= objetivo <= 10):
        return -1
    # Trabajamos con una copia de las notas (lista) para no tocar al alumno.
    notas = list(alumno.get_notas().values())
    dieces = 0
    # while: repetimos MIENTRAS no se alcance el objetivo.
    # Con objetivo 10 y alguna nota < 10 nunca se llegaría: lo limitamos a 100
    # vueltas para evitar un bucle infinito.
    while sum(notas) / len(notas) < objetivo and dieces < 100:
        notas.append(10)
        dieces += 1
    return dieces


# ---------------------------------------------------------------
# PASO 3: Bloque principal.
# ---------------------------------------------------------------
if __name__ == "__main__":
    grupo = [
        crear_alumno("Ana", Mates=7.5, Lengua=8, Python=9.5),
        crear_alumno("Luis", Mates=4, Lengua=6, Python=5.5),
        crear_alumno("Marta", Mates=9, Lengua=9.5, Python=10),
        crear_alumno("Pablo", Mates=3, Lengua=4.5, Python=6),
    ]

    print("--- Boletines ---")
    for alumno in grupo:
        print(alumno.ver_boletin())

    print("\nNota inválida (11):", grupo[0].poner_nota("Inglés", 11))
    print("Nota válida (6):", grupo[0].poner_nota("Inglés", 6))
    print(grupo[0])                                  # __str__

    print("\nAprobados:", aprobados(grupo))
    print("Suspensas de Pablo:", grupo[3].suspensas())
    print("En mayúsculas:", nombres_en_mayusculas(grupo))

    print("\n--- Ranking ---")
    for puesto, alumno in enumerate(ranking(grupo), start=1):
        print(f"{puesto}. {alumno}")

    print("\n--- Media por asignatura ---")
    for asignatura, media in medias_por_asignatura(grupo).items():
        print(f"{asignatura}: {media}")

    print("\n--- Dieces necesarios ---")
    print("Pablo para llegar a 6:", dieces_necesarios(grupo[3], 6))
    print("Marta para llegar a 9:", dieces_necesarios(grupo[2], 9))
    print("Objetivo imposible (12):", dieces_necesarios(grupo[1], 12))

    # Demostración de la copia: modificarla NO cambia al alumno.
    copia = grupo[1].get_notas()
    copia["Mates"] = 10
    print("\nCopia modificada:", copia)
    print("Original intacto:", grupo[1].get_notas())
    print("Repr del grupo:", grupo)

# ---------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# --- Boletines ---
# Boletín de Ana (IES Las Galletas)
#   Mates         7.5
#   Lengua          8
#   Python        9.5
#   Media: 8.33 - Notable
# Boletín de Luis (IES Las Galletas)
#   Mates           4 <- suspensa
#   Lengua          6
#   Python        5.5
#   Media: 5.17 - Suficiente
# Boletín de Marta (IES Las Galletas)
#   Mates           9
#   Lengua        9.5
#   Python         10
#   Media: 9.5 - Sobresaliente
# Boletín de Pablo (IES Las Galletas)
#   Mates           3 <- suspensa
#   Lengua        4.5 <- suspensa
#   Python          6
```

```
#   Media: 4.5 - Insuficiente
#
# Nota inválida (11): False
# Nota válida (6): True
# Ana: media 7.75 (Notable)
#
# Aprobados: ['Ana', 'Marta']
# Suspensas de Pablo: ['Mates', 'Lengua']
# En mayúsculas: ['ANA', 'LUIS', 'MARTA', 'PABLO']
#
# --- Ranking ---
# 1. Marta: media 9.5 (Sobresaliente)
# 2. Ana: media 7.75 (Notable)
# 3. Luis: media 5.17 (Suficiente)
# 4. Pablo: media 4.5 (Insuficiente)
#
# --- Media por asignatura ---
# Mates: 5.88
# Lengua: 7.0
# Python: 7.75
# Inglés: 6.0
#
# --- Dieces necesarios ---
# Pablo para llegar a 6: 2
# Marta para llegar a 9: 0
# Objetivo imposible (12): -1
#
# Copia modificada: {'Mates': 10, 'Lengua': 6, 'Python': 5.5}
# Original intacto: {'Mates': 4, 'Lengua': 6, 'Python': 5.5}
# Repr del grupo: [Alumno('Ana'), Alumno('Luis'), Alumno('Marta'), Alumno('Pablo')]
# ----------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - Los diccionarios son MUTABLES y se pasan por referencia: si
#   get_notas() devolviera self.__notas, quien lo recibe podría hacer
#   notas["Mates"] = 10 y cambiaría la nota "privada" del alumno sin pasar
#   por poner_nota() (que valida 0-10). La copia protege el encapsulamiento.
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - match-case: "case m if m < 5" captura la media en m; "case < 5" NO existe.
# - sum(dic) suma las CLAVES; para sumar notas hay que usar dic.values().
# - Dividir entre len() de un diccionario vacío -> ZeroDivisionError.
# - En **notas las claves son nombres válidos de variable: Mates=7 vale,
#   "Educación Física"=7 no (por eso las asignaturas son de una palabra).
# EN JAVA SERÍA... switch no admite rangos; aquí las guardas lo resuelven.
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if media < 5: ... elif media < 6: ...  -> pide match-case con guardas.
#   for + append para aprobados            -> pide comprensión de listas.
# ----------------------------------------------------------------
```

05_variantes_tipo_examen/05_cuenta_bancaria.py

```
"""
TEMA 6 (integrador) - Variante 05: "PyBank: cuentas y movimientos"
================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Implementar una cuenta bancaria con encapsulamiento estricto
    del saldo, historial de movimientos (lista de diccionarios) y
    validaciones, más funciones de análisis con lambda y comprensiones.

    Requisitos de la clase CuentaBancaria:
    1. Atributos de clase: banco = "PyBank", comision_retirada = 1.5,
       limite_operacion = 3000 y un contador num_cuentas.
    2. Atributos públicos: titular, iban. Privados: __saldo (float, por
       defecto 0) y __movimientos (lista de diccionarios
       {"tipo": ..., "importe": ..., "saldo": ...}).
       El saldo inicial, si es mayor que 0, se registra como "ingreso".
    3. Método privado __registrar(tipo, importe): añade un diccionario
       al historial.
    4. ingresar(importe): válido solo si 0 < importe <= limite_operacion
       (COMPARACIÓN ENCADENADA). Devuelve un string (return temprano si error).
    5. ingresar_varios(*importes): *args; devuelve la lista de mensajes.
    6. retirar(importe): valida el rango igual que ingresar, y que haya
       saldo para importe + comisión. Devuelve un string.
    7. transferir(destino, importe, **opciones): **kwargs con opciones
       como concepto="..." o urgente=True (urgente cobra 2 € extra).
       Devuelve un string.
    8. consultar_saldo(): GETTER que DEVUELVE un string "Saldo de X: 123.45 €".
       get_saldo(): getter que devuelve el número.
    9. get_movimientos(): devuelve una COPIA de la lista.
    10. nivel_cliente(): MATCH-CASE con GUARDAS sobre el saldo:
        < 0 imposible (no debe pasar), 0 "Sin fondos", < 1000 "Básico",
        1000 a 9999.99 "Plus", resto "Premium".
    11. __str__ y __repr__.

    Funciones de análisis:
    12. resumen_por_tipo(cuenta) -> dict: {tipo: total} recorriendo los
        movimientos; después se recorre con .items().
    13. ingresos_grandes(cuenta, minimo): FILTER + LAMBDA.
    14. importes(cuenta): MAP + LAMBDA (lista solo de importes).
    15. cuenta_mas_rica(cuentas): MAX con key=lambda.
    16. anios_para_objetivo(saldo, interes, objetivo): WHILE que calcula
        cuántos años de interés compuesto hacen falta.

    Bloque principal: 3 cuentas, todas las operaciones (correctas y
    erróneas), historial con enumerate(start=1), análisis y ranking.
    El menú interactivo (while + input) va en una función aparte.

    Cuestión teórica: ¿Por qué __saldo es privado y no hay set_saldo()?
```

```
FORMA QUE EXIGE EL PROFESOR:
    - Atributos de clase (incluido contador), privados con __, método
      privado __registrar.
    - Getter que devuelve string (consultar_saldo).
    - *args (ingresar_varios) y **kwargs (transferir).
    - Comparación encadenada, match-case con guardas, return temprano.
    - Lista de diccionarios, .items(), filter/map/max con lambda,
      comprensión, while, enumerate(start=1).

CONCEPTOS QUE PRACTICA:
    - Encapsulamiento real: el saldo solo cambia por métodos que validan
    - Dos objetos que interactúan (transferencia)

CÓMO EJECUTARLO:  python 05_cuenta_bancaria.py   (en Linux: python3 ...)
"""


# ------------------------------------------------------------------
# PASO 1: Clase CuentaBancaria.
# ------------------------------------------------------------------
class CuentaBancaria:
    """Cuenta bancaria con historial de movimientos."""

    banco = "PyBank"
    comision_retirada = 1.5
    limite_operacion = 3000
    num_cuentas = 0

    def __init__(self, titular, iban, saldo=0.0):
        """Crea la cuenta. Si hay saldo inicial se apunta como ingreso."""
        self.titular = titular
        self.iban = iban
        self.__saldo = 0.0
        self.__movimientos = []      # lista de diccionarios
        CuentaBancaria.num_cuentas += 1
        if saldo > 0:
            self.__saldo = float(saldo)
            self.__registrar("ingreso", saldo)

    def __registrar(self, tipo, importe):
        """Método PRIVADO: solo se usa dentro de la clase."""
        self.__movimientos.append({
            "tipo": tipo,
            "importe": float(importe),   # float para que todos se vean igual
            "saldo": round(self.__saldo, 2),
        })

    def ingresar(self, importe):
        """Ingresa dinero si el importe es válido."""
        if not (0 < importe <= CuentaBancaria.limite_operacion):
            return f"Error: ingreso de {importe} € no permitido."
        self.__saldo += importe
        self.__registrar("ingreso", importe)
        return f"Ingreso de {importe:.2f} € en la cuenta de {self.titular}."

    def ingresar_varios(self, *importes):
        """*args: recibe varios importes sueltos y los ingresa uno a uno."""
        # Comprensión que llama a un método: devuelve la lista de mensajes.
        return [self.ingresar(importe) for importe in importes]

    def retirar(self, importe):
        """Retira dinero cobrando comisión. Cláusulas de guarda."""
        if not (0 < importe <= CuentaBancaria.limite_operacion):
            return f"Error: retirada de {importe} € no permitida."
        total = importe + CuentaBancaria.comision_retirada
        if total > self.__saldo:
            return f"Error: saldo insuficiente para retirar {importe} € (+comisión)."
        # Restamos y registramos por separado para que el historial
        # muestre el saldo después de cada paso.
        self.__saldo -= importe
        self.__registrar("retirada", importe)
        self.__saldo -= CuentaBancaria.comision_retirada
        self.__registrar("comision", CuentaBancaria.comision_retirada)
        return f"Retirada de {importe:.2f} € (+{CuentaBancaria.comision_retirada} € de comisión)."

    def transferir(self, destino, importe, **opciones):
        """Transfiere a otra cuenta. **opciones: concepto, urgente..."""
        # opciones es un diccionario; .get(clave, valor_por_defecto)
        concepto = opciones.get("concepto", "Sin concepto")
        extra = 2 if opciones.get("urgente", False) else 0
        if not (0 < importe <= CuentaBancaria.limite_operacion):
            return f"Error: transferencia de {importe} € no permitida."
        if importe + extra > self.__saldo:
            return f"Error: {self.titular} no tiene saldo para transferir {importe} €."
        self.__saldo -= importe
        self.__registrar("transferencia_enviada", importe)
        if extra:                    # 0 cuenta como False, 2 como True
            self.__saldo -= extra
            self.__registrar("comision", extra)
        # Accedemos al otro objeto SOLO a través de sus métodos públicos:
        # destino.__saldo += importe aquí sí funcionaría (estamos dentro de
        # la misma clase), pero no quedaría registrado en SU historial;
        # recibir_transferencia() suma y registra, que es lo correcto.
        destino.recibir_transferencia(importe)
        return f"Transferencia de {importe:.2f} € a {destino.titular} ('{concepto}')."

    def recibir_transferencia(self, importe):
        """Suma una transferencia recibida y la registra."""
        self.__saldo += importe
        self.__registrar("transferencia_recibida", importe)

    def consultar_saldo(self):
```

```
        """GETTER que devuelve un string."""
        return f"Saldo de {self.titular}: {self.__saldo:.2f} €"

    def get_saldo(self):
        """Getter numérico (para ordenar, comparar...)."""
        return round(self.__saldo, 2)

    def get_movimientos(self):
        """Devuelve una copia de la lista para proteger el historial."""
        return self.__movimientos.copy()

    def nivel_cliente(self):
        """Nivel según el saldo: match-case con guardas y comparación encadenada."""
        match self.__saldo:
            case s if s < 0:
                return "ERROR: saldo negativo"
            case 0 | 0.0:                   # el | significa "o" en un patrón
                # (case 0 solo ya valdría también para 0.0, porque 0 == 0.0;
                # aquí se pone | para que veas la sintaxis)
                return "Sin fondos"
            case s if s < 1000:
                return "Básico"
            case s if 1000 <= s < 10000:    # comparación encadenada
                return "Plus"
            case _:
                return "Premium"

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return (f"[{CuentaBancaria.banco}] {self.titular} ({self.iban}) - "
                f"{self.__saldo:.2f} € - {self.nivel_cliente()}")

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"CuentaBancaria('{self.titular}', '{self.iban}', {self.get_saldo()})"


# ------------------------------------------------------------------
# PASO 2: Funciones de análisis.
# ------------------------------------------------------------------
def resumen_por_tipo(cuenta):
    """Diccionario {tipo: total de importes de ese tipo}."""
    resumen = {}
    for mov in cuenta.get_movimientos():   # cada mov es un diccionario
        tipo = mov["tipo"]
        resumen[tipo] = round(resumen.get(tipo, 0) + mov["importe"], 2)
    return resumen


def ingresos_grandes(cuenta, minimo):
    """filter + lambda: ingresos con importe >= minimo."""
    return list(filter(lambda m: m["tipo"] == "ingreso" and m["importe"] >= minimo,
                       cuenta.get_movimientos()))


def importes(cuenta):
    """map + lambda: solo los importes del historial."""
    return list(map(lambda m: m["importe"], cuenta.get_movimientos()))


def cuenta_mas_rica(cuentas):
    """max con key=lambda: devuelve el OBJETO con más saldo."""
    return max(cuentas, key=lambda c: c.get_saldo())


def anios_para_objetivo(saldo, interes, objetivo):
    """while: años de interés compuesto hasta llegar al objetivo.
    interes en tanto por uno (0.05 = 5 %)."""
    if saldo <= 0 or interes <= 0:
        return -1     # nunca llegaría: evitamos el bucle infinito
    anios = 0
    while saldo < objetivo:
        saldo *= 1 + interes
        anios += 1
    return anios


# ------------------------------------------------------------------
# PASO 3 (opcional): cajero interactivo con while + input.
# Descomenta la llamada al final del main para probarlo.
# ------------------------------------------------------------------
def cajero_interactivo(cuenta):
    """Menú repetitivo con while True y break."""
    while True:
        print(f"\n{cuenta.consultar_saldo()}")
        print("1) Ingresar  2) Retirar  3) Movimientos  0) Salir")
        opcion = input("Opción: ")
        match opcion:
            case "1":
                print(cuenta.ingresar(float(input("Importe: "))))
            case "2":
                print(cuenta.retirar(float(input("Importe: "))))
            case "3":
                for n, mov in enumerate(cuenta.get_movimientos(), start=1):
                    print(n, mov)
            case "0":
                print("Gracias por usar PyBank.")
                break                 # sale del while True
            case _:
                print("Opción incorrecta.")
```

```python
# ------------------------------------------------------------------
# PASO 4: Bloque principal.
# ------------------------------------------------------------------
if __name__ == "__main__":
    ana = CuentaBancaria("Ana", "ES01", 500)
    luis = CuentaBancaria("Luis", "ES02")
    marta = CuentaBancaria("Marta", "ES03", 12000)
    cuentas = [ana, luis, marta]

    print("--- Cuentas abiertas ---")
    for num, cuenta in enumerate(cuentas, start=1):
        print(f"{num}. {cuenta}")

    print("\n--- Operaciones ---")
    for mensaje in ana.ingresar_varios(200, 1500, -50, 5000):
        print(mensaje)
    print(ana.retirar(100))
    print(luis.retirar(10))                    # saldo 0 -> error
    print(ana.transferir(luis, 300, concepto="Cena", urgente=True))
    print(luis.transferir(marta, 1000))       # sin saldo suficiente
    print(marta.transferir(luis, 2500, concepto="Préstamo"))

    print("\n--- Saldos (getter que devuelve string) ---")
    for cuenta in cuentas:
        print(cuenta.consultar_saldo(), "|", cuenta.nivel_cliente())

    print("\n--- Historial de Ana ---")
    for num, mov in enumerate(ana.get_movimientos(), start=1):
        print(f"{num}. {mov['tipo']:<22} {mov['importe']:>8.2f} € -> saldo {mov['saldo']:.2f}")

    print("\n--- Resumen por tipo (Ana) ---")
    for tipo, total in resumen_por_tipo(ana).items():
        print(f"{tipo}: {total}")

    print("\nIngresos >= 500 de Ana:", ingresos_grandes(ana, 500))
    print("Importes de Luis:", importes(luis))
    print("Total movido por Ana (comprensión):",
          sum([m["importe"] for m in ana.get_movimientos()]))

    print("\nCuenta más rica:", cuenta_mas_rica(cuentas))
    ranking = sorted(cuentas, key=lambda c: c.get_saldo(), reverse=True)
    print("Ranking:", ranking)
    print("Años para que Luis llegue a 4000 € al 5 %:",
          anios_para_objetivo(luis.get_saldo(), 0.05, 4000))
    print(f"Cuentas creadas en {CuentaBancaria.banco}: {CuentaBancaria.num_cuentas}")

    # cajero_interactivo(ana)   # <- descomenta para probar el menú con input()

# ------------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# --- Cuentas abiertas ---
# 1. [PyBank] Ana (ES01) - 500.00 € - Básico
# 2. [PyBank] Luis (ES02) - 0.00 € - Sin fondos
# 3. [PyBank] Marta (ES03) - 12000.00 € - Premium
#
# --- Operaciones ---
# Ingreso de 200.00 € en la cuenta de Ana.
# Ingreso de 1500.00 € en la cuenta de Ana.
# Error: ingreso de -50 € no permitido.
# Error: ingreso de 5000 € no permitido.
# Retirada de 100.00 € (+1.5 € de comisión).
# Error: saldo insuficiente para retirar 10 € (+comisión).
# Transferencia de 300.00 € a Luis ('Cena').
# Error: Luis no tiene saldo para transferir 1000 €.
# Transferencia de 2500.00 € a Luis ('Préstamo').
#
# --- Saldos (getter que devuelve string) ---
# Saldo de Ana: 1796.50 € | Plus
# Saldo de Luis: 2800.00 € | Plus
# Saldo de Marta: 9500.00 € | Plus
#
# --- Historial de Ana ---
# 1. ingreso                 500.00 € -> saldo 500.00
# 2. ingreso                 200.00 € -> saldo 700.00
# 3. ingreso                1500.00 € -> saldo 2200.00
# 4. retirada                100.00 € -> saldo 2100.00
# 5. comision                  1.50 € -> saldo 2098.50
# 6. transferencia_enviada   300.00 € -> saldo 1798.50
# 7. comision                  2.00 € -> saldo 1796.50
#
# --- Resumen por tipo (Ana) ---
# ingreso: 2200.0
# retirada: 100.0
# comision: 3.5
# transferencia_enviada: 300.0
#
# Ingresos >= 500 de Ana: [{'tipo': 'ingreso', 'importe': 500.0, 'saldo': 500.0}, {'tipo': 'ingreso', 'importe': 1500.0, 'saldo': 2200.0}]
# Importes de Luis: [300.0, 2500.0]
# Total movido por Ana (comprensión): 2603.5
#
# Cuenta más rica: [PyBank] Marta (ES03) - 9500.00 € - Plus
# Ranking: [CuentaBancaria('Marta', 'ES03', 9500.0), CuentaBancaria('Luis', 'ES02', 2800.0), CuentaBancaria('Ana', 'ES01', 1796.5)]
# Años para que Luis llegue a 4000 € al 5 %: 8
# Cuentas creadas en PyBank: 3
# ------------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - Si el saldo fuera público, cualquiera podría hacer ana.saldo = 1000000
#   sin dejar movimiento ni pasar validaciones. Con __saldo, el ÚNICO modo
#   de cambiarlo es ingresar/retirar/transferir, que validan y registran.
# - No hay set_saldo() a propósito: un banco no "pone" un saldo, lo
#   calcula a partir de los movimientos.
# ------------------------------------------------------------------
```

```
# OJO / ERRORES TÍPICOS:
# - Dentro de la clase se escribe self.__saldo; fuera da AttributeError.
# - max(cuentas) sin key da TypeError (no sabe comparar objetos).
# - opciones.get("urgente", False): si no se pasa urgente=..., vale False.
#   opciones["urgente"] daría KeyError si no se pasa.
# - Un while sin condición de salida alcanzable = bucle infinito
#   (por eso anios_para_objetivo comprueba antes saldo > 0 e interes > 0).
# EN JAVA SERÍA... private double saldo; y sin setter, solo métodos.
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   if importe > 0 and importe <= 3000      -> pide comparación encadenada.
#   transferir(destino, importe, concepto="", urgente=False) con parámetros
#   fijos -> funciona, pero el enunciado pide **kwargs.
# ------------------------------------------------------------
```

05_variantes_tipo_examen/06_videojuego_herencia.py

```
"""
TEMA 6 (integrador) - Variante 06: "Arena Pythonia" (herencia y polimorfismo)
================================================================================
ENUNCIADO (redactado al estilo del profesor, modelo Teletubbies):

    Objetivo: Diseñar una jerarquía de clases para un videojuego de combate
    por turnos: una clase base y dos clases hijas que la amplían con super().

    Requisitos de la clase base Personaje:
    1. Atributos de clase: mundo = "Pythonia" y total_personajes (contador).
    2. Atributos públicos: nombre, nivel (int, por defecto 1).
       Privados: __vida y __vida_max (100 + 10 * nivel).
    3. get_vida(): getter numérico. esta_vivo(): devuelve bool.
    4. recibir_danio(cantidad): resta vida sin bajar de 0 y DEVUELVE un string.
    5. curar(cantidad): sube vida sin pasar de __vida_max. Devuelve string.
    6. calcular_danio(): daño base = nivel * 3.
    7. atacar(objetivo): el objetivo recibe calcular_danio(). Devuelve string.
    8. ver_estado(): GETTER que DEVUELVE un string con nombre, clase, vida
       y una barra de vida hecha con "#" (una por cada 10 %).
    9. __str__ y __repr__.

    Requisitos de la clase hija Guerrero(Personaje):
    10. Atributos extra: fuerza y armadura. El constructor llama a
        super().__init__(...) para reutilizar el del padre.
    11. Sobrescribe calcular_danio(): daño base del padre (super()) + fuerza.
    12. Sobrescribe recibir_danio(): la armadura reduce el daño (mínimo 1)
        y luego llama a super().recibir_danio(...).

    Requisitos de la clase hija Mago(Personaje):
    13. Atributo extra: privado __mana (por defecto 30).
    14. Sobrescribe calcular_danio(): si tiene 10 de maná o más, lanza un
        hechizo (gasta 10, daño = nivel * 6); si no, golpe de bastón (2).
    15. __str__ de las hijas reutiliza super().__str__() y añade sus datos.

    Funciones:
    16. crear_personaje(tipo, **datos): fábrica con MATCH-CASE sobre el tipo
        ("guerrero", "mago", case _ -> None), usando **kwargs.
    17. describir(p): IF/ELIF con ISINSTANCE() para saber de qué clase es:
        Guerrero con fuerza >= 15 -> "Guerrero pesado", Guerrero -> "Guerrero",
        Mago -> "Mago", resto -> "Aventurero".
    18. combate(a, b, max_turnos=10): WHILE que alterna ataques mientras los
        dos sigan vivos y no se pase de max_turnos. Devuelve el ganador
        (o None si empatan por turnos) y una lista con el registro.
    19. clasificacion(personajes): SORTED + LAMBDA por vida (desc).
    20. vivos(personajes): COMPRENSIÓN DE LISTAS con los nombres vivos.
    21. contar_por_clase(personajes): DICCIONARIO {nombre_clase: cantidad}.

    Bloque principal: crea al menos 4 personajes (con la fábrica), muestra
    POLIMORFISMO (el mismo bucle llama a atacar() de cada clase),
    un combate completo, la clasificación con enumerate(start=1), el
    diccionario con .items(), isinstance() e issubclass().

    Cuestión teórica: ¿Por qué Guerrero NO puede escribir self.__vida
    aunque herede de Personaje? ¿Qué hace exactamente super()?

FORMA QUE EXIGE EL PROFESOR:
    - Herencia class Hija(Padre) y super().__init__ en las hijas.
    - Sobrescritura de métodos reutilizando super().metodo().
    - Atributos de clase, privados con __, getter que devuelve string.
    - match-case sobre el texto del tipo (con case _) y **kwargs en la fábrica.
    - if/elif con isinstance() para comprobar la clase de un objeto.
    - while con condición compuesta, comprensión, sorted+lambda,
      dict con .items(), enumerate(start=1), return y no print.

CONCEPTOS QUE PRACTICA:
    - Herencia, polimorfismo, name mangling en herencia, isinstance
    - type(obj).__name__ para obtener el nombre de la clase

CÓMO EJECUTARLO:  python 06_videojuego_herencia.py   (en Linux: python3 ...)
"""


# ------------------------------------------------------------
# PASO 1: Clase base (padre).
# ------------------------------------------------------------
class Personaje:
    """Clase base de todos los personajes del juego."""

    mundo = "Pythonia"
    total_personajes = 0

    def __init__(self, nombre, nivel=1):
        """Constructor del padre: lo reutilizan las hijas con super()."""
        self.nombre = nombre
```

```
        self.nivel = nivel
        self.__vida_max = 100 + 10 * nivel
        self.__vida = self.__vida_max
        Personaje.total_personajes += 1

    def get_vida(self):
        """Getter numérico de la vida."""
        return self.__vida

    def esta_vivo(self):
        """True si le queda vida."""
        return self.__vida > 0

    def recibir_danio(self, cantidad):
        """Resta vida (sin bajar de 0). Devuelve un string."""
        self.__vida = max(0, self.__vida - cantidad)
        texto = f"{self.nombre} recibe {cantidad} de daño (vida {self.__vida})"
        if not self.esta_vivo():
            texto += " y cae derrotado"
        return texto

    def curar(self, cantidad):
        """Sube vida sin pasar del máximo."""
        if not self.esta_vivo():
            return f"{self.nombre} no puede curarse: está derrotado."
        self.__vida = min(self.__vida_max, self.__vida + cantidad)
        return f"{self.nombre} se cura hasta {self.__vida}."

    def calcular_danio(self):
        """Daño base. Las hijas lo sobrescriben."""
        return self.nivel * 3

    def atacar(self, objetivo):
        """Ataca a otro personaje. Gracias al POLIMORFISMO, self.calcular_danio()
        y objetivo.recibir_danio() ejecutan la versión de la clase real."""
        danio = self.calcular_danio()
        return f"{self.nombre} ataca -> " + objetivo.recibir_danio(danio)

    def ver_estado(self):
        """GETTER que devuelve un string con barra de vida."""
        # Porcentaje de vida -> número de '#' (cada uno es un 10 %).
        bloques = round(self.__vida / self.__vida_max * 10)
        barra = "#" * bloques + "." * (10 - bloques)   # repetir strings con *
        # type(self).__name__ da "Guerrero" o "Mago" aunque estemos en el padre.
        return (f"{self.nombre:<8} {type(self).__name__:<9} "
                f"[{barra}] {self.__vida}/{self.__vida_max}")

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return f"{self.nombre} (nivel {self.nivel}) de {Personaje.mundo}"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"Personaje('{self.nombre}', {self.nivel})"


# ------------------------------------------------------------------
# PASO 2: Clases hijas. class Hija(Padre): hereda TODO lo público.
# ------------------------------------------------------------------
class Guerrero(Personaje):
    """Personaje cuerpo a cuerpo con fuerza y armadura."""

    def __init__(self, nombre, nivel=1, fuerza=10, armadura=2):
        """Llama al constructor del padre y añade sus propios atributos."""
        super().__init__(nombre, nivel)   # crea nombre, nivel, vida...
        self.fuerza = fuerza
        self.armadura = armadura

    def calcular_danio(self):
        """Sobrescribe: daño del padre + fuerza."""
        return super().calcular_danio() + self.fuerza

    def recibir_danio(self, cantidad):
        """Sobrescribe: la armadura reduce el daño y luego actúa el padre."""
        reducido = max(1, cantidad - self.armadura)
        # NO podemos hacer self.__vida -= reducido: dentro de Guerrero eso
        # sería self._Guerrero__vida (otro nombre). Delegamos en el padre.
        return super().recibir_danio(reducido)

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return f"{super().__str__()} | Guerrero F{self.fuerza} A{self.armadura}"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return (f"Guerrero('{self.nombre}', {self.nivel}, "
                f"{self.fuerza}, {self.armadura})")


class Mago(Personaje):
    """Personaje a distancia que gasta maná en hechizos."""

    coste_hechizo = 10   # atributo de clase propio de Mago

    def __init__(self, nombre, nivel=1, mana=30):
        """Constructor: guarda los datos iniciales del objeto."""
        super().__init__(nombre, nivel)
        self.__mana = mana   # privado de Mago: _Mago__mana

    def get_mana(self):
        """Getter: devuelve el maná que le queda."""
        return self.__mana
```

```python
    def calcular_danio(self):
        """Sobrescribe: hechizo si hay maná, si no golpe de bastón."""
        if self.__mana >= Mago.coste_hechizo:
            self.__mana -= Mago.coste_hechizo
            return self.nivel * 6
        return 2

    def __str__(self):
        """Texto amigable para el usuario (lo usa print)."""
        return f"{super().__str__()} | Mago maná {self.__mana}"

    def __repr__(self):
        """Texto técnico: cómo se crearía este objeto."""
        return f"Mago('{self.nombre}', {self.nivel}, {self.__mana})"


# ------------------------------------------------------------------
# PASO 3: Funciones.
# ------------------------------------------------------------------
def crear_personaje(tipo, **datos):
    """Fábrica: crea el personaje según el tipo. **datos va al constructor."""
    match tipo.lower():
        case "guerrero":
            return Guerrero(**datos)
        case "mago":
            return Mago(**datos)
        case _:
            return None


def describir(p):
    """Devuelve el tipo de aventurero usando if/elif con isinstance().

    isinstance(p, Guerrero) devuelve True si p es un Guerrero (o de una
    clase hija de Guerrero). El orden importa: el caso más concreto
    ("Guerrero pesado") va ANTES que el general ("Guerrero").
    """
    if isinstance(p, Guerrero) and p.fuerza >= 15:
        return "Guerrero pesado"
    elif isinstance(p, Guerrero):
        return "Guerrero"
    elif isinstance(p, Mago):
        return "Mago"
    else:
        return "Aventurero"
    # Si te lo pidieran con MATCH-CASE, se puede hacer con lo visto en clase
    # (match sobre un texto + guardas), usando el nombre de la clase:
    #     match type(p).__name__:
    #         case "Guerrero" if p.fuerza >= 15:
    #             return "Guerrero pesado"
    #         case "Guerrero":
    #             return "Guerrero"
    #         case "Mago":
    #             return "Mago"
    #         case _:
    #             return "Aventurero"


def combate(a, b, max_turnos=10):
    """Combate por turnos con while. Devuelve (ganador o None, registro)."""
    registro = []
    turno = 1
    atacante, defensor = a, b
    # Condición compuesta: los dos vivos Y sin pasarnos de turnos.
    while a.esta_vivo() and b.esta_vivo() and turno <= max_turnos:
        registro.append(f"T{turno}: {atacante.atacar(defensor)}")
        atacante, defensor = defensor, atacante   # intercambio de turnos
        turno += 1
    if a.esta_vivo() and b.esta_vivo():
        return None, registro          # nadie cayó: empate
    ganador = a if a.esta_vivo() else b
    return ganador, registro           # return de dos valores = tupla


def clasificacion(personajes):
    """sorted + lambda por vida, de mayor a menor."""
    return sorted(personajes, key=lambda p: p.get_vida(), reverse=True)


def vivos(personajes):
    """Comprensión de listas: nombres de los que siguen en pie."""
    return [p.nombre for p in personajes if p.esta_vivo()]


def contar_por_clase(personajes):
    """Diccionario {nombre de la clase: cuántos hay}."""
    conteo = {}
    for p in personajes:
        clase = type(p).__name__
        conteo[clase] = conteo.get(clase, 0) + 1
    return conteo


# ------------------------------------------------------------------
# PASO 4: Bloque principal.
# ------------------------------------------------------------------
if __name__ == "__main__":
    personajes = [
        crear_personaje("guerrero", nombre="Brok", nivel=3, fuerza=16, armadura=4),
        crear_personaje("mago", nombre="Elora", nivel=4, mana=40),
        crear_personaje("Guerrero", nombre="Tess", nivel=2),
```

```
        crear_personaje("mago", nombre="Fizz", nivel=1, mana=10),
        Personaje("Aldeano"),                    # objeto de la clase base
    ]
    print("Tipo desconocido ->", crear_personaje("dragón", nombre="X"))

    print(f"\n--- Personajes de {Personaje.mundo} ---")
    for num, p in enumerate(personajes, start=1):
        print(f"{num}. {p}  <{describir(p)}>")

    print("\n--- Polimorfismo: todos atacan al Aldeano ---")
    aldeano = personajes[-1]                     # último elemento
    for p in personajes[:-1]:                    # todos menos el último
        print(p.atacar(aldeano))                 # cada uno usa SU calcular_danio

    print("\n--- Combate: Brok vs Elora ---")
    brok, elora = personajes[0], personajes[1]
    ganador, registro = combate(brok, elora, max_turnos=20)
    for linea in registro:
        print(linea)
    print("Ganador:", ganador.nombre if ganador else "empate")

    print("\n--- Curaciones ---")
    print(brok.curar(50))
    print(elora.curar(50))

    print("\n--- Clasificación por vida ---")
    for puesto, p in enumerate(clasificacion(personajes), start=1):
        print(f"{puesto}. {p.ver_estado()}")

    print("\nSiguen vivos:", vivos(personajes))
    print("\n--- Personajes por clase ---")
    for clase, cantidad in contar_por_clase(personajes).items():
        print(f"{clase}: {cantidad}")

    print("\nisinstance(brok, Personaje):", isinstance(brok, Personaje))
    print("isinstance(brok, Mago):", isinstance(brok, Mago))
    print("issubclass(Mago, Personaje):", issubclass(Mago, Personaje))
    print("Repr:", personajes)
    print("Total personajes:", Personaje.total_personajes)

# ------------------------------------------------------------------
# SALIDA ESPERADA (copiada de la ejecución real):
# Tipo desconocido -> None
#
# --- Personajes de Pythonia ---
# 1. Brok (nivel 3) de Pythonia | Guerrero F16 A4  <Guerrero pesado>
# 2. Elora (nivel 4) de Pythonia | Mago maná 40  <Mago>
# 3. Tess (nivel 2) de Pythonia | Guerrero F10 A2  <Guerrero>
# 4. Fizz (nivel 1) de Pythonia | Mago maná 10  <Mago>
# 5. Aldeano (nivel 1) de Pythonia  <Aventurero>
#
# --- Polimorfismo: todos atacan al Aldeano ---
# Brok ataca -> Aldeano recibe 25 de daño (vida 85)
# Elora ataca -> Aldeano recibe 24 de daño (vida 61)
# Tess ataca -> Aldeano recibe 16 de daño (vida 45)
# Fizz ataca -> Aldeano recibe 6 de daño (vida 39)
#
# --- Combate: Brok vs Elora ---
# T1: Brok ataca -> Elora recibe 25 de daño (vida 115)
# T2: Elora ataca -> Brok recibe 20 de daño (vida 110)
# T3: Brok ataca -> Elora recibe 25 de daño (vida 90)
# T4: Elora ataca -> Brok recibe 20 de daño (vida 90)
# T5: Brok ataca -> Elora recibe 25 de daño (vida 65)
# T6: Elora ataca -> Brok recibe 20 de daño (vida 70)
# T7: Brok ataca -> Elora recibe 25 de daño (vida 40)
# T8: Elora ataca -> Brok recibe 1 de daño (vida 69)
# T9: Brok ataca -> Elora recibe 25 de daño (vida 15)
# T10: Elora ataca -> Brok recibe 1 de daño (vida 68)
# T11: Brok ataca -> Elora recibe 25 de daño (vida 0) y cae derrotado
# Ganador: Brok
#
# --- Curaciones ---
# Brok se cura hasta 118.
# Elora no puede curarse: está derrotado.
#
# --- Clasificación por vida ---
# 1. Tess     Guerrero  [##########] 120/120
# 2. Brok     Guerrero  [#########.] 118/130
# 3. Fizz     Mago      [##########] 110/110
# 4. Aldeano  Personaje [####......] 39/110
# 5. Elora    Mago      [..........] 0/140
#
# Siguen vivos: ['Brok', 'Tess', 'Fizz', 'Aldeano']
#
# --- Personajes por clase ---
# Guerrero: 2
# Mago: 2
# Personaje: 1
#
# isinstance(brok, Personaje): True
# isinstance(brok, Mago): False
# issubclass(Mago, Personaje): True
# Repr: [Guerrero('Brok', 3, 16, 4), Mago('Elora', 4, 0), Guerrero('Tess', 2, 10, 2), Mago('Fizz', 1, 0), Personaje('Aldeano', 1)]
# Total personajes: 5
# ------------------------------------------------------------------
# RESPUESTA A LA CUESTIÓN TEÓRICA:
# - Por el name mangling: self.__vida escrito dentro de Personaje se
#   convierte en self._Personaje__vida; escrito dentro de Guerrero sería
#   self._Guerrero__vida, que NO existe. Los privados con __ no se
#   heredan "visibles": la hija usa los métodos del padre (get_vida,
#   super().recibir_danio...). En Java pasa lo mismo con private
#   (lo parecido a protected en Python sería un solo guion: _vida).
```

```
# - super() permite llamar a los métodos de la clase padre desde la hija
#   (en herencia simple, "el padre"; en herencia múltiple, la siguiente
#   clase del MRO): super().__init__(...) reutiliza el constructor del
#   padre y super().calcular_danio() reutiliza su cálculo para ampliarlo.
# ----------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - Olvidar super().__init__ en la hija: el objeto no tendrá nombre,
#   nivel ni vida -> AttributeError al usarlos.
# - isinstance(objeto, Clase): primero el OBJETO y luego la CLASE (sin
#   comillas y sin paréntesis): isinstance(p, Guerrero), no isinstance(Guerrero, p).
# - El orden de los if importa: "Guerrero pesado" antes que "Guerrero";
#   si no, todos los guerreros entrarían en el primero.
# EN JAVA SERÍA... class Guerrero extends Personaje { super(nombre, nivel); }
# ALTERNATIVA (funciona, pero NO es lo que pide el enunciado):
#   type(p) == Guerrero en vez de isinstance(p, Guerrero): funciona aquí,
#   pero no acepta clases hijas; lo habitual es isinstance().
#   Copiar el código del constructor del padre en cada hija -> pide super().
# ----------------------------------------------------------------
```

Kit de examen: plantillas, errores y checklist

06_chuleta_y_kit_examen/00_INDICE.md

06 - Chuleta y kit de examen

Archivo	Qué contiene	Forma exigida	Conceptos clave
`chuleta_python.html` / `chuleta_python.pdf`	Chuleta visual de los 6 temas, con índice enlazado: sintaxis, tablas "Si me piden... → uso...", recuadros con la forma exigida, errores típicos, métodos de str/list/dict, conversión de tipos, Java → Python y un resumen final de las formas exigidas	Recuadros amarillos "★ FORMA QUE EXIGE EL PROFESOR" en cada tema + tabla final "Resumen: formas exigidas"	Todo el temario
`plantillas_examen.py`	Esqueletos comentados `[P1]` … `[P8]`: validar números, menú `while True` + `match`, clase con privados/getters/setters/ `__str__` / `__repr__`, clase gestora de lista de objetos, contador con dict, `*args` / `**kwargs`, decorador, main	Usa las formas del profesor (`enumerate(start=1)`, `case _`, comprensiones, `key=lambda`, `@decorador`, `if __name__`)	input + isdigit / try-except, match-case, POO, listas de objetos, dict, lambda
`errores_y_traceback.md`	Cómo leer un traceback y tablas de errores típicos (con mensaje y sin mensaje) con causa y solución	—	Depuración
`checklist_examen.md`	Plan de tiempo para las 2h45, reglas de oro, checklist antes de entregar y atajos para buscar en el material	—	Estrategia de examen

Abre la chuleta en el navegador (doble clic en el `.html`) y usa `Ctrl+F` o el índice.

06_chuleta_y_kit_examen/checklist_examen.md

Checklist del examen de Python (2h45: 14:30 a 17:15)

1. Plan de tiempo

Hora	Minutos	Qué hago
14:30-14:45	15	**Leer TODO el examen** antes de escribir nada. Subrayar en cada enunciado la **forma exigida** (palabras como "usando", "sin usar", "con match", "en una línea", "retorna", "*args", "else del bucle", "privado", "decorador @"). Apuntar los puntos de cada ejercicio.
14:45-15:00	15	Crear la carpeta y los archivos (`ejercicio1.py`, ...) con el esqueleto: docstring, funciones vacías e `if __name__ == "__main__":`.
15:00-16:30	90	**Resolver empezando por lo seguro** (lo que sé hacer seguro y lo que más puntúa). Probar **cada parte al terminarla**. Si me atasco más de 10 minutos, paso al siguiente y vuelvo luego.
16:30-16:55	25	Volver a lo que dejé a medias. Algo que funcione a medias puntúa más que nada.
16:55-17:10	15	**Checklist de entrega** (abajo) archivo por archivo.
17:10-17:15	5	Guardar todo, ejecutar una última vez cada archivo y entregar.

Reglas de oro

- **Guardar a menudo**: `Ctrl+S` después de cada cambio (o activa *File > Auto Save* en VS Code al empezar).
- **Ejecutar a menudo**: no escribas 50 líneas sin probar. Escribe una función, pruébala, sigue.
- **La forma pedida manda**: el profe solo da el máximo si se resuelve como pide. Si dice `match`, no vale `if/elif`; si dice "sin `sum()`", nada de `sum()`; si dice "comprensión en una línea", nada de `for` + `append`.
- **Si no sé hacerlo de la forma pedida**: lo hago de otra forma que funcione (algo puntúa) y dejo un comentario `# TODO: el enunciado pide X`. Si sobra tiempo, vuelvo.
- **No borres código que funciona** para probar otra cosa: comenta con `Ctrl+K Ctrl+C` (descomentar: `Ctrl+K Ctrl+U`) o copia el archivo.

2. Al empezar cada ejercicio

- [ ] Copio el enunciado como docstring al principio del archivo (así no lo pierdo de vista).
- [ ] Marco la **forma exigida** en un comentario: `# FORMA EXIGIDA: match-case con case _`.
- [ ] ¿Pide **imprimir** (`print`) o **devolver** (`return`)? Lo apunto.
- [ ] ¿Pide pedir datos al usuario (`input`)? Entonces hay que **convertir** (`int()`, `float()`) y, si da tiempo, **validar**.
- [ ] Busco en mi material una plantilla parecida (ver apartado 4) y la adapto.

3. Checklist antes de entregar (por cada archivo)

Funciona

- [ ] Se ejecuta sin errores: `python archivo.py` (Linux: `python3 archivo.py`).
- [ ] Probado con un caso normal **y con casos límite**: lista vacía, 0, número negativo, texto en vez de número, opción de menú no válida, elemento que no existe al buscar o borrar, mayúsculas/minúsculas.
- [ ] La salida coincide con lo que pide el enunciado (textos y formato, p. ej. `:.2f`).

Forma exigida

- [ ] He usado exactamente la estructura que pide cada apartado (repasa lo que subrayaste).
- [ ] Nada prohibido: `sum()` donde dice "sin sum", `and` en rangos (`a <= x <= b`), `print` dentro de funciones que "retornan", `lista=[]` como parámetro por defecto.

PEP 8 y estilo

- [ ] Sangría de 4 espacios, sin tabuladores mezclados.
- [ ] Variables y funciones en `snake_case`; clases en `PascalCase`; constantes en `MAYUSCULAS`.
- [ ] Nombres que se entienden (`precio_total`, no `pt` ni `x2`).
- [ ] Espacios alrededor de `=`, `+`, `==` y después de las comas.
- [ ] 2 líneas en blanco entre funciones y clases; 1 entre métodos.
- [ ] Líneas no excesivamente largas (≈ 79 caracteres).

Documentación

- [ ] Docstring `"""..."""` al inicio del archivo, en cada función, en cada clase y en cada método.
- [ ] Comentarios `#` en las partes que no son obvias (el porqué, no lo evidente).
- [ ] Respuestas a preguntas teóricas, si las hay, como comentario o docstring en el archivo.

Estructura

- [ ] Pruebas dentro de `if __name__ == "__main__":` (nada de código suelto al final).
- [ ] Clases: `__init__` con `self`, privados con `__`, getters/setters con `return`, `__str__` devuelve str.
- [ ] Nombre del archivo correcto, sin tildes ni espacios, extensión `.py`.
- [ ] **Guardado** (en VS Code la pestaña no tiene el punto blanco de "sin guardar").

4. Cómo buscar rápido en tu material

Quiero...	Atajo / dónde
Buscar una palabra en **todos** los archivos de la carpeta	`Ctrl+Shift+F` en VS Code (p. ej. `enumerate`, `match`, `**kwargs`, `__repr__`, `lambda`)
Buscar dentro del archivo abierto	`Ctrl+F`
Abrir un archivo por su nombre	`Ctrl+P` y escribe parte del nombre
Ir a una línea (la del traceback)	`Ctrl+G`
Ir a una función del archivo	`Ctrl+Shift+O`
Buscar en la chuleta HTML/PDF	Ábrela en el navegador y `Ctrl+F`; usa el **índice** con enlaces
Plantillas listas	`plantillas_examen.py`: busca `[P1]` ... `[P8]`
Un error que no entiendo	`errores_y_traceback.md`: `Ctrl+F` con el nombre del error (`KeyError`, `TypeError`...)

Palabras clave útiles para buscar (en los `00_INDICE.md` de cada carpeta y con `Ctrl+Shift+F`):

`FORMA QUE EXIGE`, `match`, `case _`, `guarda`, `enumerate`, `range(`, `while True`, `break`, `continue`, `else:` (bucle), `comprensión`, `lambda`, `map(`, `sorted(`, `*args`, `**kwargs`, `@`, `__init__`, `__str__`, `__repr__`, `get_`, `set_`, `.items()`, `.get(`, `isdigit`, `try`.

5. Antes de salir de casa / al sentarte

- [ ] Llevar la carpeta `ejercicios/` completa (con `.py` y PDF) en el pendrive o donde se pueda.
- [ ] Comprobar en el ordenador del examen: `python --version` (o `python3 --version`) y que VS Code abre la terminal con `Ctrl+ñ`.
- [ ] Abrir la chuleta en el navegador desde el principio.
- [ ] Respirar: lee, marca la forma, empieza por lo seguro, prueba, guarda.

06_chuleta_y_kit_examen/errores_y_traceback.md

Errores y traceback: qué hacer cuando Python se queja

1. Cómo leer un traceback paso a paso

Ejemplo real:

```
Traceback (most recent call last):
  File "C:\examen\tienda.py", line 42, in <module>
    total = calcular_total(precios)
  File "C:\examen\tienda.py", line 15, in calcular_total
    suma = suma + precio
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- **Lee primero la ÚLTIMA línea.** Te da el tipo de error (`TypeError`) y el motivo (`+` entre `int` y `str`).
- **Sube a la línea de encima.** Esa es la instrucción exacta que falló (`suma = suma + precio`) y el archivo, la línea (`line 15`) y la función (`calcular_total`) donde está.
- **Sigue subiendo si hace falta.** Cada bloque `File ... line ...` es una llamada anterior. El de más arriba es el punto de partida (`<module>` = tu código principal). Así ves el camino: la línea 42 llamó a `calcular_total`, y dentro falló la línea 15.
- **Ve a esa línea en VS Code**: `Ctrl+G` y escribe el número. También puedes hacer `Ctrl+clic` sobre la ruta en la terminal.
- **Pregúntate qué valor tenía cada variable.** Si no lo sabes, pon un `print(f"{precio=} {type(precio)=}")` justo antes de la línea que falla y vuelve a ejecutar.
- **Arregla, guarda (`Ctrl+S`) y vuelve a ejecutar.** Arregla un error cada vez: muchas veces el segundo desaparece al arreglar el primero.

> **Ojo:** en `SyntaxError` e `IndentationError` el programa **no llega a ejecutarse**. La línea señalada a veces es la **siguiente** a la que tiene el fallo (p. ej. si falta un `:` o un paréntesis de cierre, mira también la línea anterior). El `^` marca dónde se dio cuenta Python.

2. Tabla de errores típicos en el examen

Error que ves	Causa más probable	Ejemplo que falla	Solución
`IndentationError: expected an indented block`	Falta sangría tras `if` / `for` / `def` ...	`if x > 0:` `print(x)`	Sangra 4 espacios el cuerpo
`IndentationError: unexpected indent` / `unindent does not match`	Sangría de más, o mezcla de tabuladores y espacios	`x = 1` suelto	Alinea con el bloque; en VS Code: "Convert Indentation to Spaces"
`SyntaxError: expected ':'`	Olvidaste los dos puntos	`for i in range(5)`	`for i in range(5):` (también `if`, `elif`, `else`, `while`, `def`, `class`, `case`, `try`, `except`)
`SyntaxError: invalid syntax` (con `=`)	`=` en vez de `==` en una condición	`if x = 5:`	`if x == 5:`
`SyntaxError: '(' was never closed`	Paréntesis/comillas sin cerrar	`print("hola"`	Cierra el paréntesis (mira la línea anterior a la señalada)
`NameError: name 'x' is not defined`	Variable/función mal escrita, sin crear todavía, o local de otra función	`print(totla)`	Revisa mayúsculas y nombre; define antes de usar; usa `return` para sacar valores de funciones
`NameError: name 'true' is not defined`	Costumbre de Java	`activo = true`	`True`, `False`, `None` con mayúscula
`TypeError: can only concatenate str (not "int") to str`	Sumar texto y número	`"Edad: " + 20`	`f"Edad: {20}"` o `"Edad: " + str(20)`
`TypeError: '>=' not supported between instances of 'str' and 'int'`	`input()` sin convertir	`edad = input()` `if edad >= 18:`	`edad = int(input())`

`TypeError: jugar() takes 0 positional arguments but 1 was given`	Olvidaste `self` en un método	`def jugar():` dentro de la clase	`def jugar(self):`
`TypeError: f() missing 1 required positional argument`	Faltan argumentos al llamar	`multiplicar(3)`	Pasa todos o pon valor por defecto
`TypeError: __str__ returned non-string (type NoneType)`	`__str__` hace `print` en lugar de `return`	`print(f"...")` dentro de `__str__`	`return f"..."`
`TypeError: 'NoneType' object is not ...`	Usas el resultado de una función sin `return` (o de `sort()`)	`lista = lista.sort()`	Añade `return`; usa `sorted(lista)` o `lista.sort()` sin asignar
`ValueError: invalid literal for int() with base 10: 'hola'`	`int()` de algo que no es entero (`"hola"`, `"3.5"`, `""`)	`int(input())` y escriben `abc`	Valida con `isdigit()` o `try/except ValueError`; para decimales `float()`
`ValueError: list.remove(x): x not in list`	Borrar algo que no está	`lista.remove(99)`	Comprueba antes: `if 99 in lista:`
`IndexError: list index out of range`	Índice ≥ `len(lista)` o lista vacía	`range(len(l) + 1)`, `while i <= len(l)`	`range(len(l))`, `i < len(l)`, o mejor `for x in lista:`
`KeyError: 'edad'`	Clave que no existe en el diccionario	`persona["edad"]`	`persona.get("edad", valor_por_defecto)` o `if "edad" in persona:`
`AttributeError: 'Producto' object has no attribute '__precio'`	Acceder a un privado desde fuera de la clase	`p.__precio`	Usa el getter `p.get_precio()`
`AttributeError: ... has no attribute 'nombre'`	En `__init__` pusiste `nombre = nombre` sin `self.` (o lo escribiste distinto)	`nombre = nombre`	`self.nombre = nombre`
`AttributeError: 'list' object has no attribute 'add'`	Método de Java	`lista.add(3)`	`lista.append(3)`
`RecursionError: maximum recursion depth exceeded`	Recursividad sin caso base	`return n * f(n - 1)` sin `if`	`if n <= 1: return 1`
`ZeroDivisionError: division by zero`	Dividir entre 0 (p. ej. media de lista vacía)	`sum(l) / len(l)` con `l = []`	Guarda: `if not l: return 0`

3. Errores que NO dan mensaje (los más traicioneros)

Síntoma	Causa	Solución
Sale `<function saludar at 0x...>` o `<bound method ...>`	Olvidaste los paréntesis al llamar	`saludar()`, `obj.metodo()`
Sale `None` donde esperabas un valor	La función no tiene `return` (o hace `print` en vez de `return`)	Añade `return resultado`
La función solo mira el primer elemento	`return` **dentro** del bucle (o un `else: return None` dentro del `for`)	Saca el `return` fuera del bucle (menos sangría)
El programa se queda colgado	`while` infinito: la variable de la condición nunca cambia	Actualiza la variable dentro del bucle o usa `break`. Para pararlo: `Ctrl+C` en la terminal
El total sale mal o 0	Acumulador inicializado **dentro** del bucle	`total = 0` antes del `for`
Sale `<map object at ...>`	Falta `list()`	`list(map(...))`
`range(1, 10)` no imprime el 10	El final de `range` no se incluye	`range(1, 11)`
`7 / 2` da `3.5`	`/` siempre da float	`//` para división entera
Todos los objetos comparten la misma lista	Lista como atributo de clase o `def f(l=[])`	Créala en `__init__` / usa `None` por defecto
`p.__precio = 0` "funciona" pero el precio no cambia	Creaste un atributo nuevo; el privado real es `_Producto__precio`	Usa el setter

4. Rutina rápida cuando algo falla

- Última línea del traceback → tipo de error.
- Línea señalada → `Ctrl+G` y número.
- Busca el error en la tabla de arriba (`Ctrl+F` con el nombre, p. ej. `KeyError`).
- Si sigue sin verse, `print(f"{variable=}")` antes de la línea.
- Arregla, guarda, ejecuta. Un error cada vez.

06_chuleta_y_kit_examen/plantillas_examen.py

```
"""
KIT DE EXAMEN - Plantillas listas para copiar y adaptar
============================================================
QUÉ ES:
    Esqueletos de las estructuras que más salen en el examen (temas 1-6).
    Copia el bloque que necesites, cambia los nombres y adapta la lógica.

ÍNDICE (busca con Ctrl+F el texto entre corchetes):
    [P1]  Leer un número validando (while + isdigit / try-except)
    [P2]  Menú con while True + match-case + opción salir
    [P3]  Clase con atributos privados + getters/setters + __str__/__repr__
    [P4]  Clase gestora con lista de objetos (añadir, listar, buscar,
          eliminar, filtrar con comprensión, ordenar con lambda)
    [P5]  Contador con diccionario
    [P6]  Función con *args y **kwargs
    [P7]  Decorador básico
    [P8]  Bloque main (demos con datos fijos, sin input())

CÓMO EJECUTARLO:  python plantillas_examen.py   (Linux/Docker: python3 ...)
    El main NO pide nada por teclado: usa datos fijos para que veas
    cómo funciona todo. Las funciones con input() están listas para usar
    en tu examen, pero aquí solo se llaman si quitas el comentario.
"""


# ============================================================
# [P1] LEER UN NÚMERO VALIDANDO
# ============================================================
# input() SIEMPRE devuelve str. Si haces int("hola") salta ValueError.
```

```python
# Dos formas de validar. Usa la que pida el enunciado; si no pide
# ninguna, cualquiera vale.

def pedir_entero(mensaje):
    """Pide un entero >= 0 hasta que sea válido (forma while + isdigit).

    isdigit() es True solo si TODOS los caracteres son dígitos:
    "12" -> True, "-3" -> False, "3.5" -> False, "" -> False.
    Por eso sirve para enteros >= 0, no para negativos ni decimales.
    """
    texto = input(mensaje).strip()          # strip quita espacios sobrantes
    while not texto.isdigit():              # mientras NO sea válido...
        print("Error: escribe un número entero (0 o mayor), sin signo ni decimales.")
        texto = input(mensaje).strip()      # ...vuelvo a pedir (¡si no, bucle infinito!)
    return int(texto)                       # aquí ya es seguro convertir


def pedir_decimal(mensaje):
    """Pide un número (puede ser negativo o decimal) con try-except.

    Intenta convertir; si falla (ValueError) avisa y repite.
    """
    while True:
        try:
            return float(input(mensaje))    # si convierte bien, sale con return
        except ValueError:
            print("Error: eso no es un número. Ejemplo válido: 3.5")


def es_entero_valido(texto):
    """Versión SIN input() para probar la validación con datos fijos."""
    return texto.strip().isdigit()


# ======================================================================
# [P2] MENÚ CON while True + match-case + SALIR
# ======================================================================
# Estructura: bucle infinito -> mostrar opciones -> leer -> match -> break
# en la opción salir. case _ recoge cualquier opción no válida.

def mostrar_menu():
    """Imprime las opciones del menú."""
    print("\n--- MENÚ ---")
    print("1. Añadir")
    print("2. Listar")
    print("3. Buscar")
    print("0. Salir")


def ejecutar_opcion(opcion):
    """Ejecuta una opción. Devuelve False si hay que salir, True si no.

    Separar esto del input() permite probar el menú sin teclado.
    """
    match opcion:
        case "1":                           # comparo con str: input da str
            print("-> Has elegido AÑADIR")
        case "2":
            print("-> Has elegido LISTAR")
        case "3":
            print("-> Has elegido BUSCAR")
        case "0":
            print("-> ¡Hasta luego!")
            return False                    # señal de salir
        case _:                             # comodín: cualquier otra cosa
            print("-> Opción no válida")
    return True


def menu_interactivo():
    """Menú real del examen: while True + match-case + break."""
    while True:
        mostrar_menu()
        opcion = input("Elige una opción: ").strip()
        if not ejecutar_opcion(opcion):
            break                           # sale del while True


# ======================================================================
# [P3] CLASE CON ATRIBUTOS PRIVADOS + GETTERS/SETTERS + __str__/__repr__
# ======================================================================

class Producto:
    """Representa un producto de una tienda.

    Atributos:
        categoria_tienda (de CLASE): compartido por todos los productos.
        nombre (público), __precio y __stock (privados).
    """

    categoria_tienda = "Alimentación"       # atributo de CLASE (fuera de __init__)

    def __init__(self, nombre, precio, stock=0):
        """Constructor: crea los atributos de INSTANCIA de cada objeto."""
        self.nombre = nombre                # público: se puede leer desde fuera
        self.__precio = precio              # privado: __ activa el name mangling
        self.__stock = stock                # (por fuera se llama _Producto__stock)

    # ---- getters: DEVUELVEN el valor (return, no print) ----
    def get_precio(self):
        """Devuelve el precio."""
        return self.__precio
```

```python
    def get_stock(self):
        """Devuelve las unidades disponibles."""
        return self.__stock

    # ---- setters: cambian el valor VALIDANDO (cláusula de guarda) ----
    def set_precio(self, nuevo_precio):
        """Cambia el precio solo si es positivo. Devuelve True si lo cambia."""
        if nuevo_precio <= 0:                # guarda: error -> salgo ya
            print(f"Error: precio no válido ({nuevo_precio})")
            return False
        self.__precio = nuevo_precio
        return True

    def vender(self, unidades):
        """Resta unidades del stock si hay suficientes."""
        if unidades > self.__stock:
            print(f"No hay stock suficiente de {self.nombre}")
            return False
        self.__stock -= unidades
        return True

    # ---- métodos mágicos ----
    def __str__(self):
        """Texto amigable para el usuario: lo usa print(objeto)."""
        return f"{self.nombre} - {self.__precio:.2f} € ({self.__stock} uds.)"

    def __repr__(self):
        """Texto técnico: cómo se creó el objeto. Lo usa print(lista)."""
        return f"Producto('{self.nombre}', {self.__precio}, {self.__stock})"


# ======================================================================
# [P4] CLASE GESTORA CON LISTA DE OBJETOS
# ======================================================================

class Tienda:
    """Gestiona una lista de objetos Producto."""

    def __init__(self, nombre):
        """Crea la tienda con una lista VACÍA propia (en __init__, no en la clase)."""
        self.nombre = nombre
        self.productos = []

    def anadir(self, producto):
        """Añade un producto si no existe otro con el mismo nombre."""
        if self.buscar(producto.nombre) is not None:
            print(f"Ya existe {producto.nombre}")
            return False
        self.productos.append(producto)
        return True

    def listar(self):
        """Muestra los productos numerados desde 1."""
        if not self.productos:               # lista vacía = False
            print("(no hay productos)")
            return
        for i, producto in enumerate(self.productos, start=1):
            print(f"{i}. {producto}")        # usa __str__

    def buscar(self, nombre):
        """Devuelve el producto con ese nombre o None si no está."""
        for producto in self.productos:
            if producto.nombre.lower() == nombre.lower():   # sin distinguir mayúsculas
                return producto
        return None                          # FUERA del for: ya miré todos

    def eliminar(self, nombre):
        """Elimina el producto con ese nombre. Devuelve True si lo borra."""
        producto = self.buscar(nombre)
        if producto is None:
            return False
        self.productos.remove(producto)
        return True

    def filtrar_baratos(self, precio_max):
        """Devuelve una lista NUEVA con los productos de precio <= precio_max."""
        return [p for p in self.productos if p.get_precio() <= precio_max]

    def ordenados_por_precio(self, descendente=False):
        """Devuelve una lista NUEVA ordenada por precio (la original no cambia)."""
        return sorted(self.productos, key=lambda p: p.get_precio(),
                      reverse=descendente)

    def valor_total(self):
        """Devuelve la suma de precio * stock de todos los productos."""
        total = 0
        for p in self.productos:             # acumulador clásico
            total += p.get_precio() * p.get_stock()
        return total

    def mas_caro(self):
        """Devuelve el producto más caro (o None si no hay)."""
        if not self.productos:
            return None
        return max(self.productos, key=lambda p: p.get_precio())


# ======================================================================
# [P5] CONTADOR CON DICCIONARIO
# ======================================================================
```

```python
def contar_palabras(texto):
    """Devuelve un dict {palabra: veces} sin distinguir mayúsculas."""
    conteo = {}
    for palabra in texto.lower().split():
        conteo[palabra] = conteo.get(palabra, 0) + 1   # get evita KeyError
    return conteo


def contar_vocales(texto):
    """Devuelve un dict con cuántas veces aparece cada vocal."""
    conteo = {v: 0 for v in "aeiou"}         # comprensión de diccionario
    for letra in texto.lower():
        if letra in conteo:                  # ojo: "é" NO cuenta como "e"
            conteo[letra] += 1
    return conteo


# ================================================================
# [P6] FUNCIÓN CON *args Y **kwargs
# ================================================================

def calcular_promedio(*args):
    """Devuelve la media de los números recibidos (args es una TUPLA)."""
    if not args:                             # guarda: evita dividir entre 0
        return 0
    return sum(args) / len(args)


def crear_perfil_usuario(nombre, apellido, **kwargs):
    """Devuelve un dict con nombre, apellido y los datos extra (kwargs es un DICT)."""
    perfil = {"nombre": nombre, "apellido": apellido}
    for clave, valor in kwargs.items():
        perfil[clave] = valor
    return perfil


def mostrar_datos(**kwargs):
    """Imprime cada dato como 'Clave: valor' con la clave capitalizada."""
    for clave, valor in kwargs.items():
        print(f"{clave.capitalize()}: {valor}")


# ================================================================
# [P7] DECORADOR BÁSICO
# ================================================================
# Un decorador es una función que recibe una función y devuelve otra
# (la "envoltura") que añade algo antes y/o después.

def anunciar_ejecucion(funcion):
    """Decorador: imprime un mensaje antes y otro después de la función."""
    def envoltura(*args, **kwargs):          # *args/**kwargs: vale para cualquier función
        """Función que sustituye a la original: añade los mensajes."""
        print("--- Iniciando función ---")
        resultado = funcion(*args, **kwargs) # ejecuto la función original
        print("--- Función terminada ---")
        return resultado                     # no pierdo lo que devuelva
    return envoltura                         # devuelvo la función SIN ()


@anunciar_ejecucion                          # = tarea_importante = anunciar_ejecucion(tarea_importante)
def tarea_importante():
    """Función de ejemplo decorada."""
    print("Procesando datos en la base de datos...")


@anunciar_ejecucion
def sumar(a, b):
    """Suma dos números (decorada, con parámetros y return)."""
    return a + b


# ================================================================
# [P8] BLOQUE MAIN: pruebas con datos fijos
# ================================================================

def main():
    """Ejecuta una demo de cada plantilla sin pedir nada por teclado."""
    print("=== [P1] Validación de números ===")
    for dato in ["25", "-3", "3.5", "hola", ""]:
        print(f"{dato!r:8} -> ¿entero válido? {es_entero_valido(dato)}")

    print("\n=== [P2] Menú (opciones simuladas) ===")
    for opcion in ["1", "2", "9", "0"]:      # simulo lo que escribiría el usuario
        print(f"Usuario escribe: {opcion}")
        if not ejecutar_opcion(opcion):
            break

    print("\n=== [P3] Clase con privados ===")
    leche = Producto("Leche", 1.15, 20)
    print(leche)                             # __str__
    print(repr(leche))                       # __repr__
    leche.set_precio(-5)                     # setter rechaza
    leche.set_precio(1.25)
    print("Precio nuevo:", leche.get_precio())
    print("Categoría (atributo de clase):", Producto.categoria_tienda)
    try:
        print(leche.__precio)                # NO se puede: es privado
    except AttributeError as error:
        print("AttributeError:", error)

    print("\n=== [P4] Tienda con lista de objetos ===")
    tienda = Tienda("Super DAW")
```

```python
    tienda.anadir(leche)
    tienda.anadir(Producto("Pan", 0.90, 50))
    tienda.anadir(Producto("Aceite", 8.50, 5))
    tienda.anadir(Producto("Café", 4.20, 12))
    tienda.anadir(Producto("pan", 1.00, 1))  # repetido -> lo rechaza
    tienda.listar()
    print("Buscar 'café':", tienda.buscar("café"))
    print("Buscar 'sal':", tienda.buscar("sal"))
    print("Baratos (<= 2 €):", tienda.filtrar_baratos(2))     # usa __repr__
    print("Ordenados por precio desc.:")
    for p in tienda.ordenados_por_precio(descendente=True):
        print("  ", p)
    print("Más caro:", tienda.mas_caro())
    print(f"Valor total del stock: {tienda.valor_total():.2f} €")
    print("¿Eliminado Pan?", tienda.eliminar("Pan"))
    print("¿Eliminado Sal?", tienda.eliminar("Sal"))
    tienda.listar()

    print("\n=== [P5] Contadores con diccionario ===")
    print(contar_palabras("el sol y el mar y EL cielo"))
    print(contar_vocales("Murciélago estudia Python"))

    print("\n=== [P6] *args y **kwargs ===")
    print("Promedio:", calcular_promedio(4, 6, 8))
    print("Promedio vacío:", calcular_promedio())
    print(crear_perfil_usuario("Ana", "Ruiz", edad=20, ciudad="Adeje"))
    mostrar_datos(nombre="Luis", edad=19, ciudad="Arona")

    print("\n=== [P7] Decorador ===")
    tarea_importante()
    print("Resultado de sumar:", sumar(2, 3))

    # Para probar las versiones con teclado, quita el # de estas líneas:
    # numero = pedir_entero("Dame un entero: ")
    # precio = pedir_decimal("Dame un precio: ")
    # menu_interactivo()


if __name__ == "__main__":
    main()

# ------------------------------------------------------------------
# OJO / ERRORES TÍPICOS:
# - return None DENTRO del for de buscar(): solo miraría el primero.
# - sort() cambia la lista y devuelve None; sorted() devuelve una nueva.
# - __str__ y __repr__ DEVUELVEN un str (return), no imprimen.
# - En el menú compara con "1" (str), porque input() devuelve str.
# - Lista de objetos en __init__ (self.productos = []), nunca como
#   atributo de clase: la compartirían todas las tiendas.
# EN JAVA SERÍA: private double precio; + getPrecio(); aquí self.__precio.
# ------------------------------------------------------------------
```